MASTER AZURE – BECOME CLOUD READY • STUDENT NOTES

Session 03 – Live

Azure Data Factory Implementation

Linked services • Datasets • Copy activity • Azure SQL DB → ADLS Gen2 (CSV & JSON)

Hands-on: build your first ADF pipelines • Azure SQL Database • ADLS Gen2

Introduction – why this session matters

- Every data platform starts with one job: getting data IN (ingestion)
- Azure Data Factory (ADF) is Azure's main tool for that job
- Today we build a real ingestion pipeline end-to-end, with zero code
- Source: Azure SQL Database (OLTP) → Target: ADLS Gen2 data lake
- Skills you learn today are used in almost every Azure Data Engineer interview and project

Recap of Session 02

- Storage account: Blob, ADLS Gen2, File shares, Tables
- Blob vs ADLS Gen2: hierarchical namespace, ACLs, big-data analytics
- Subscription → Resource group → Resource; Region = data-centre location
- OLTP (transactions) vs OLAP (analytics) and why we copy data between them
- End-to-end architecture: Sources → ADF → ADLS → Databricks → BI / AI
- We created our first Azure Data Factory resource

Learning objectives

- Explain the ADF end-to-end flow and its 5 building blocks
- Create an Azure SQL Database with the AdventureWorksLT sample data
- Configure networking / firewall so tools can connect to the database
- Create linked services for Azure SQL Database and ADLS Gen2
- Create source and sink datasets (table, CSV, JSON)
- Build, debug, monitor and publish a pipeline with a Copy activity
- Validate the result: row counts, files, formats

Session agenda

#	Topic	Type
1	Recap whiteboard: storage, environments, OLTP/OLAP, architecture	Theory
2	What is Azure Data Factory? ADF Studio tour	Theory + demo
3	ADF end-to-end flow: linked service, dataset, activity, pipeline, IR	Theory
4	Create the first cloud database: Azure SQL Database	Hands-on
5	Lab 1 – Azure SQL DB → ADLS Gen2 as CSV	Hands-on
6	Monitor, validate and publish	Hands-on
7	Lab 2 – Azure SQL DB → ADLS Gen2 as JSON	Hands-on
8	Troubleshooting, best practices, Q&A, homework	Discussion

Before we start – prerequisites

Item	Example name used in these notes	Created in
Azure subscription	Free trial / Pay-as-you-go / Visual Studio	Session 01
Resource group	rg-adf-training	Session 02
ADLS Gen2 storage account + container	stdemoadls / landing	Session 02
Azure Data Factory (V2)	adf-demo-dev	Session 02 / today
Azure SQL logical server	sql-demo-server	Today
Azure SQL Database (sample data)	sqldb-demo (AdventureWorksLT)	Today

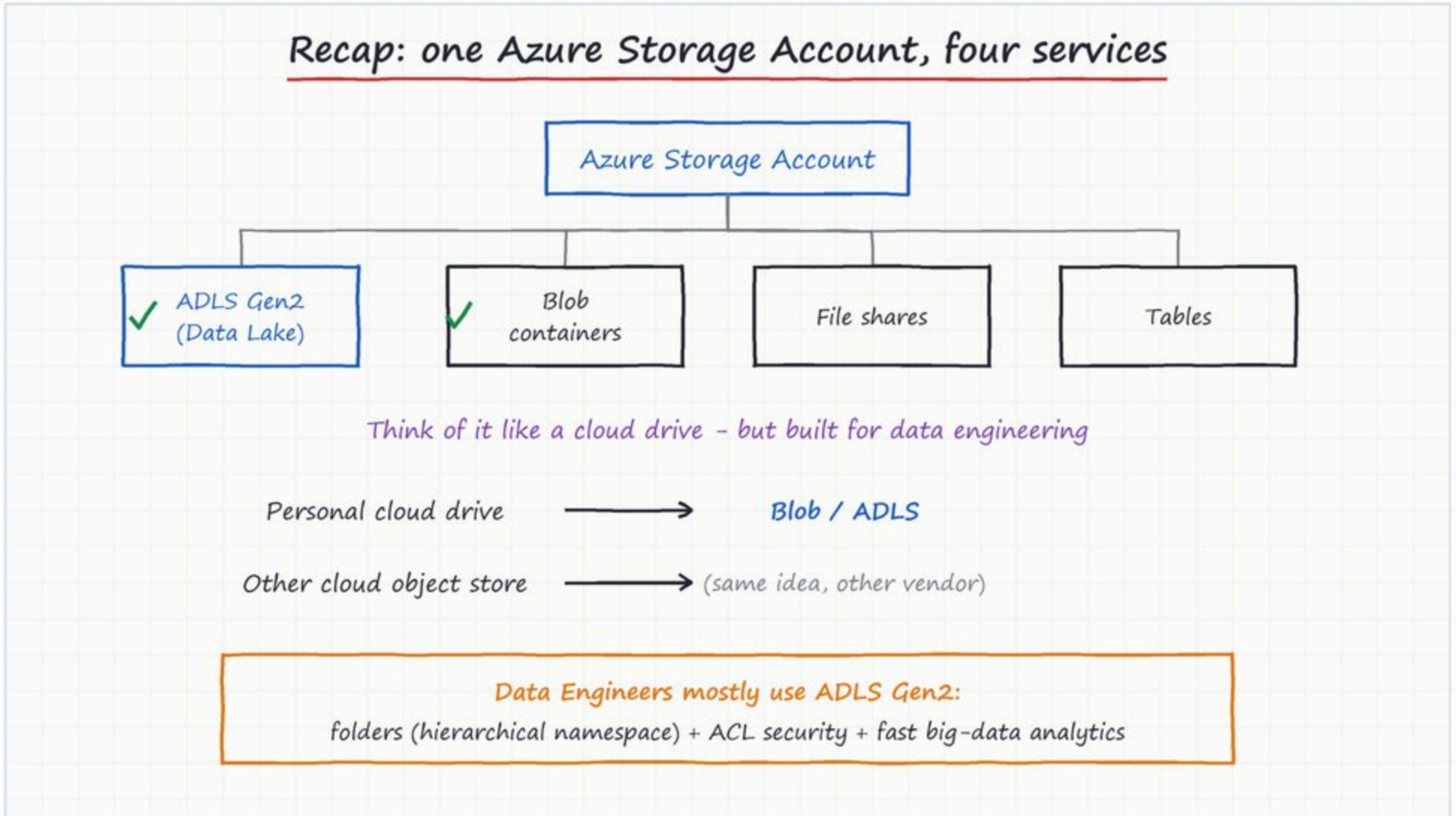
⚠ **All names in these notes are examples. Storage, ADF and SQL server names must be globally unique – add your initials or a number.**

PART 1

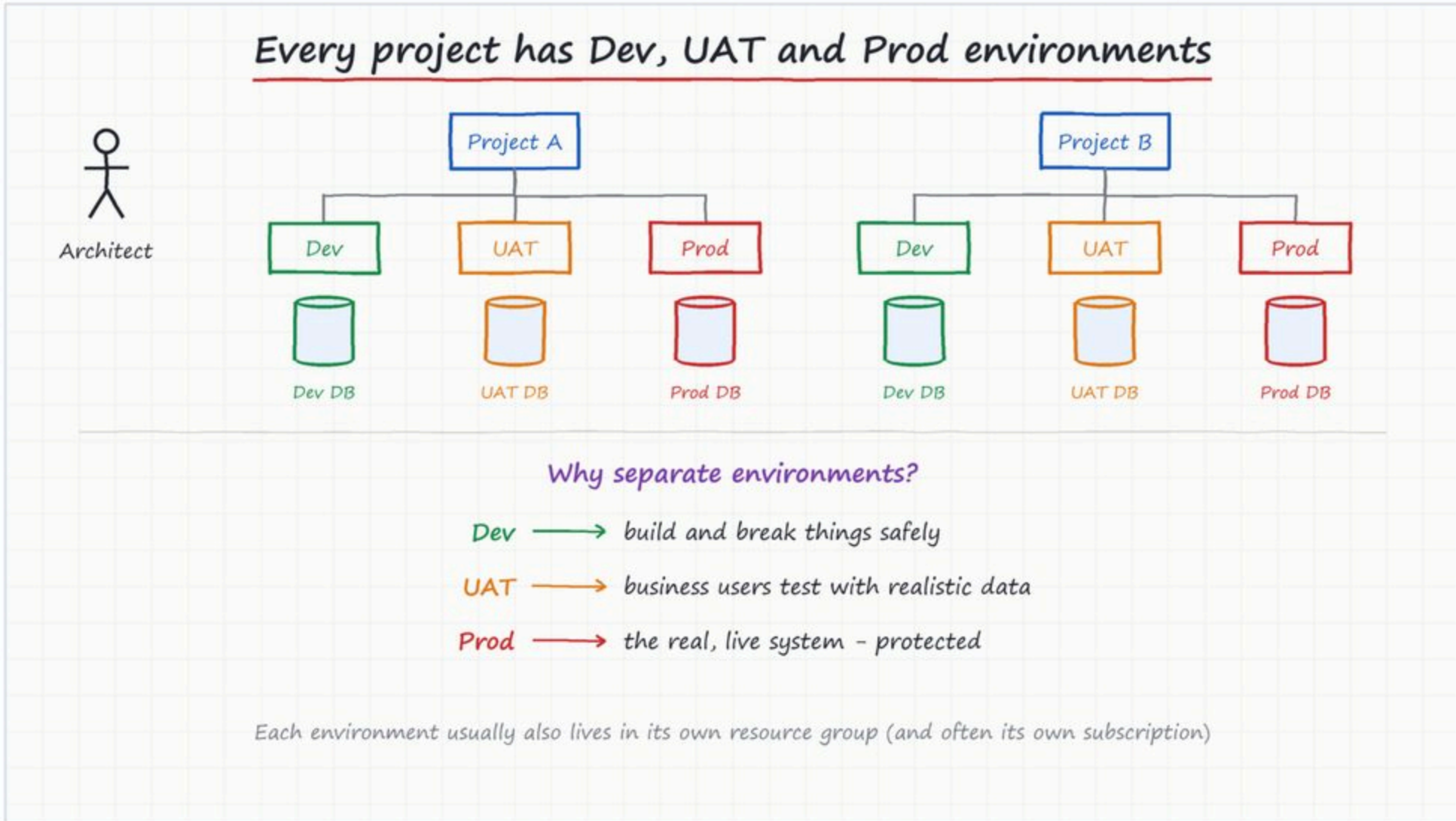
Recap on the whiteboard

Storage • Environments • OLTP vs OLAP • Architecture

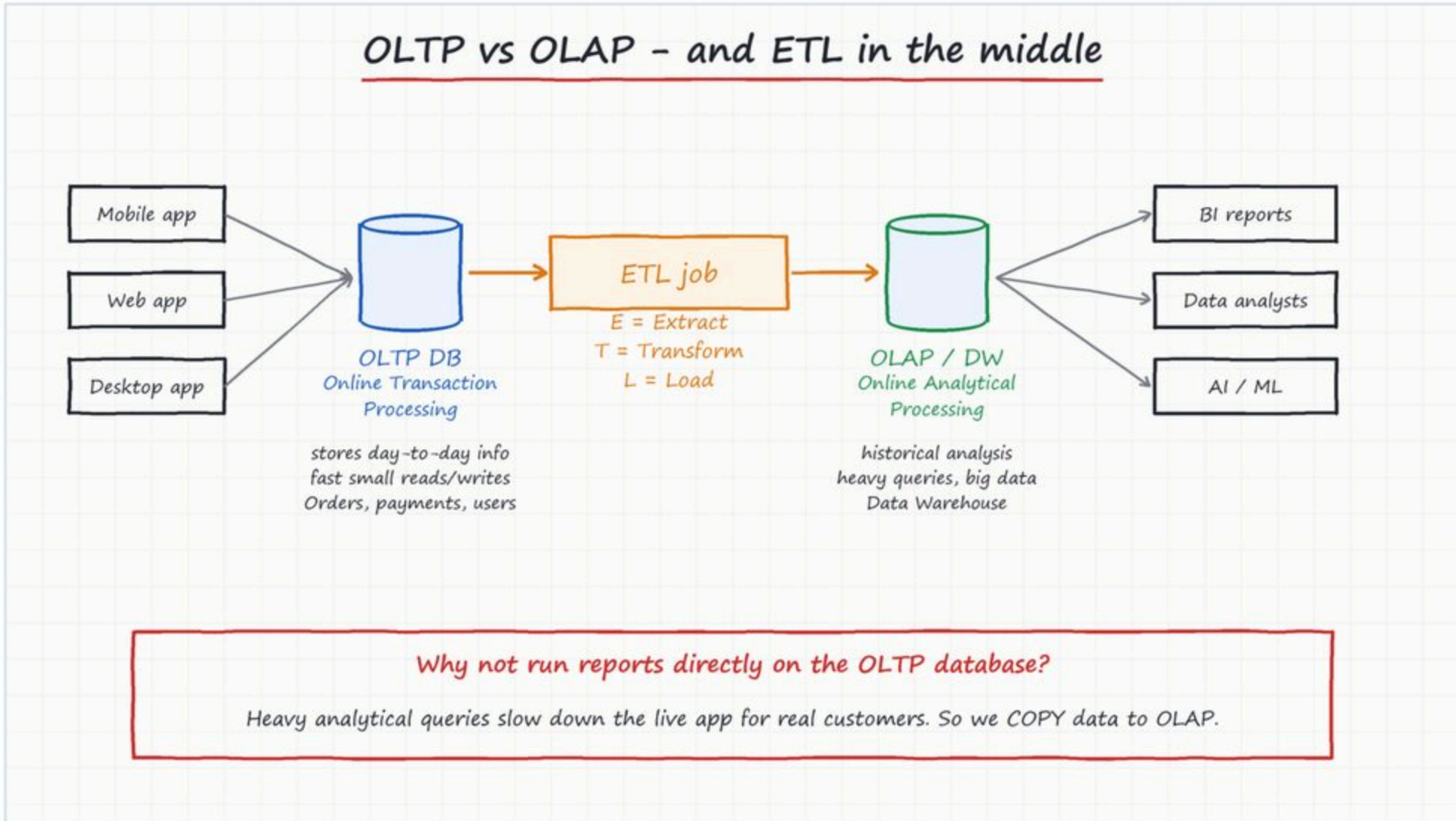
Recap: one storage account, four services



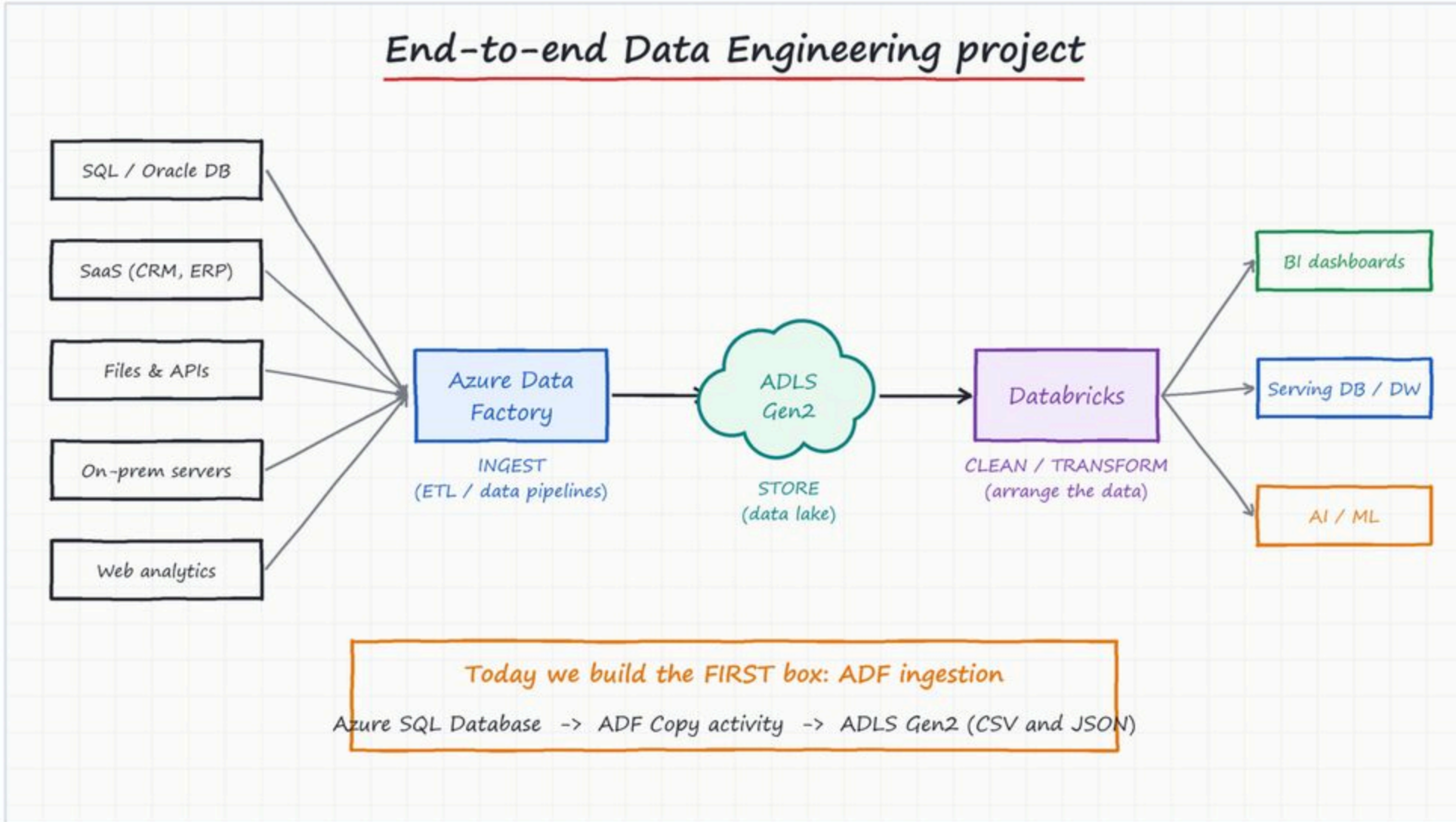
Every project has Dev, UAT and Prod



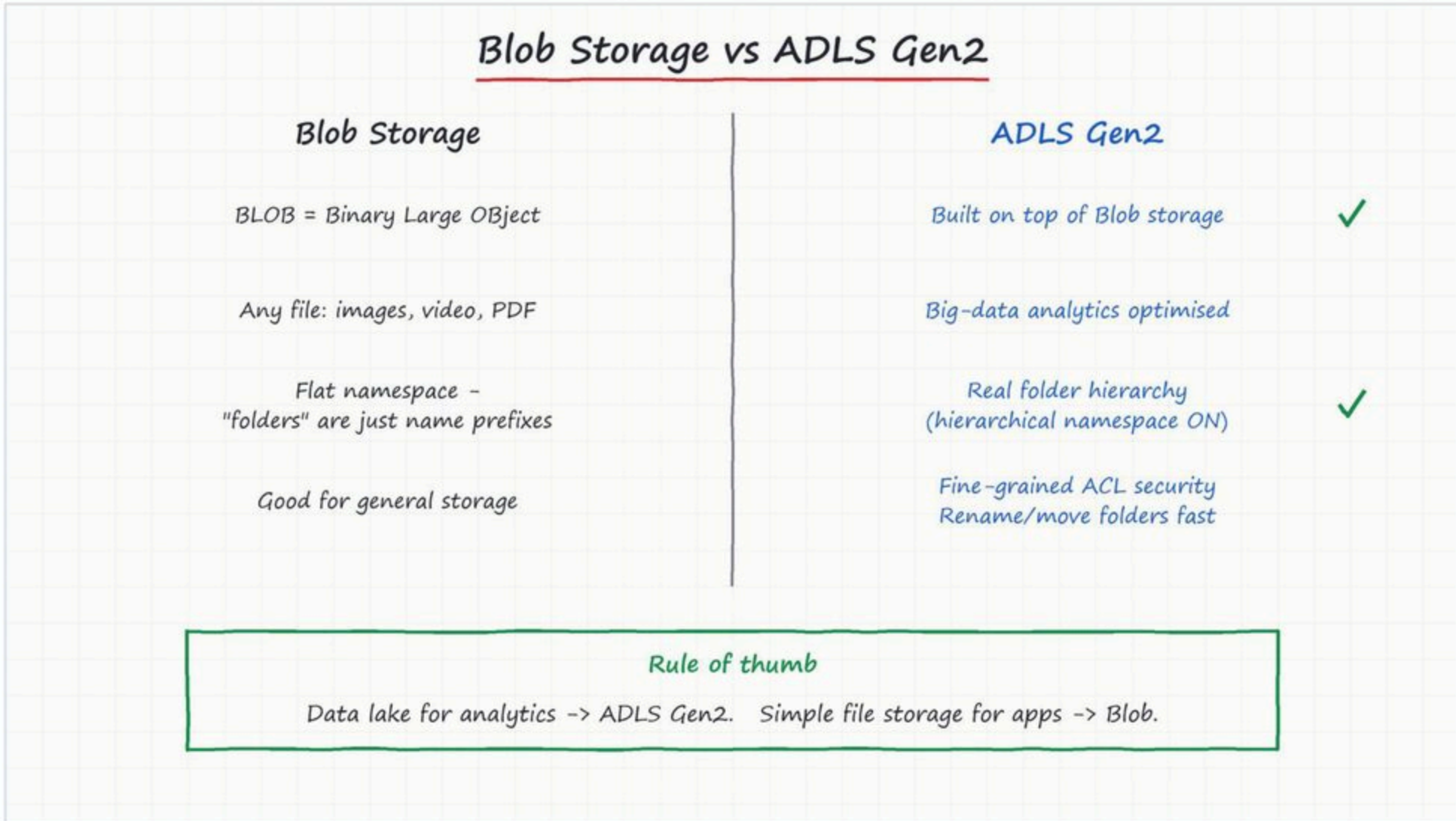
OLTP vs OLAP – and ETL in the middle



End-to-end Data Engineering project



Blob Storage vs ADLS Gen2



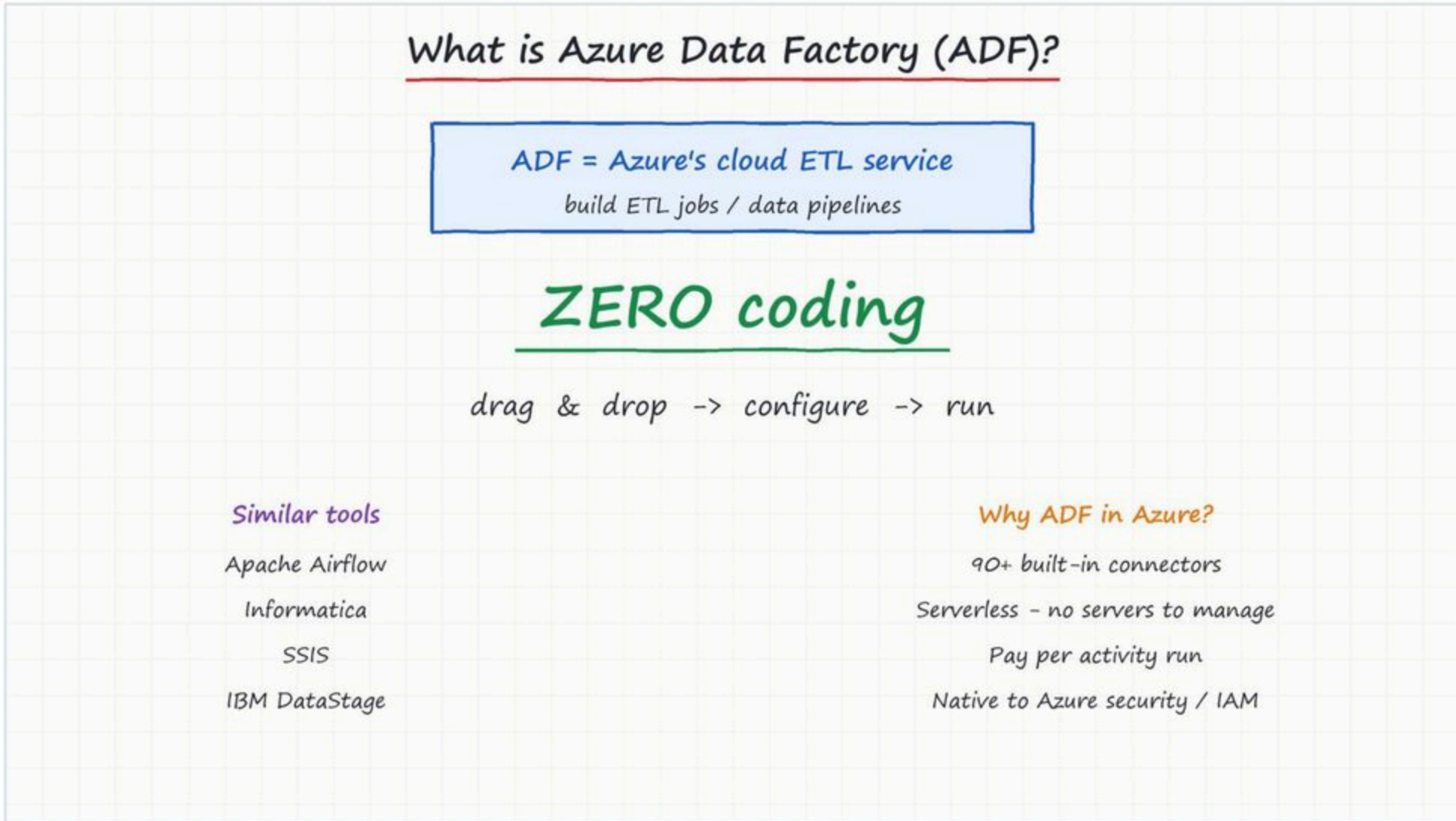
PART 2

Azure Data Factory

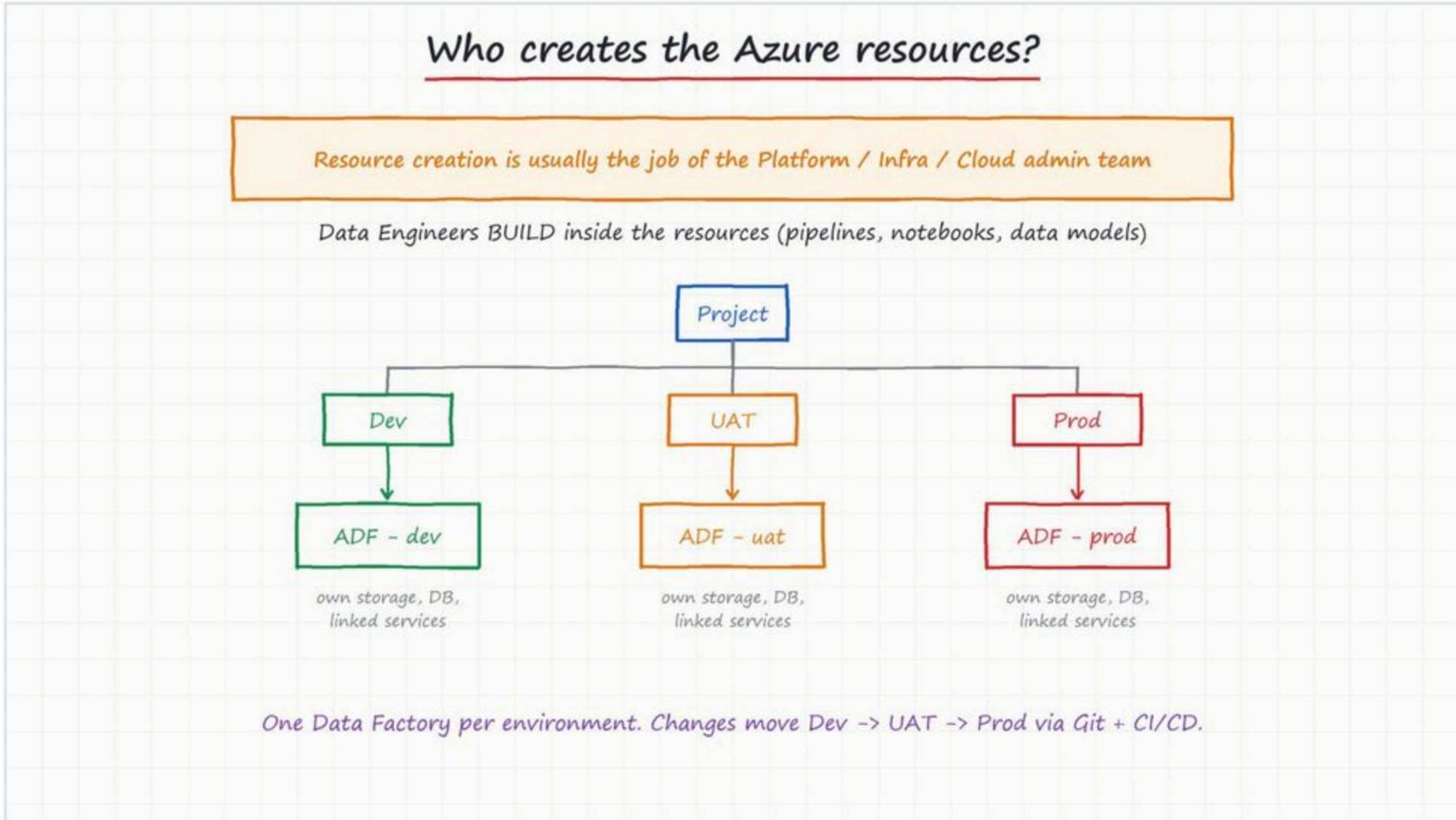
What it is • Who creates it • Create the factory • ADF Studio tour

PART 2

What is Azure Data Factory?

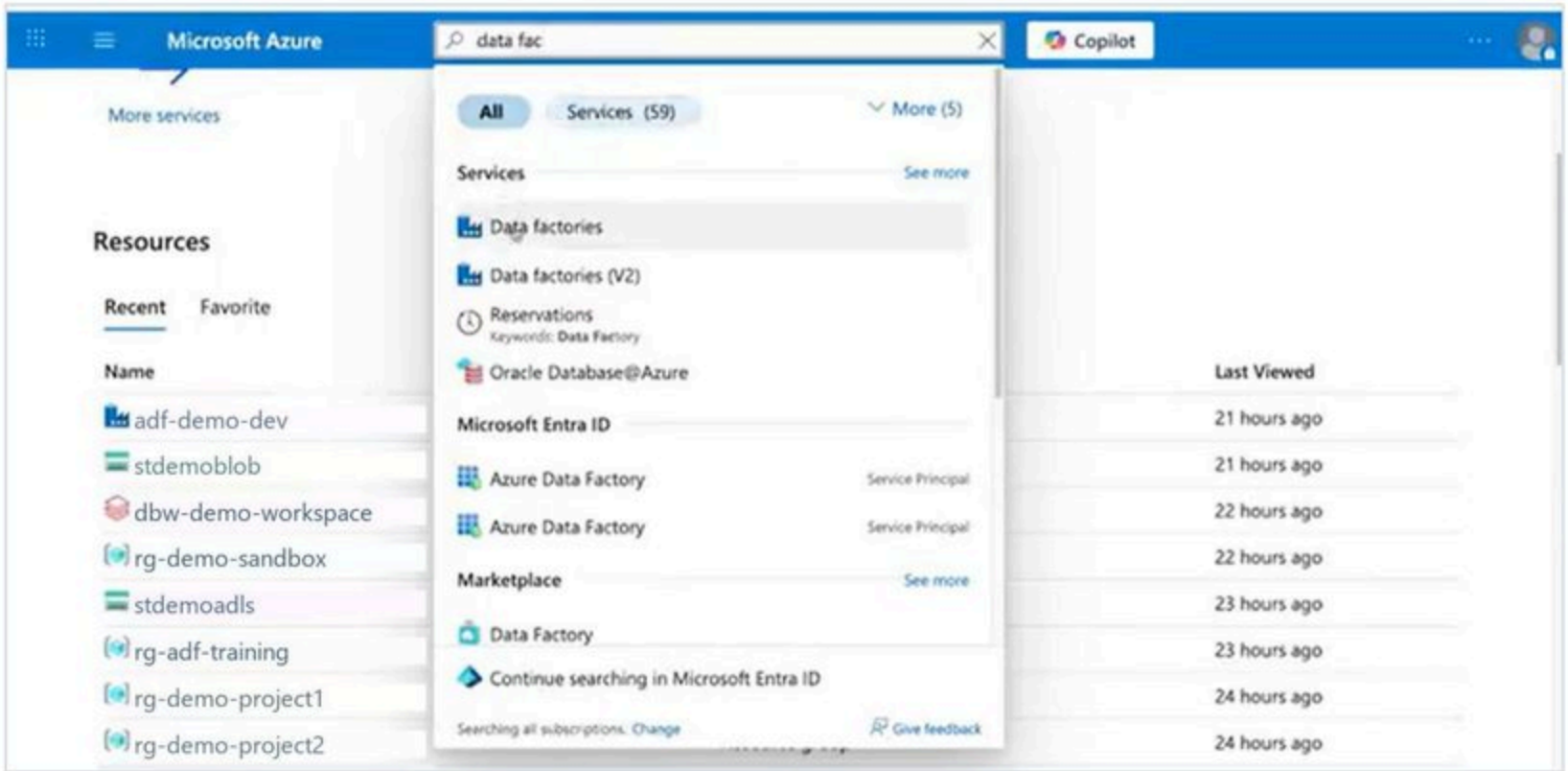


Who creates the Azure resources?



PART 2

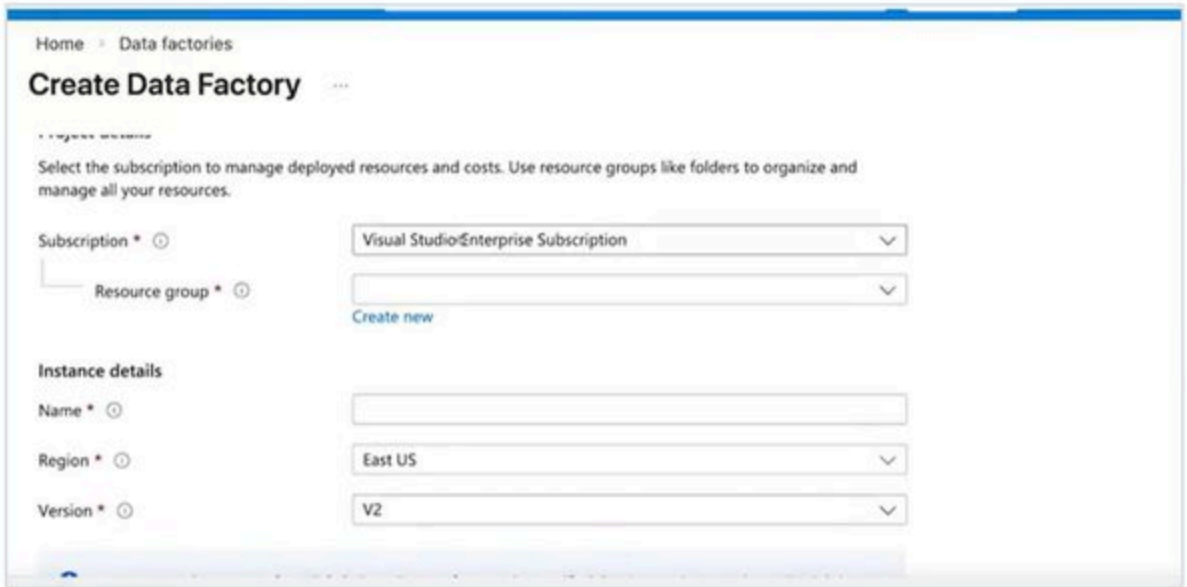
Create the Data Factory – find the service

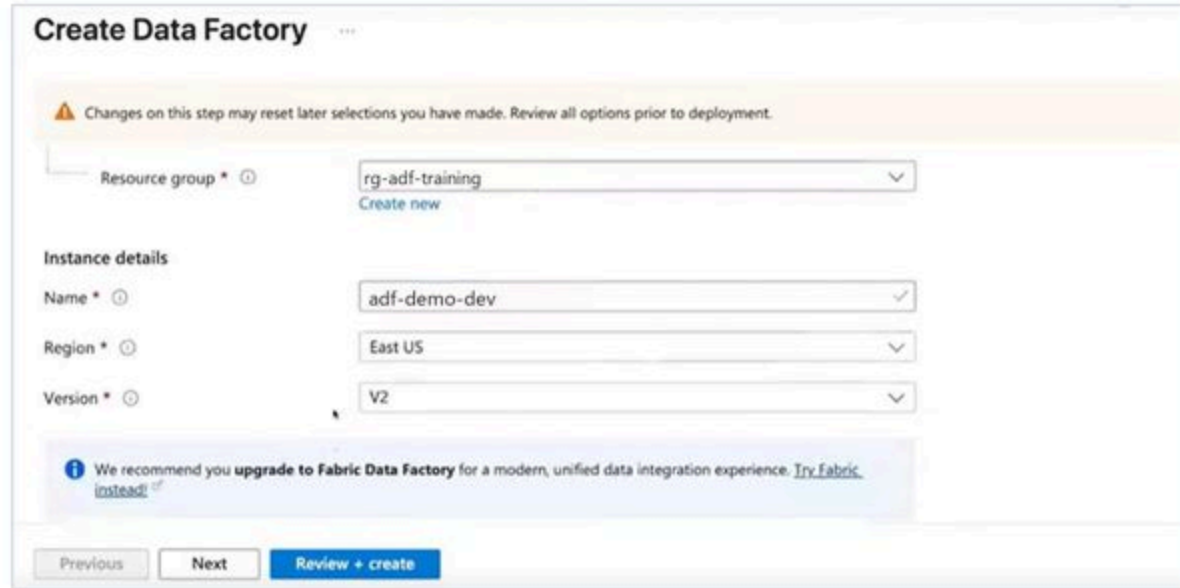


- Portal search: type “data factories”
- Select Data factories → + Create
- Recent resources list shows what you created in earlier sessions

PART 2

Create Data Factory – Basics tab

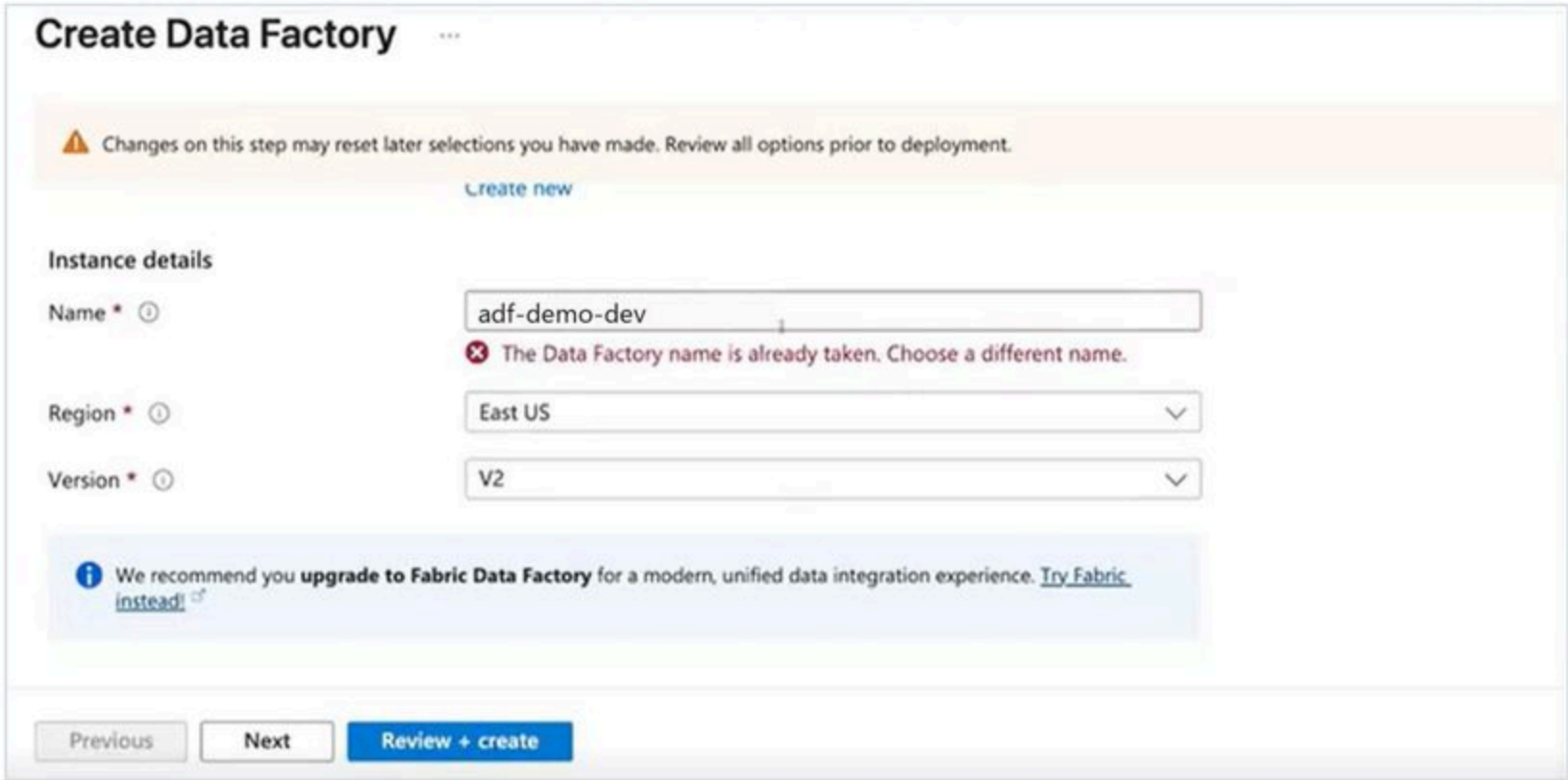




- Subscription and resource group: rg-adf-training
- Name: globally unique, e.g. adf-demo-dev
- Region: same as your other resources
- Version: V2 (always)

PART 2

Common error: name already taken

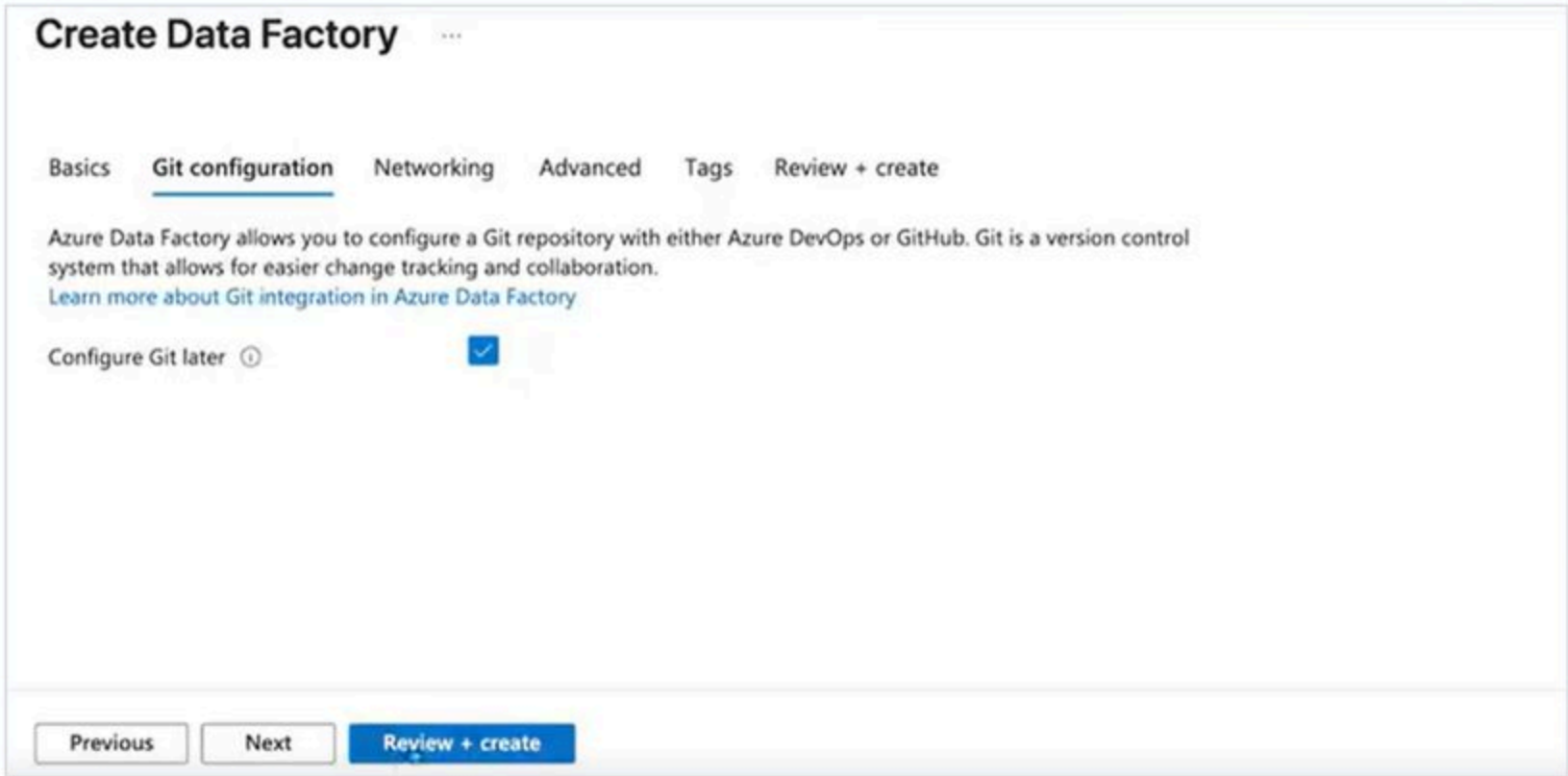


- "The Data Factory name is already taken"
- Names are global – another customer may own it
- Fix: add initials or a number, e.g. adf-demo-dev-jw01

⚠ Globally unique names: Data Factory, storage account, SQL server, Key Vault, web apps.

PART 2

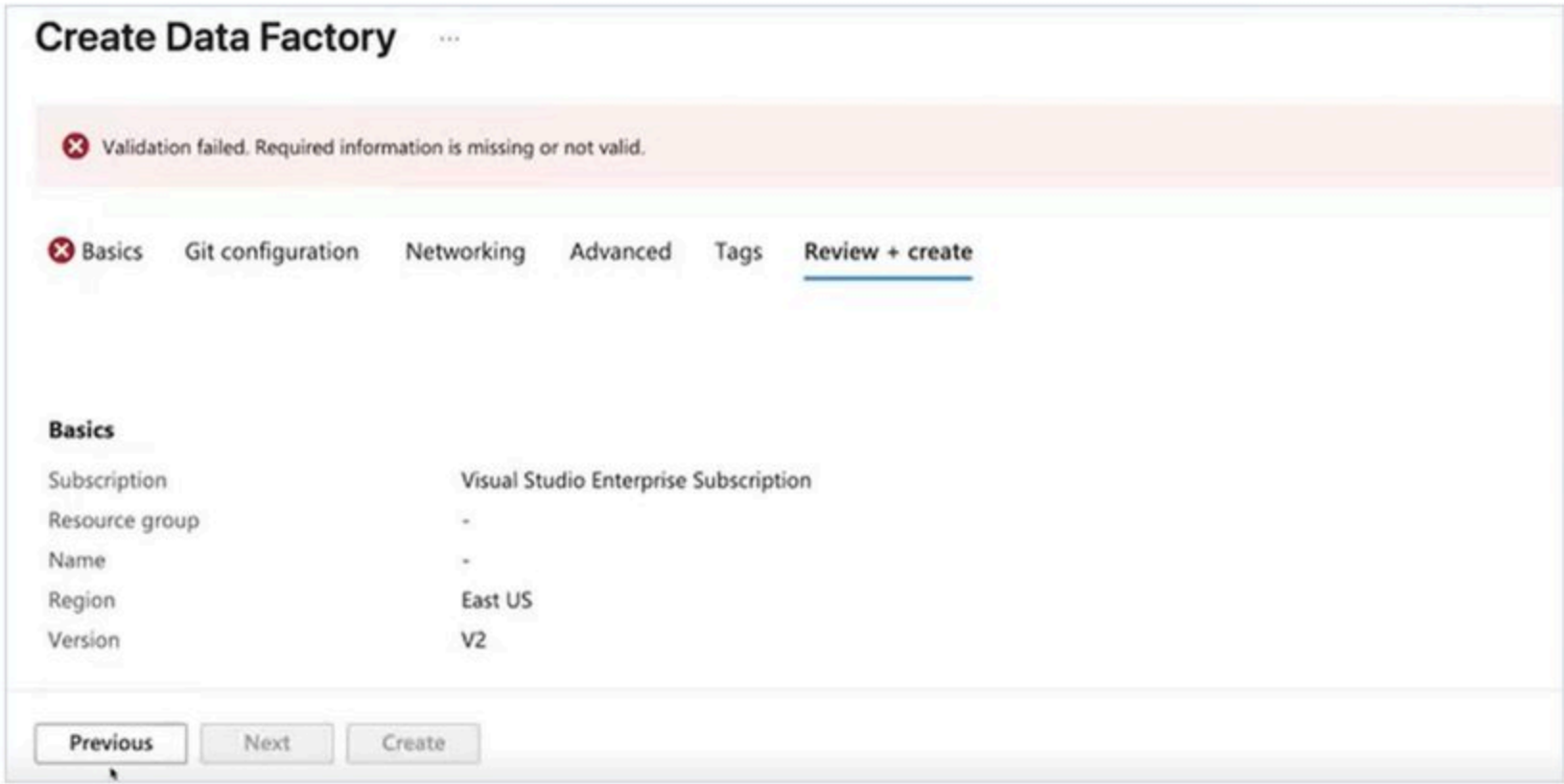
Git configuration – choose “Configure Git later”



- ADF can save its code in Azure DevOps or GitHub
- Git = version control: history, review, team work
- For today: tick Configure Git later
- We use Live mode and click Publish to save

PART 2

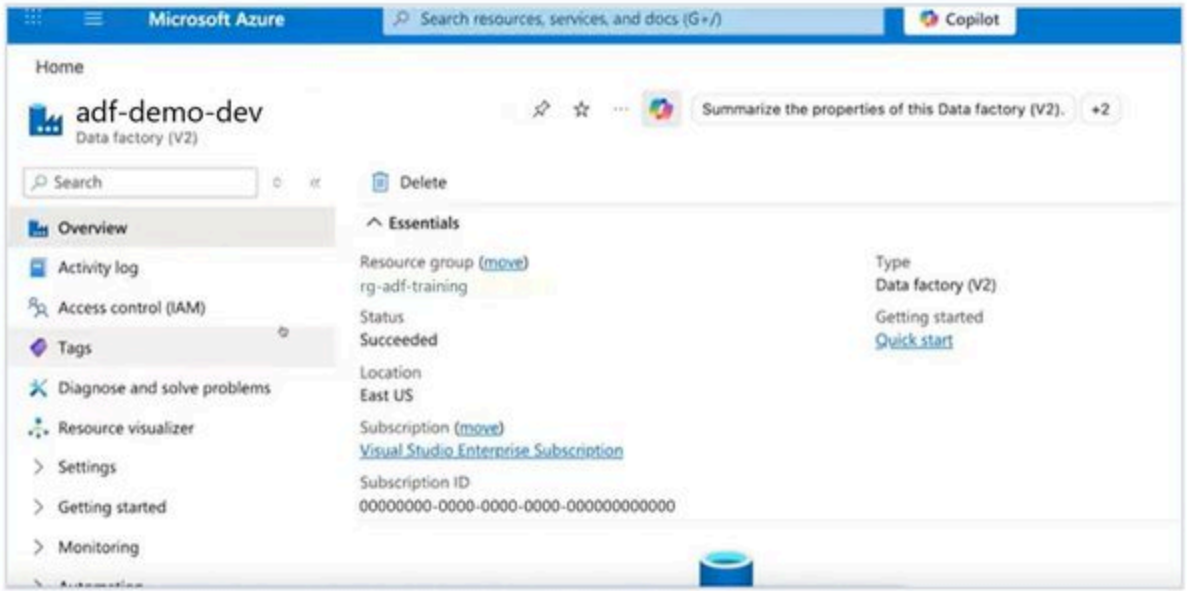
Review + create – validation

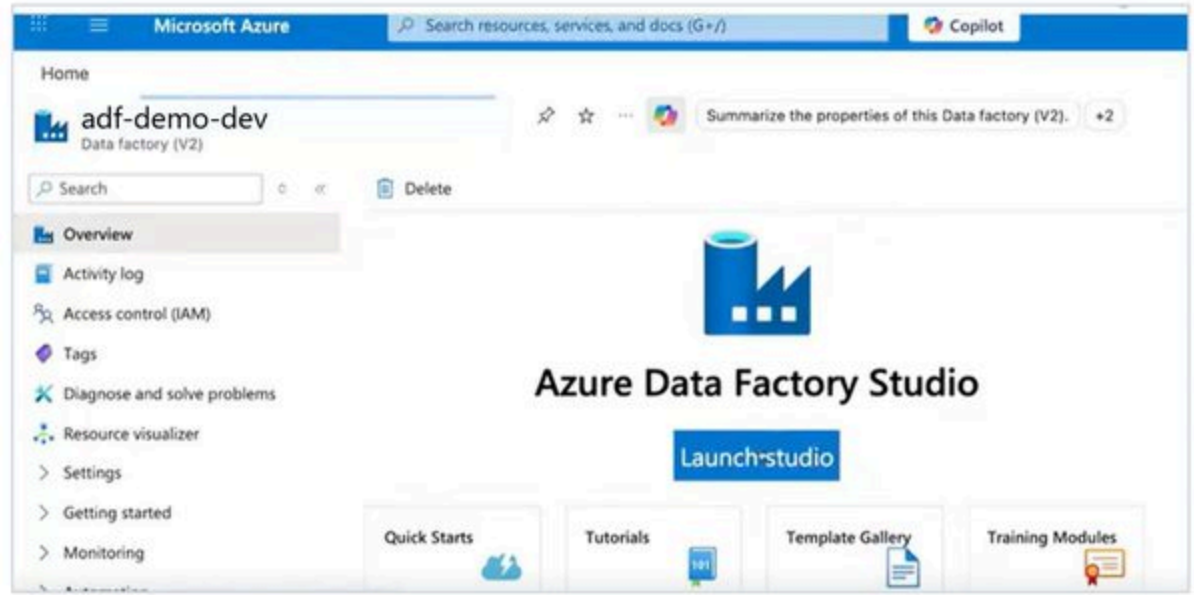


- Validation checks required fields
- "Validation failed" = a field on a tab is missing / invalid
- Fix the tab marked with a red dot, then Create

PART 2

The factory is ready – Overview page



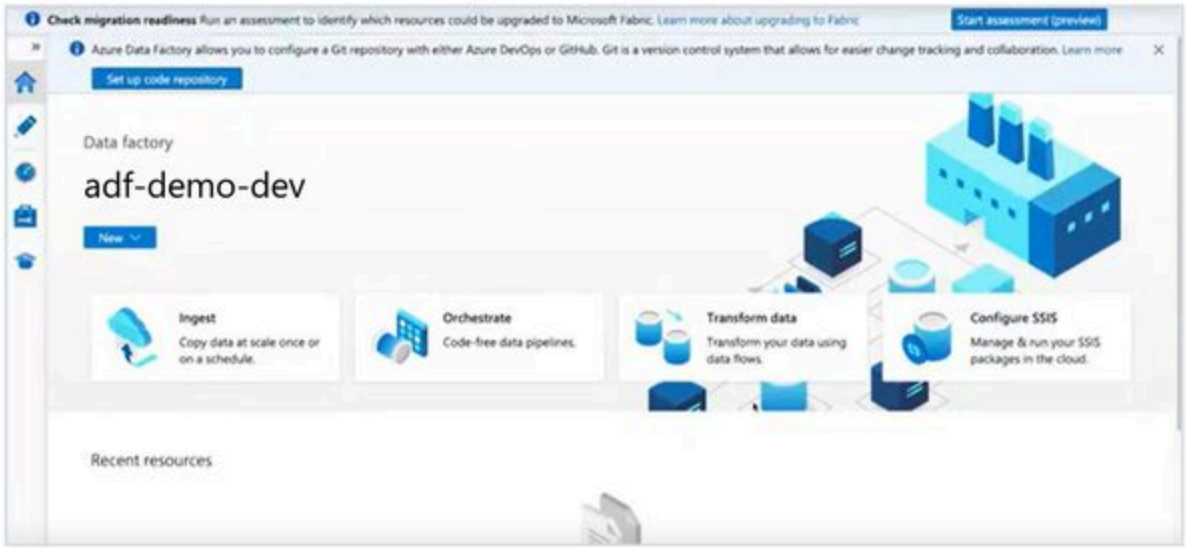


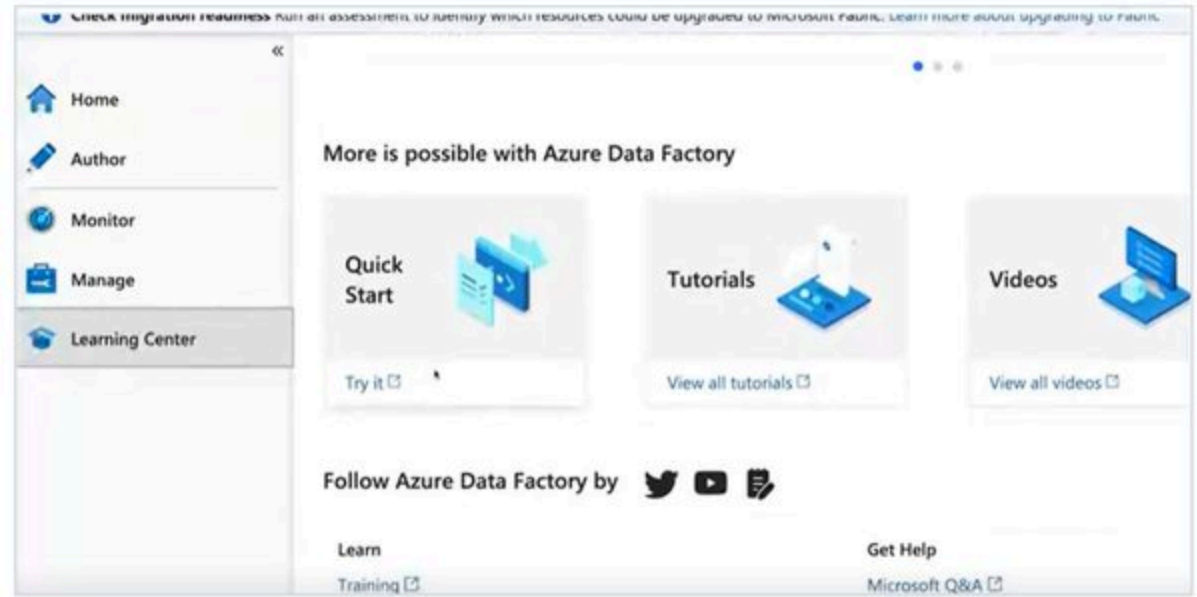
- Resource group, region, subscription
- Type: Data factory (V2)

- Launch studio opens ADF Studio (the authoring UI)

PART 2

ADF Studio – Home

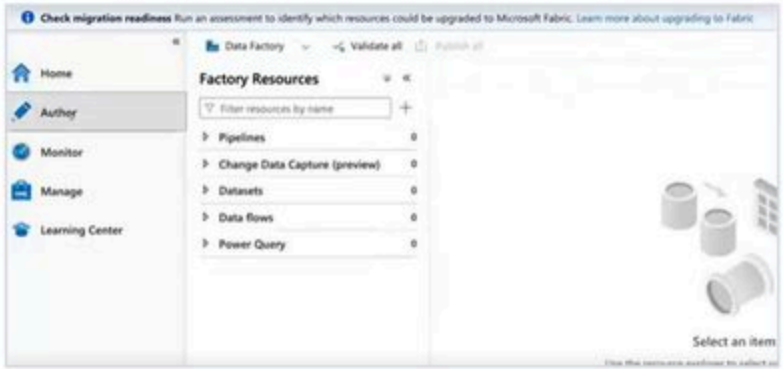


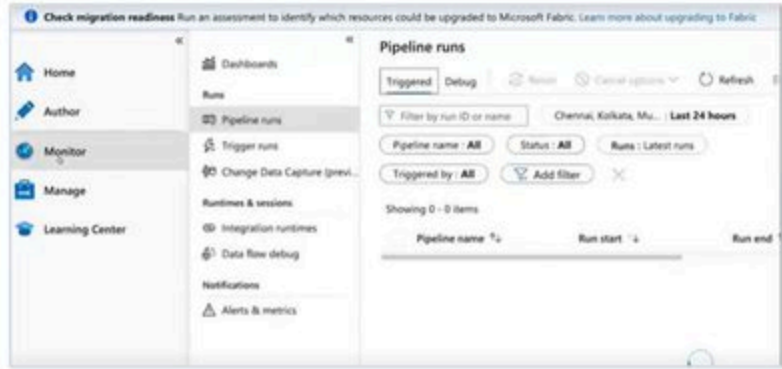


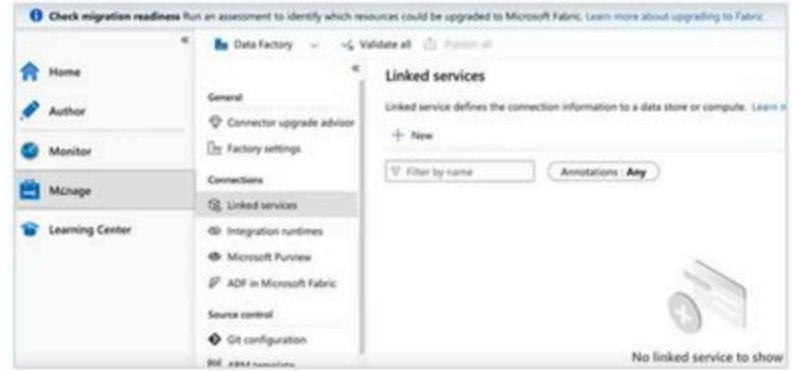
- Home: quick starts – Ingest, Orchestrate, Transform, Configure SSIS
- Banner: Set up code repository (Git)
- Learning Center: tutorials and videos

PART 2

ADF Studio – the hubs







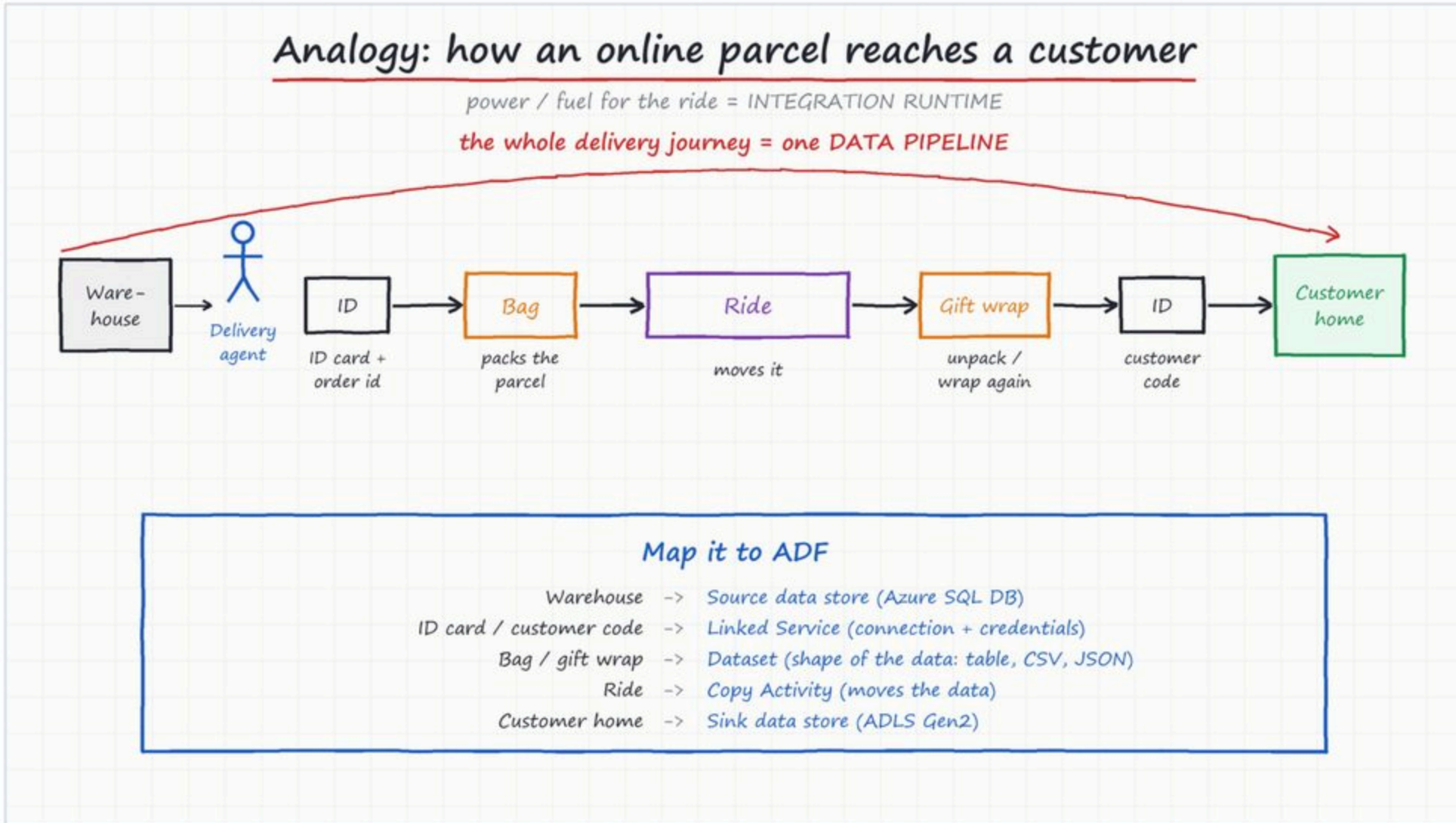
- Author (pencil): pipelines, datasets, data flows
- Monitor (gauge): pipeline runs, trigger runs, IR status
- Manage (toolbox): linked services, integration runtimes, Git, triggers
- Learning Center: guided tutorials

PART 3

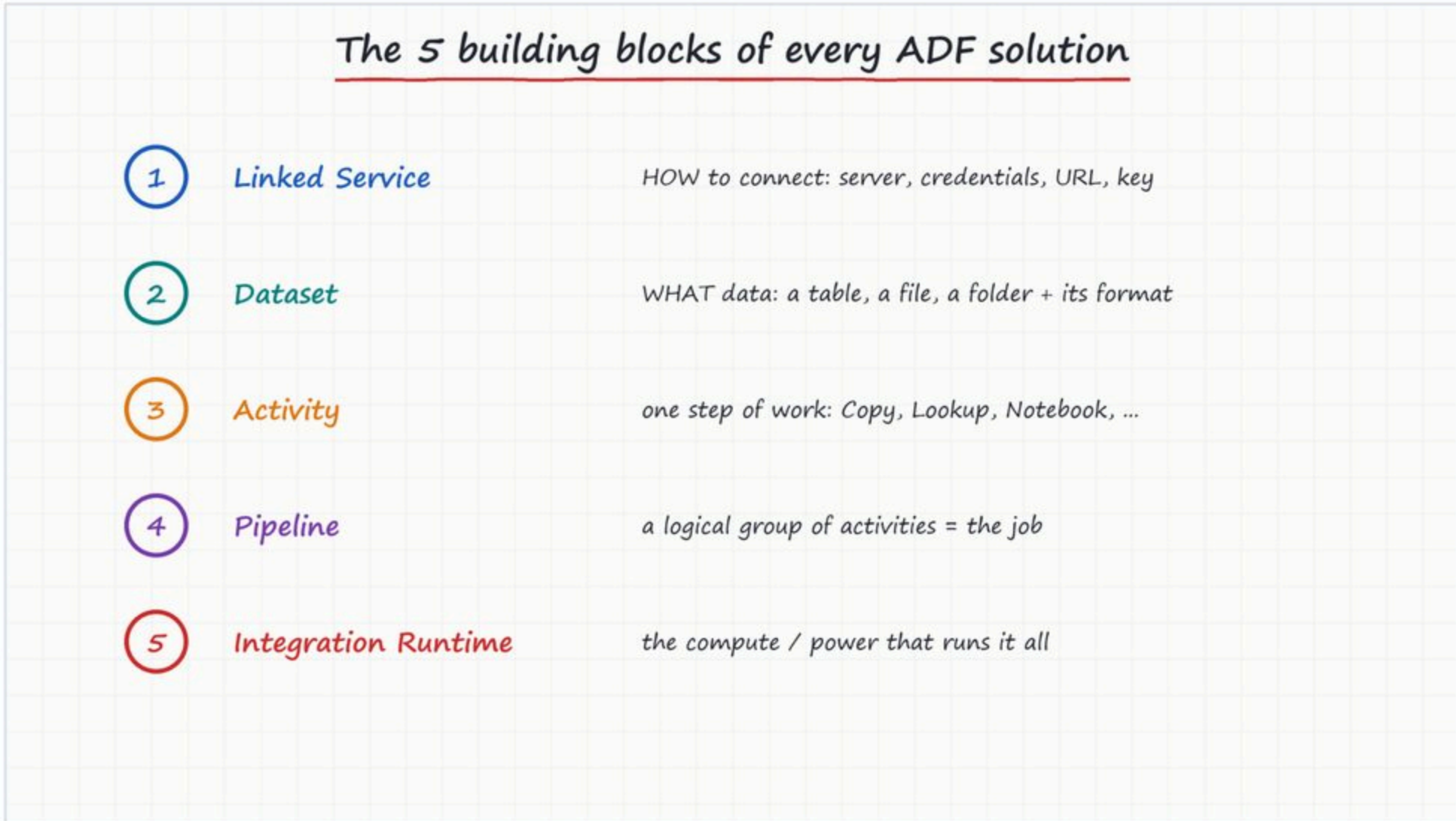
The ADF end-to-end flow

Linked service • Dataset • Activity • Pipeline • Integration runtime

Analogy: delivering an online order



The 5 building blocks of ADF

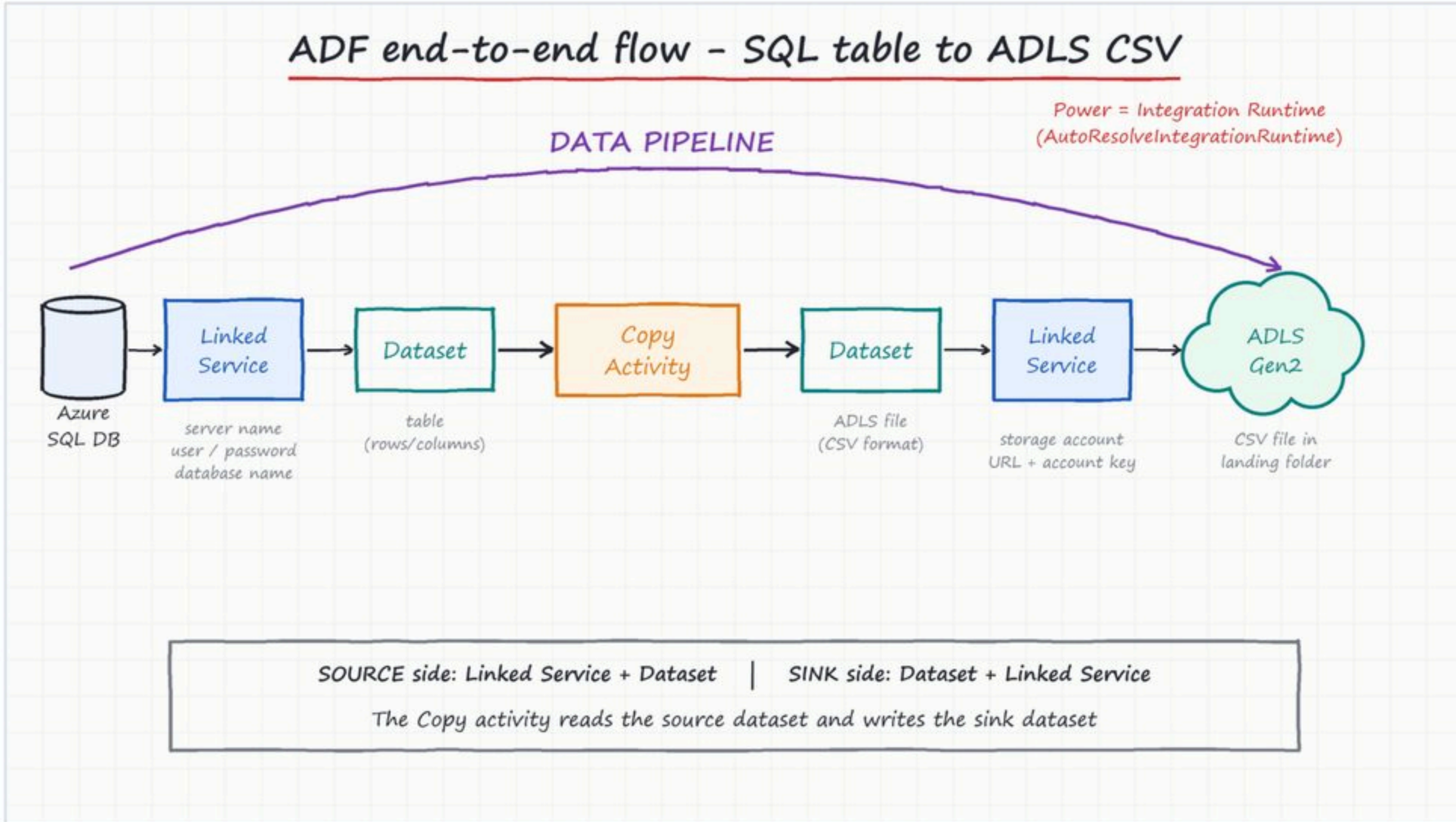


PART 3

Map the analogy to ADF

Delivery story	ADF component	In today's lab
Warehouse	Source data store	Azure SQL Database (sqldb-demo)
Agent ID card + order ID	Source linked service	SQL_DB_LS
Bag with the parcel	Source dataset	Customer_SQL_DB_DS (SalesLT.Customer)
The ride	Copy activity	Customer table to ADLS CSV
Gift wrap at the door	Sink dataset	ADLS_CSV_DS (DelimitedText)
Customer address + door code	Sink linked service	ADLS_LS (account key)
Customer home	Sink data store	ADLS Gen2 container landing
Whole journey	Pipeline	Ingest_customer_table_to_adls
Fuel / power	Integration runtime	AutoResolveIntegrationRuntime

ADF end-to-end flow – SQL table to CSV



Integration runtime – the compute behind ADF

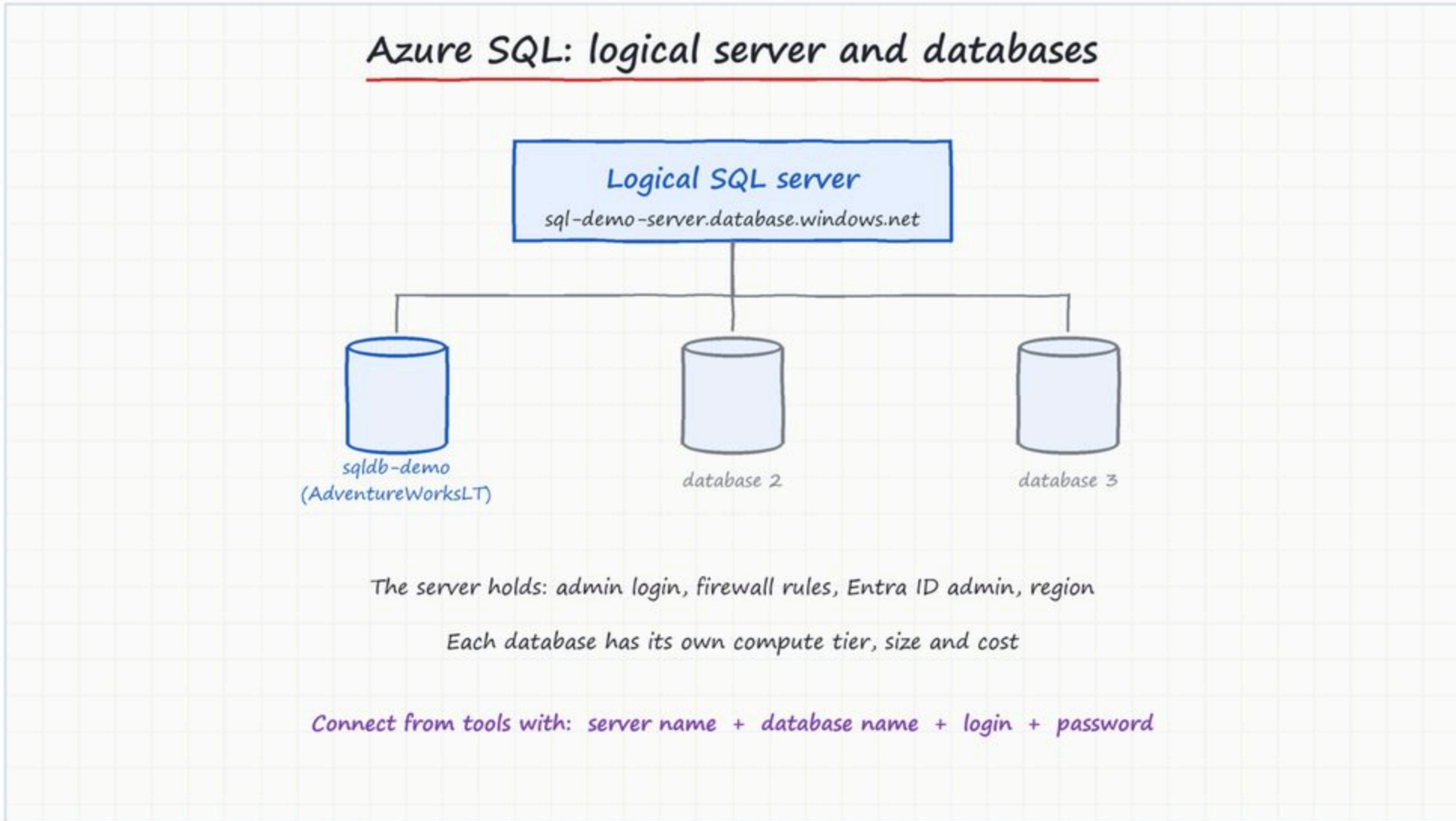
- Azure IR (AutoResolveIntegrationRuntime): managed, serverless, for cloud sources
- Self-hosted IR: installed on a VM/PC to reach on-premises or private networks
- Azure-SSIS IR: lift-and-shift of existing SSIS packages
- Copy performance unit: DIU (Data Integration Unit) – more DIUs = more power, more cost
- Today we use the default AutoResolveIntegrationRuntime

PART 4

Create the first cloud database

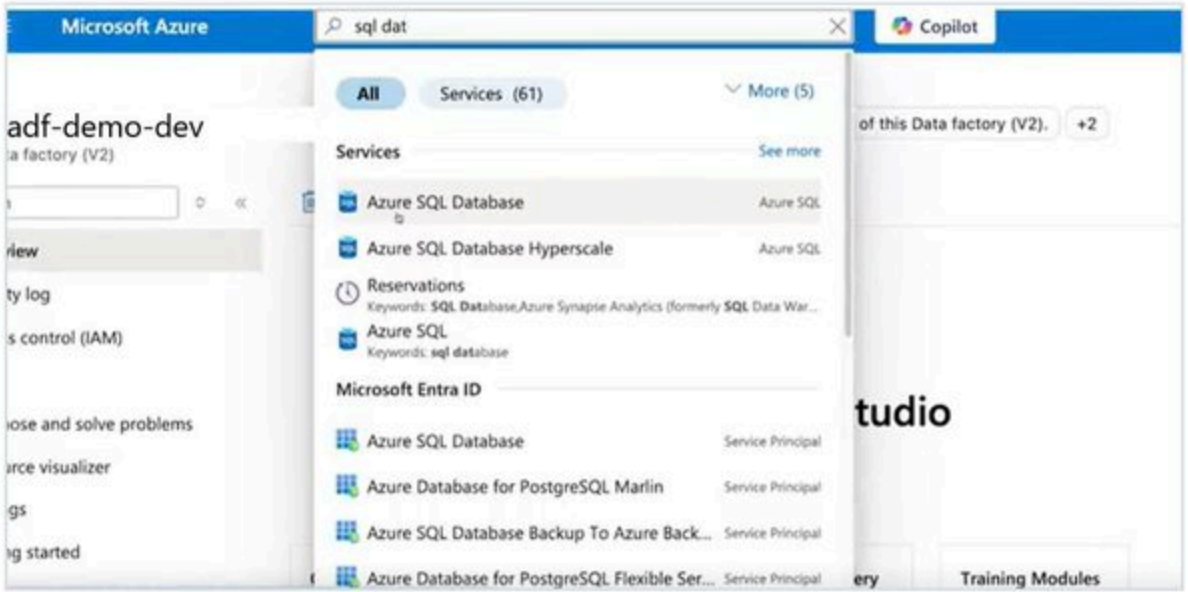
Azure SQL Database with AdventureWorksLT sample data

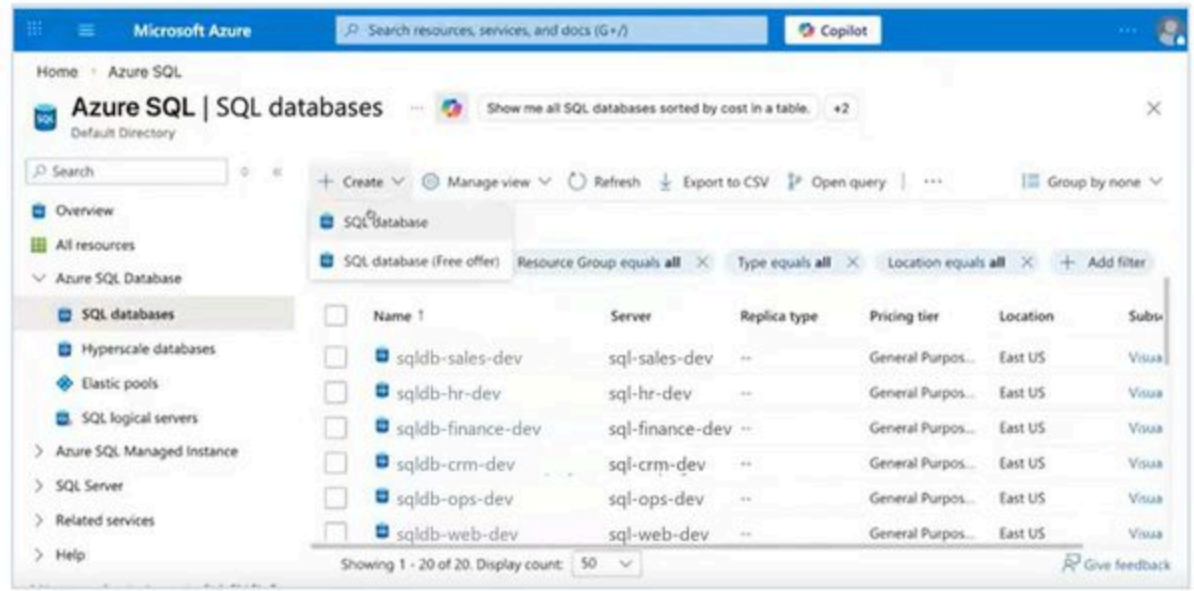
Azure SQL: logical server and databases



PART 4

Find Azure SQL → SQL databases → Create

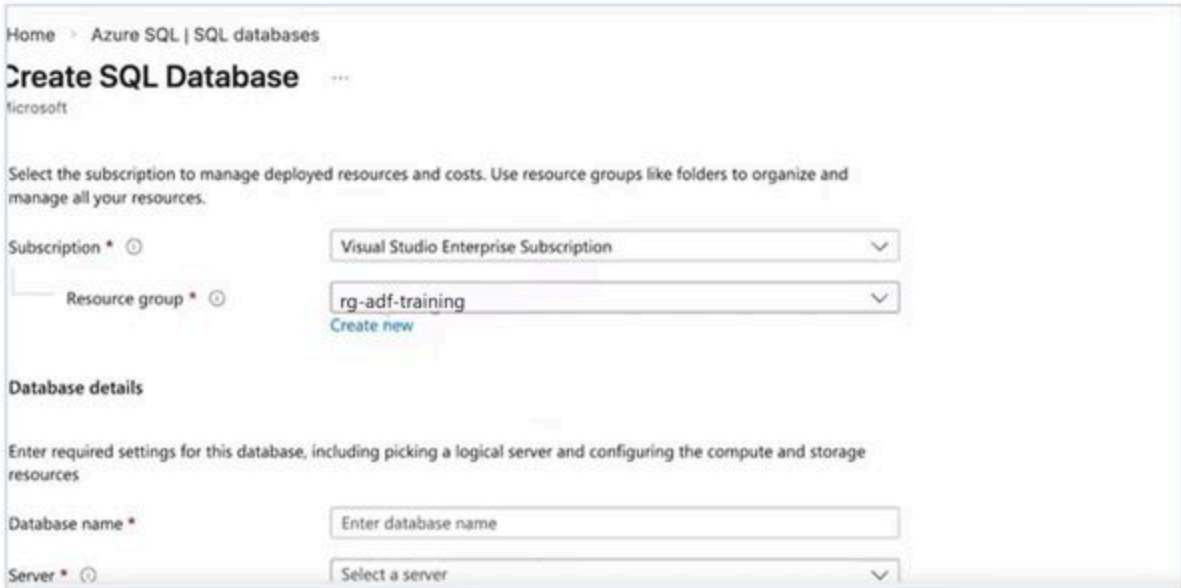


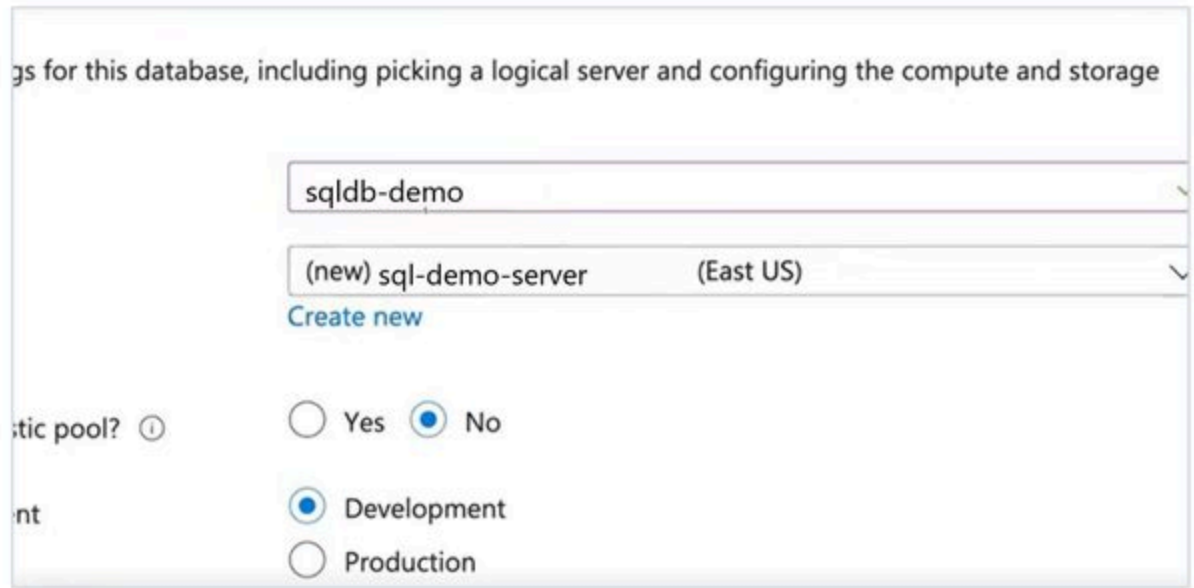


- Search "sql database" → Azure SQL
- SQL databases → + Create → SQL database
- Free offer option exists – good for learning

PART 4

Basics – resource group and database name

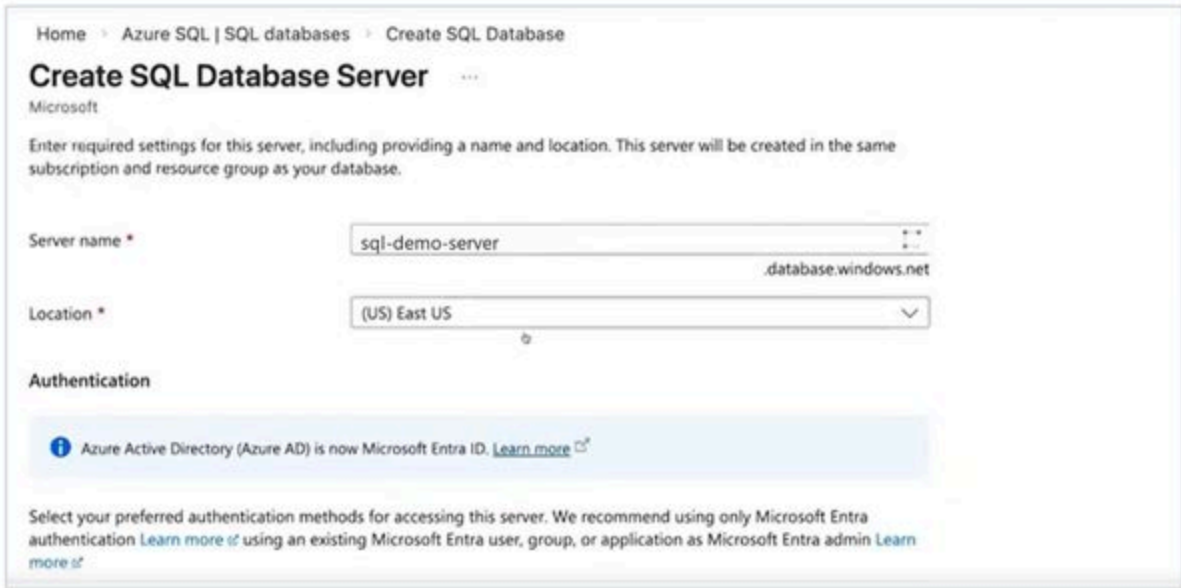


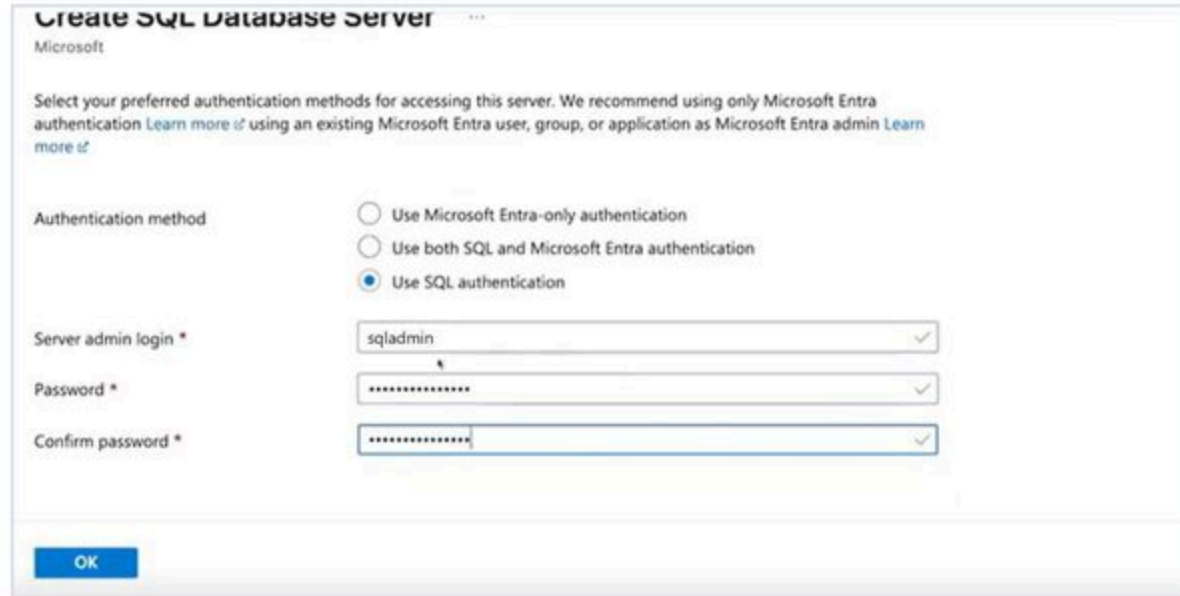


- Resource group: rg-adf-training
- Database name: sqldb-demo
- Server: Create new (next slide)
- Elastic pool: No; Workload: Development

PART 4

Create the logical SQL server



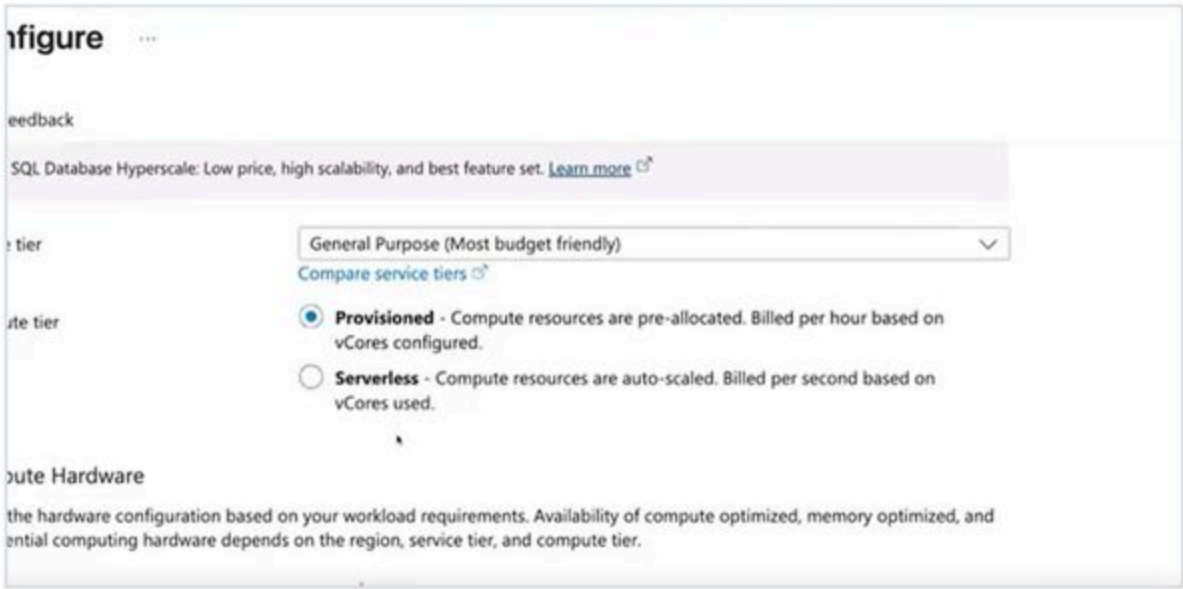


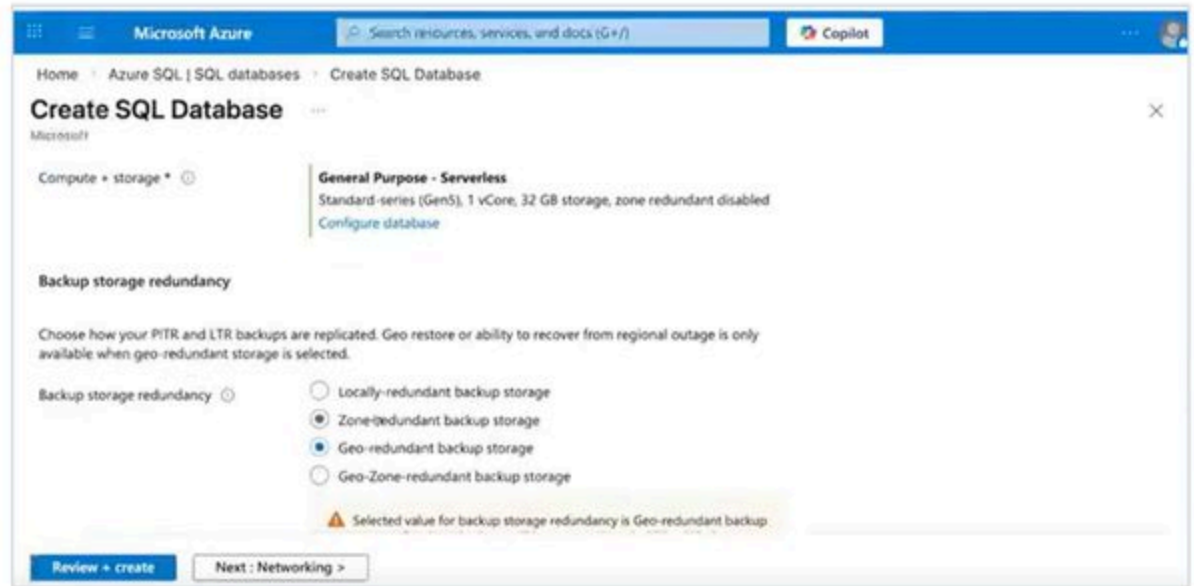
- Server name: globally unique (sql-demo-server)
- Location: same region as the database
- Authentication: SQL authentication (for the lab)
- Admin login + strong password – write them down!

⚠ Password rules: 8–128 characters, 3 of: uppercase, lowercase, number, symbol. Cannot contain the login name.

PART 4

Compute + storage – keep it cheap

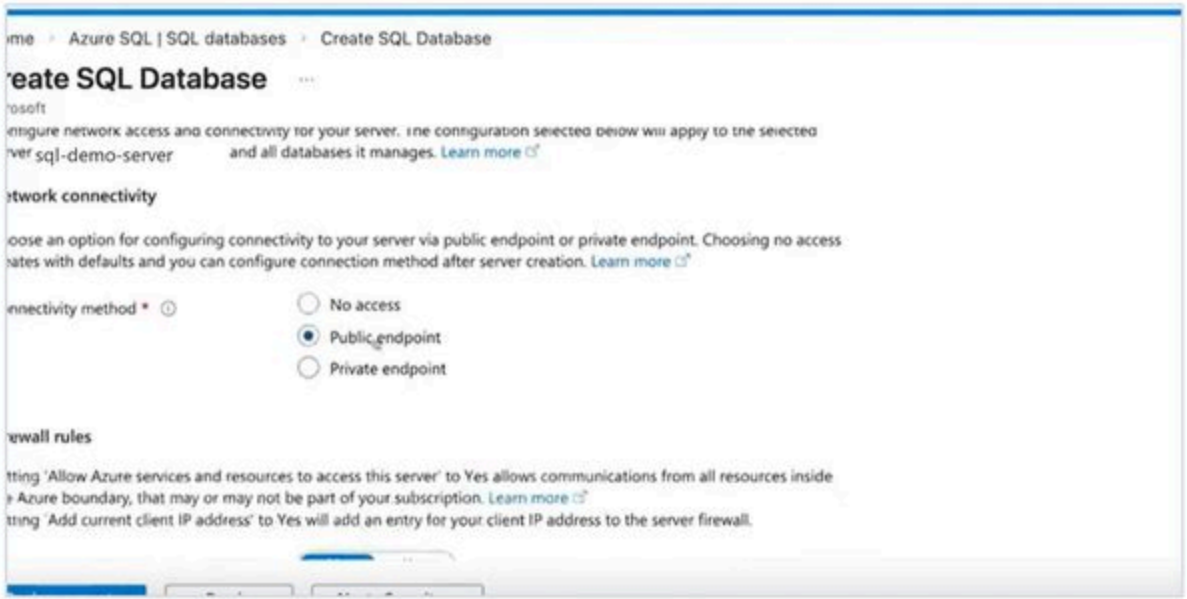


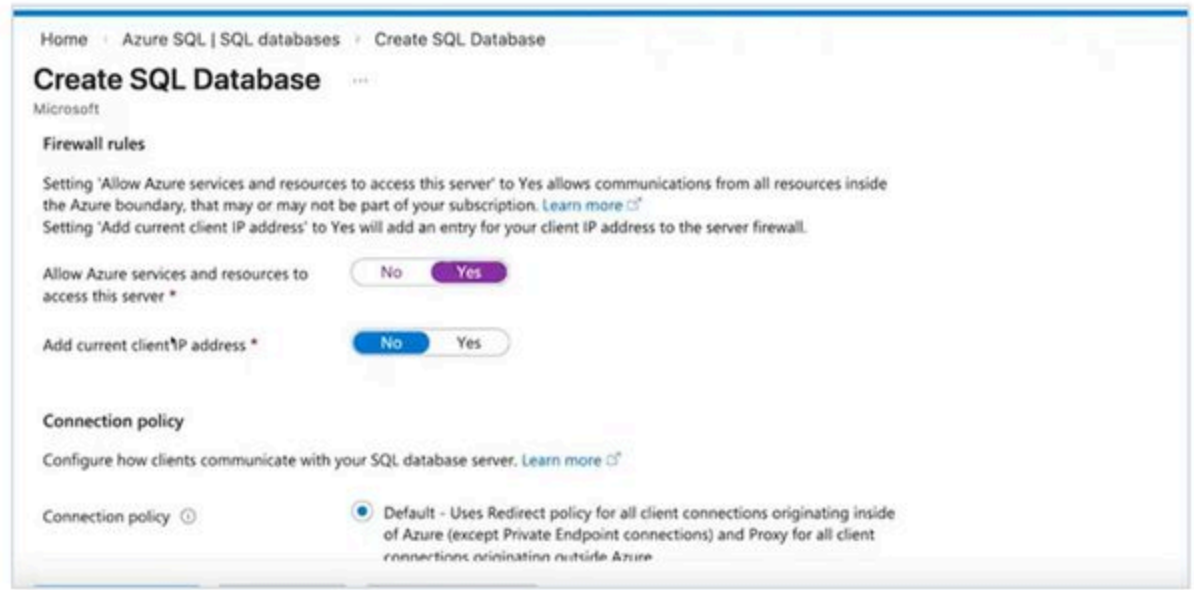


- Service tier: General Purpose
- Compute tier: Serverless (auto-pause, billed per second)
- Hardware: Standard-series, 1 vCore, small max size
- Backup redundancy: Locally-redundant (cheapest)

PART 4

Networking – public endpoint and firewall

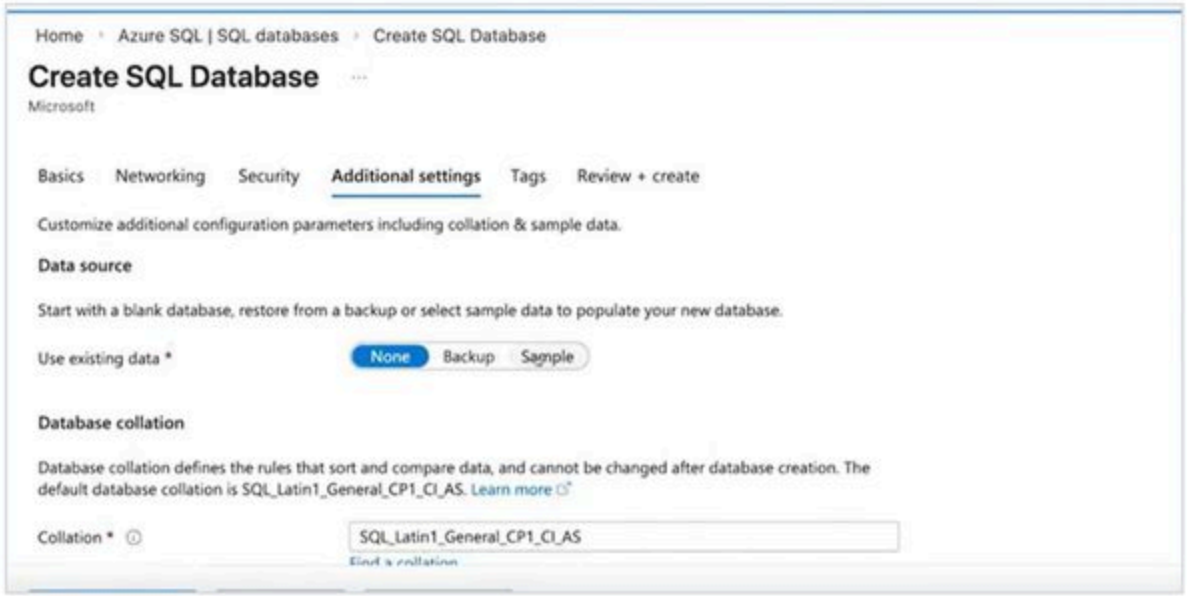




- Connectivity method: Public endpoint
- Allow Azure services and resources to access this server: Yes
- Add current client IP address: Yes
- Production: prefer Private endpoint

PART 4

Additional settings – load sample data



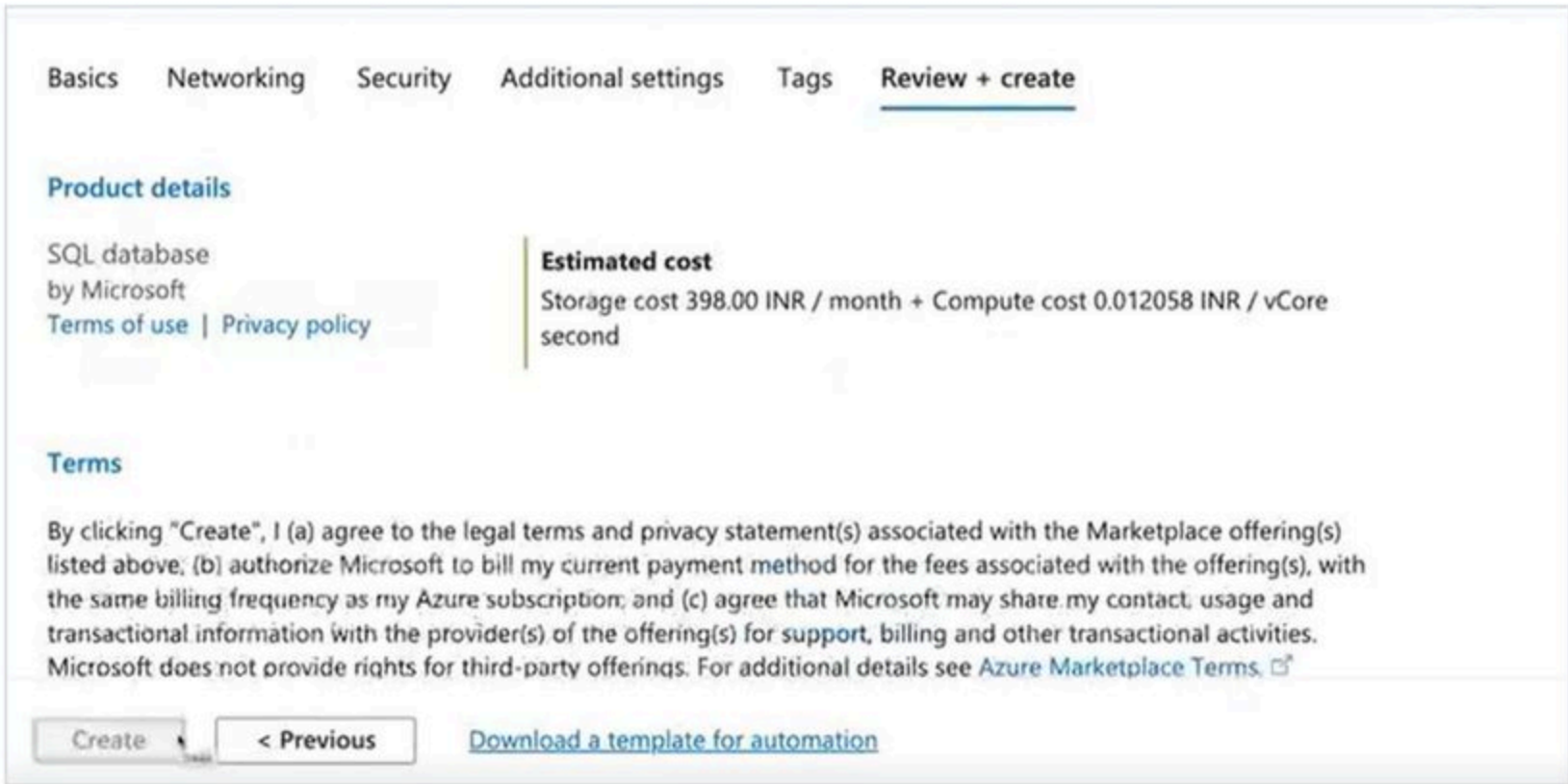


- Use existing data: Sample
- Creates the AdventureWorksLT database

- Tables in schema SalesLT: Customer, Product, Address, SalesOrderHeader …
- Gives us real data to copy – no scripts needed

PART 4

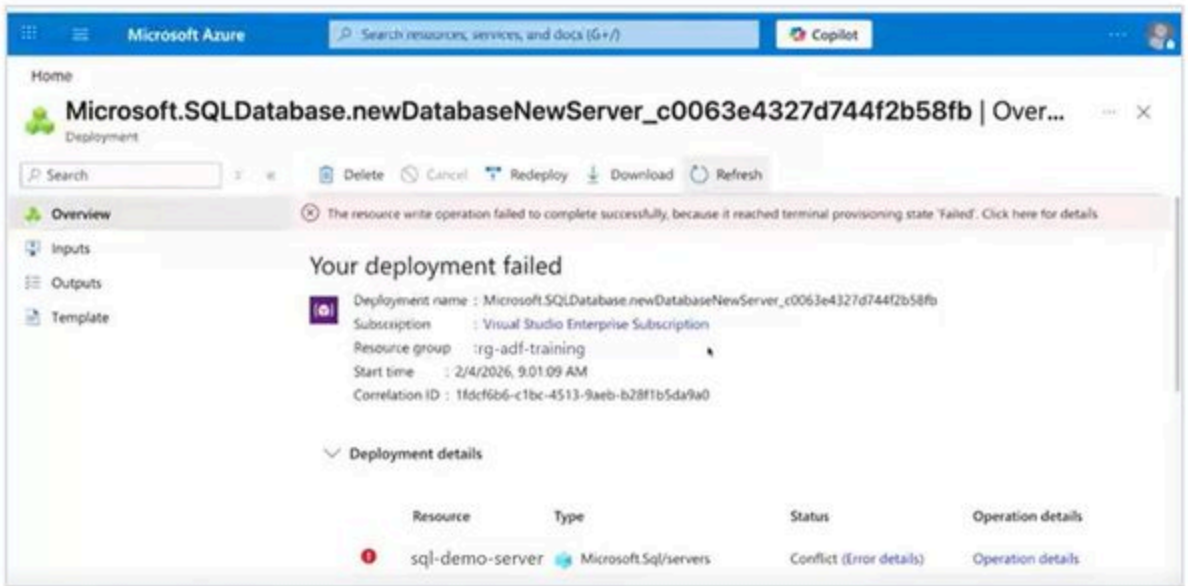
Review + create

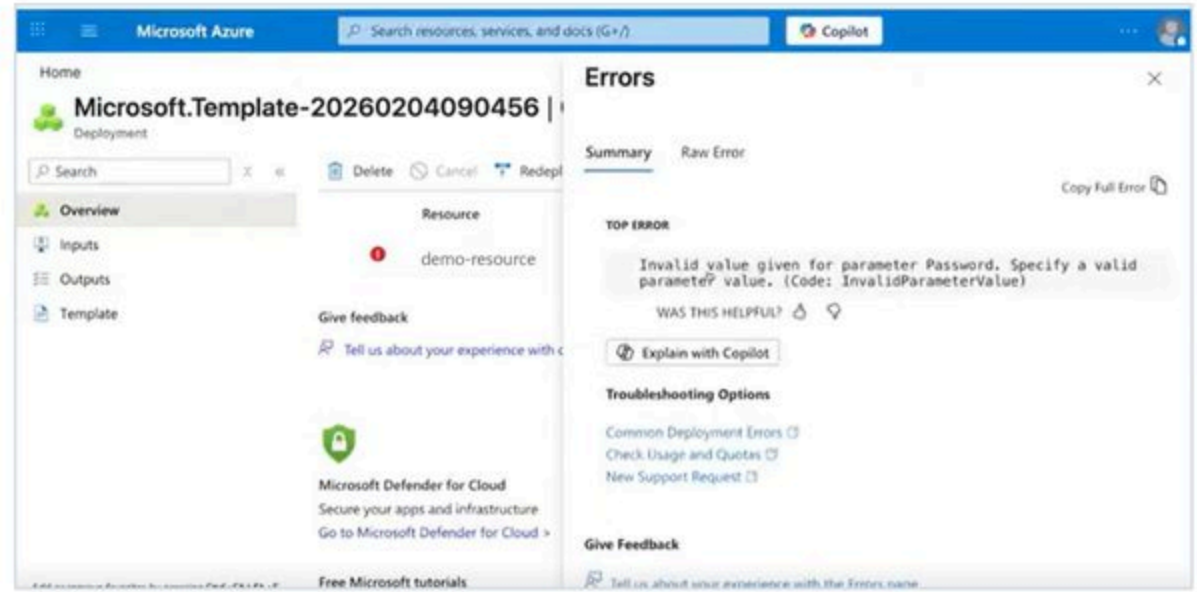


- Check the estimated cost (storage + compute per vCore-second)
- Click Create
- Deployment takes a few minutes

PART 4

Deployment failed? Read the error!





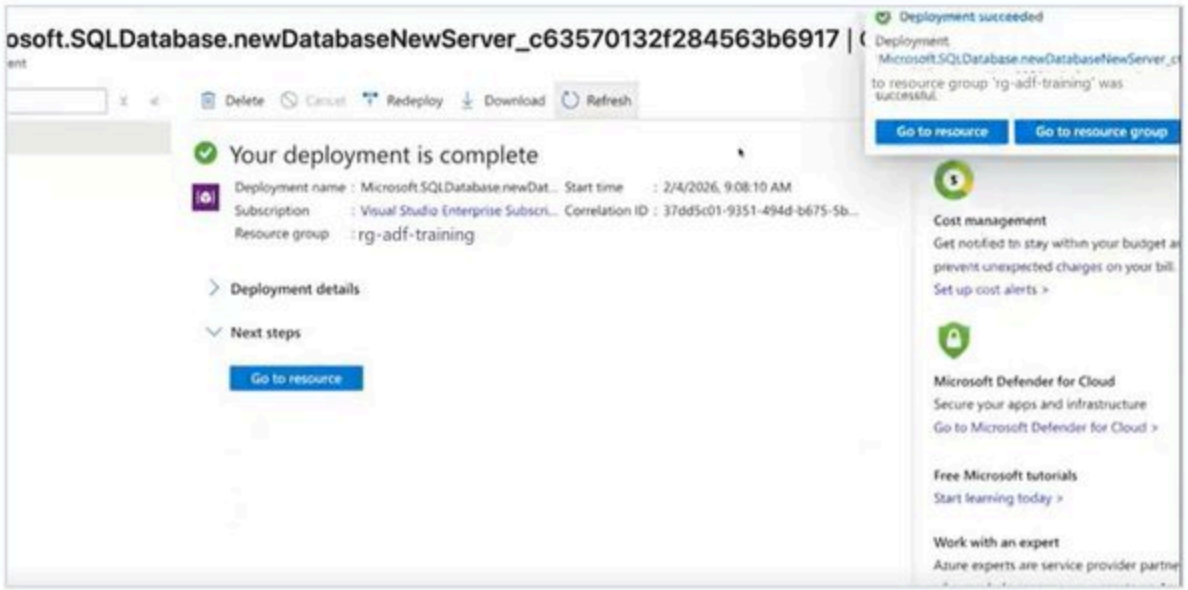
- Status: Conflict → Error details
- "Invalid value given for parameter Password"

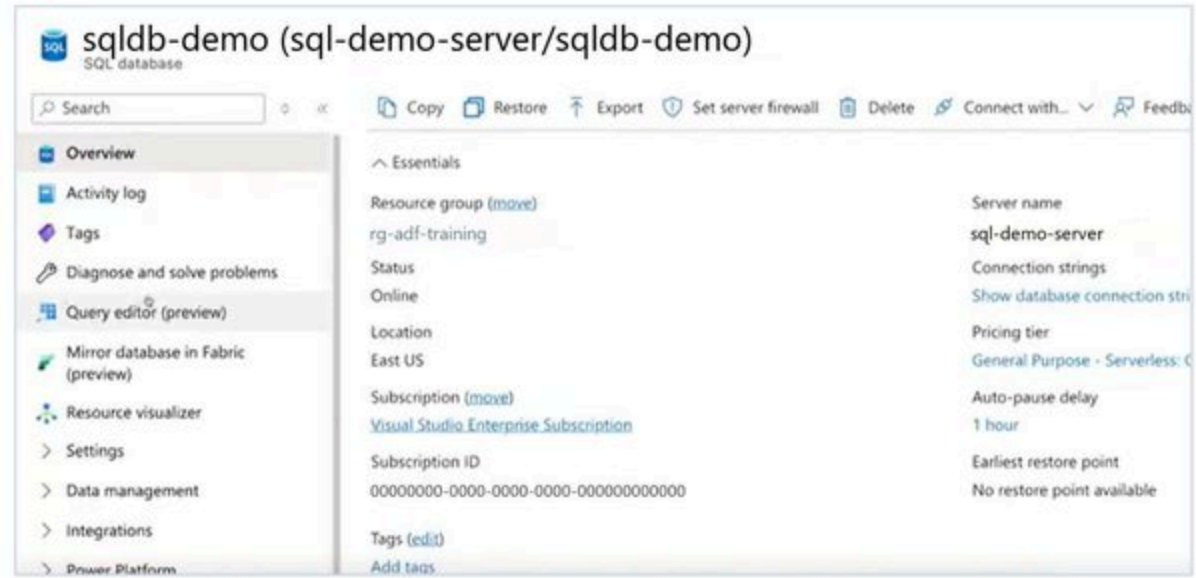
- Fix the password (complexity rules) and Redeploy / create again

⚠ Always open Error details / Operation details – the real reason is written there.

PART 4

Deployment complete – database Overview



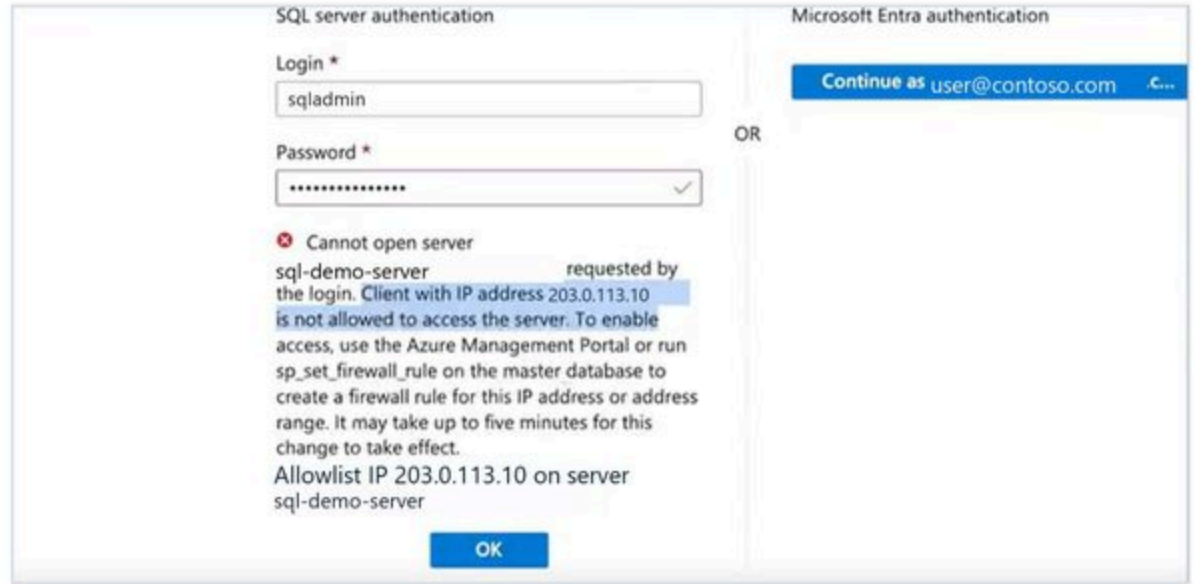


- Your deployment is complete → Go to resource
- Server name, status Online, pricing tier, auto-pause delay
- Connection strings available from the Overview page

PART 4

Query editor – sign in

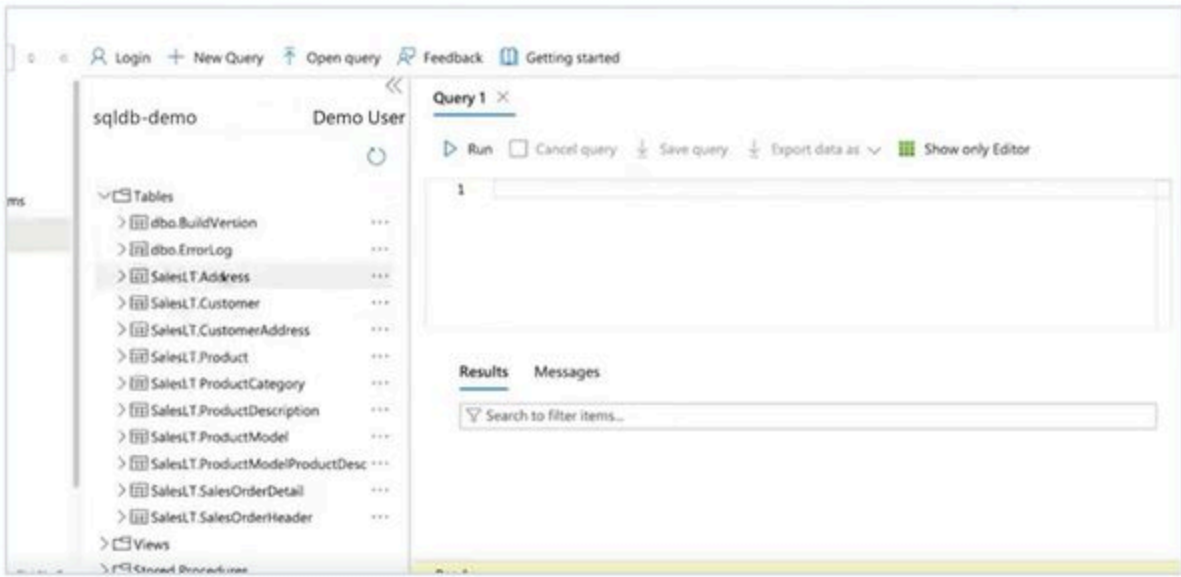


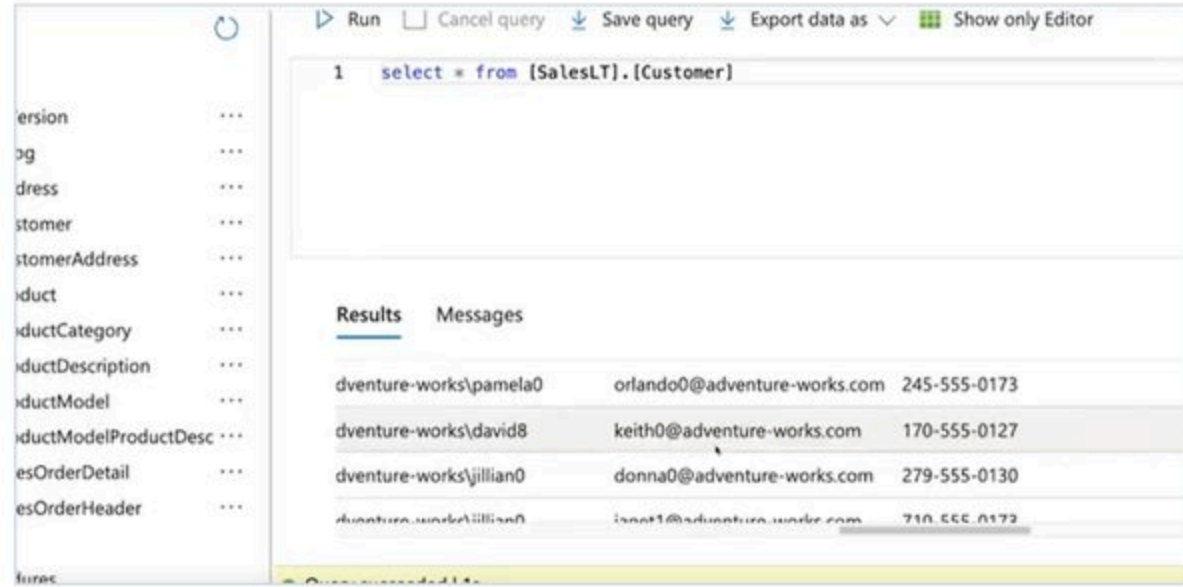


- Database → Query editor (preview)
- Login with SQL authentication (sqladmin + password)
- Error: "Client with IP address … is not allowed"
- Click Allowlist IP … to add a firewall rule

PART 4

Explore the sample tables





- Tables: SalesLT.Customer, Product, Address, SalesOrderHeader …
- SELECT * FROM [SalesLT].[Customer]

- Query succeeded – the database is ready

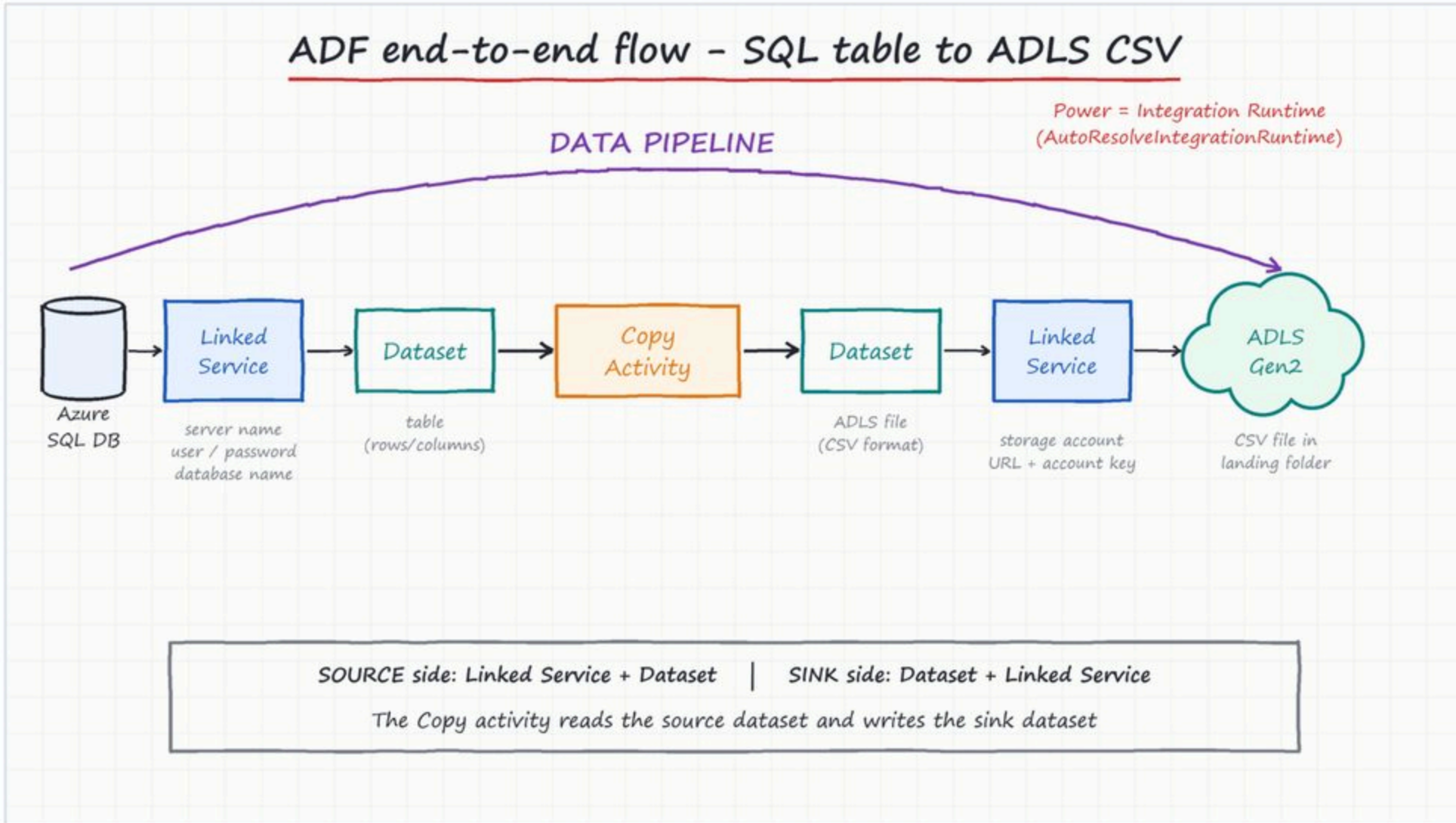
PART 5

Lab 1: Azure SQL DB → ADLS as CSV

Pipeline → Linked services → Datasets → Copy activity → Debug → Publish

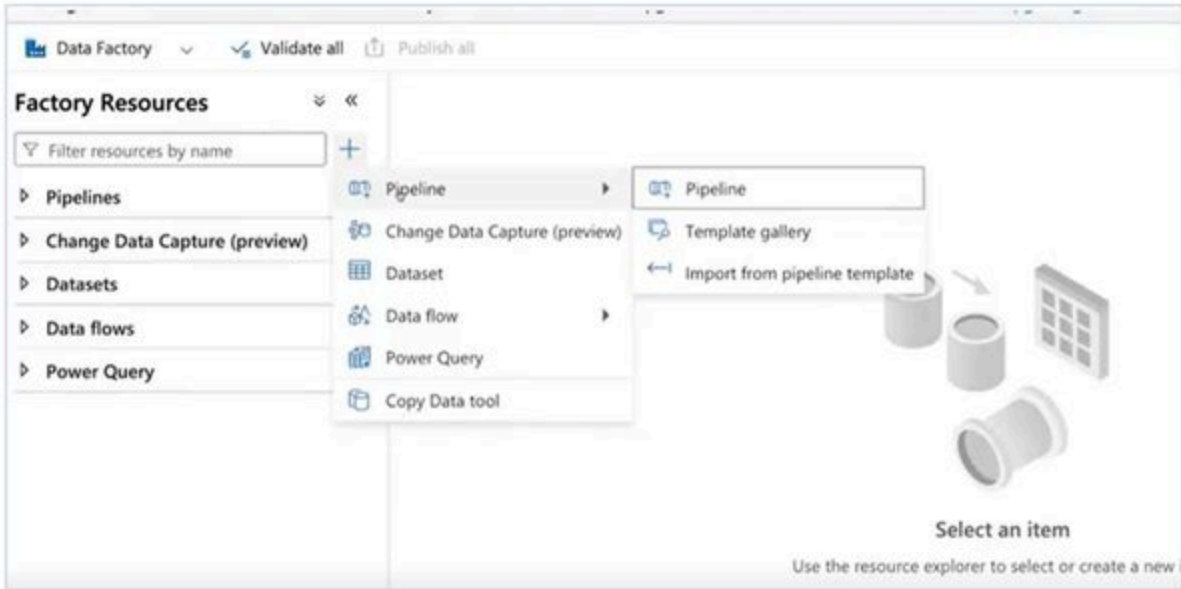
PART 5

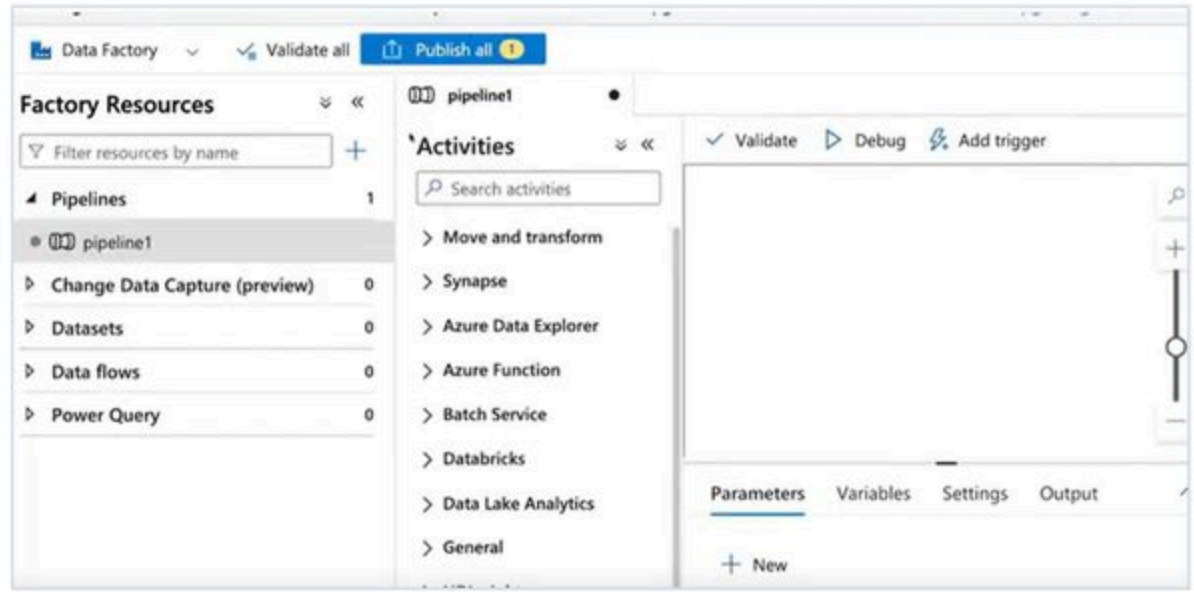
Lab 1 plan



PART 5

Step 1 – Create a pipeline

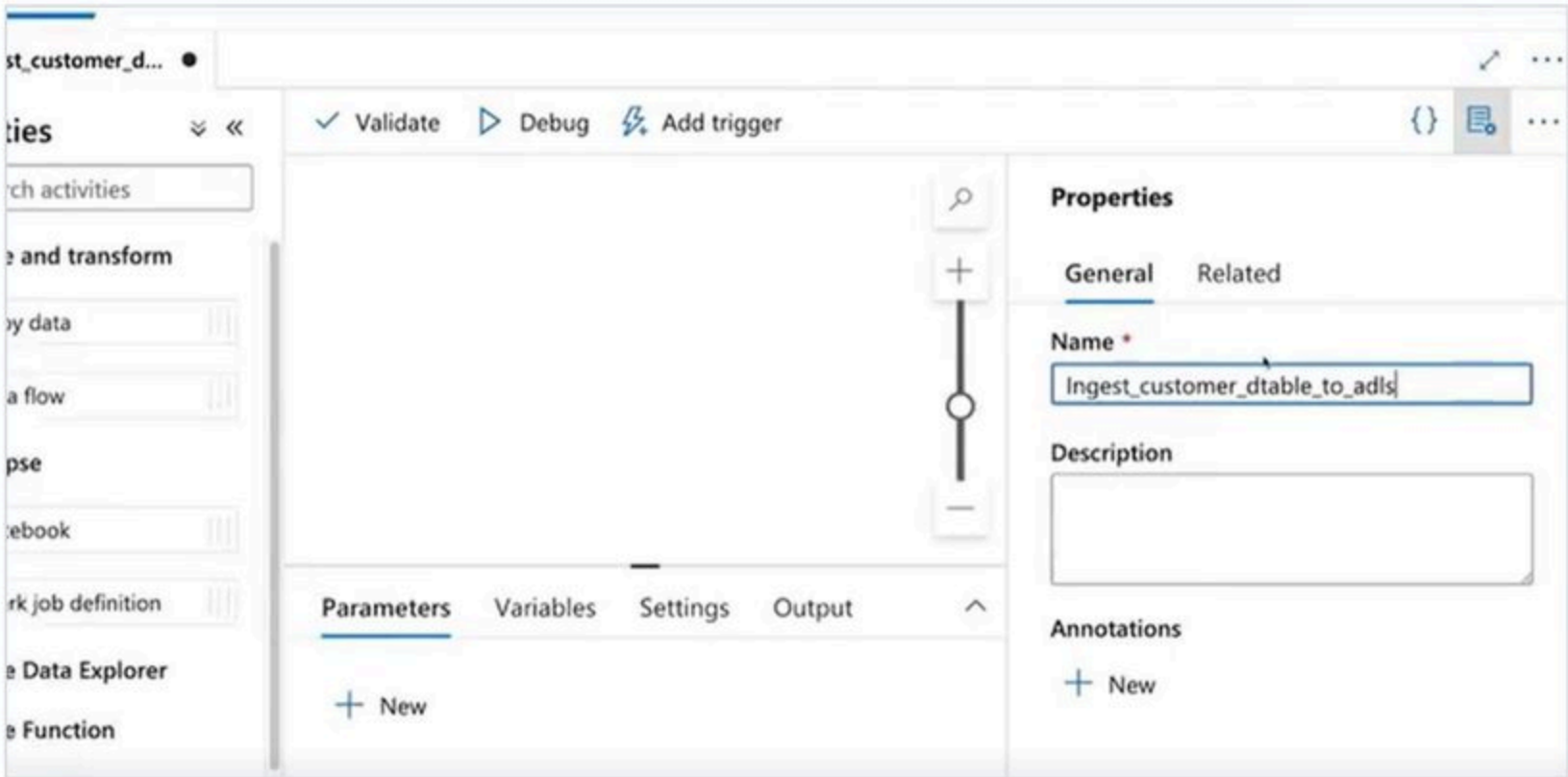




- Author hub → + (Factory Resources) → Pipeline → Pipeline
- A blank canvas opens with the Activities pane
- Publish all shows the number of unsaved changes

PART 5

Step 1 – Name the pipeline

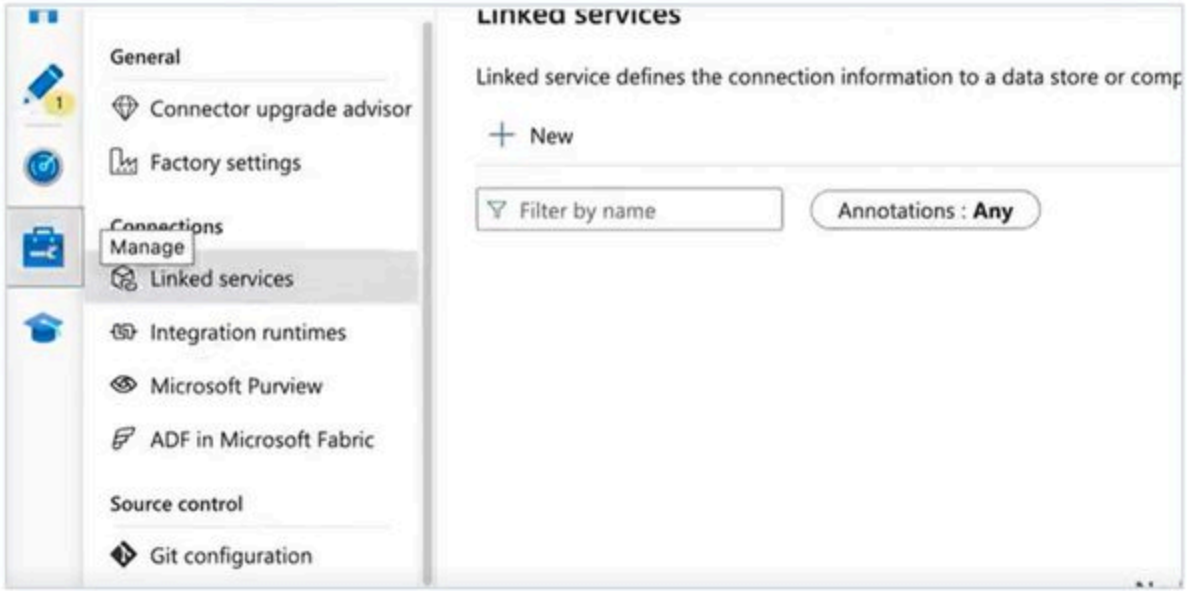


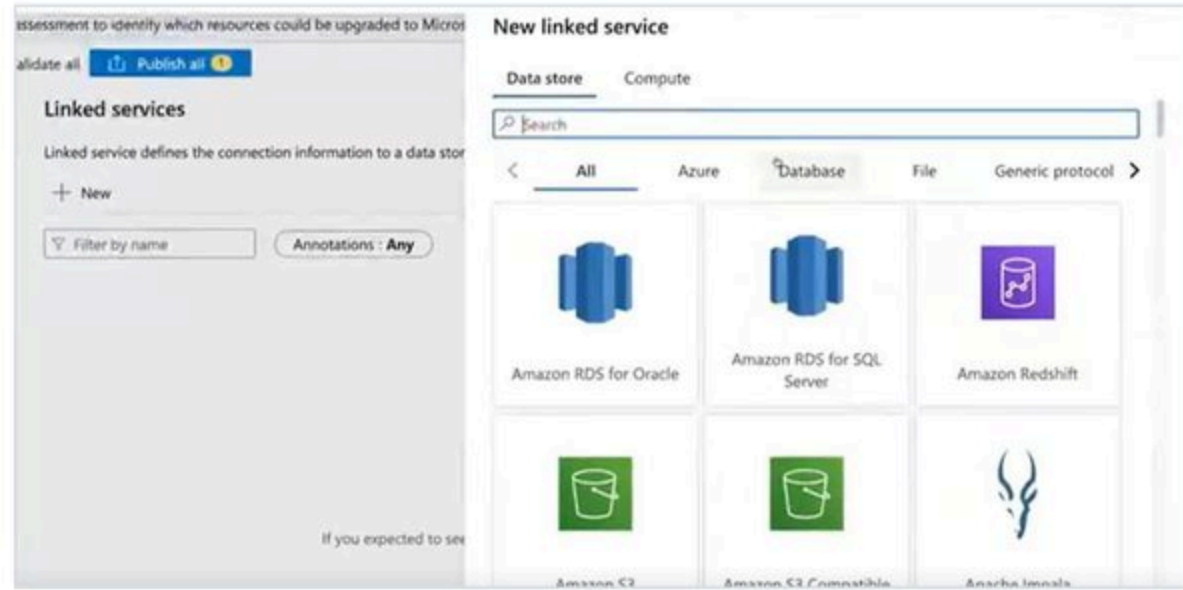
- Properties pane → Name
- Ingest_customer_table_to_adls
- Use a clear verb + object + target convention

⚠ Naming: letters, numbers and underscores are the safest choice for every ADF object name.

PART 5

Step 2 – Manage hub → Linked services

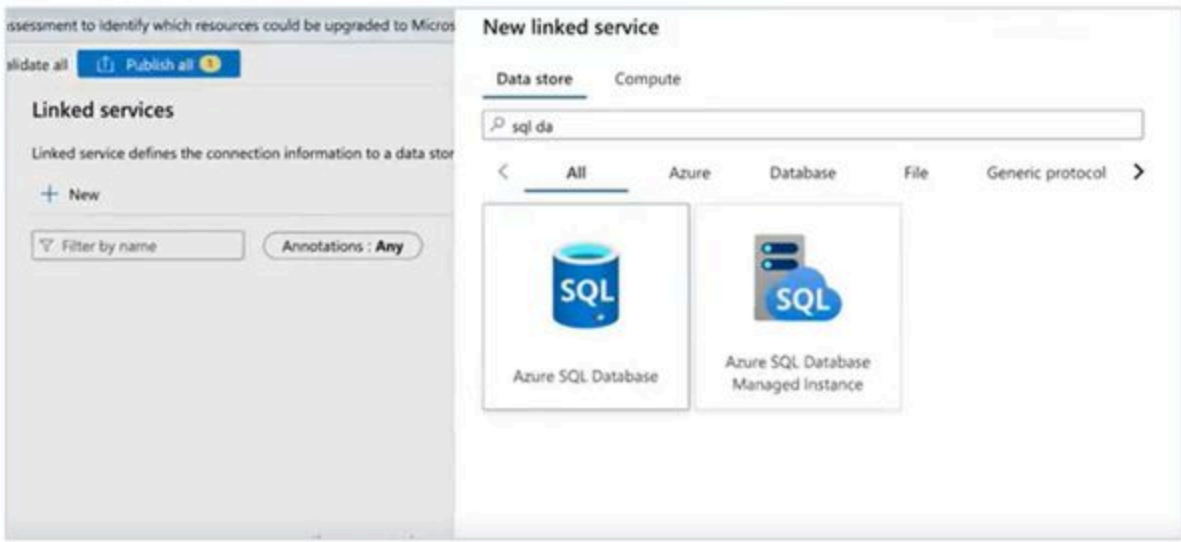


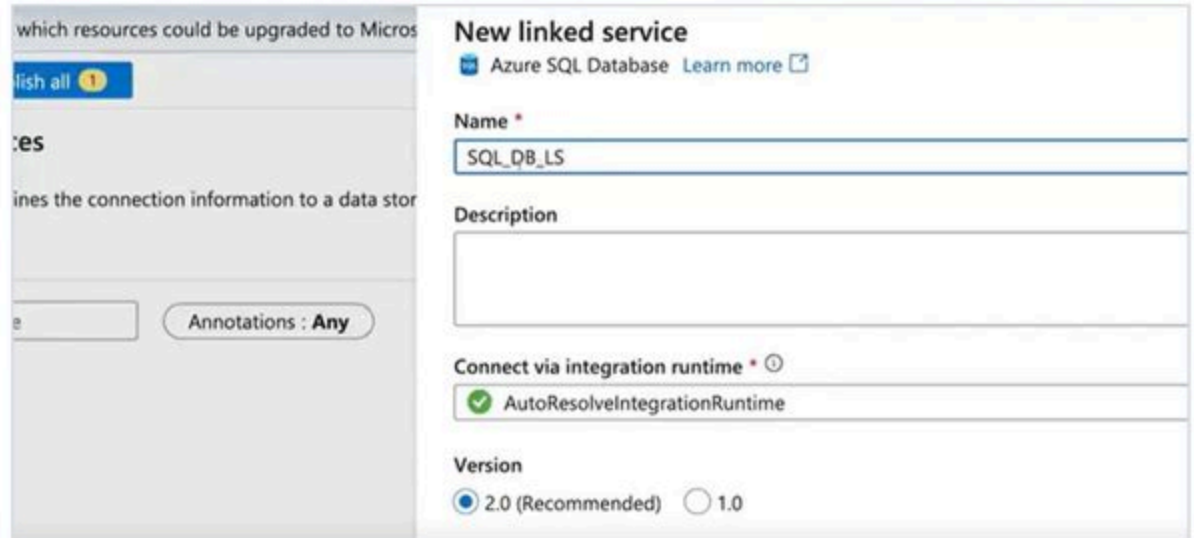


- Manage (toolbox) → Connections → Linked services
- + New → Data store gallery
- 90+ connectors: Azure, AWS, databases, files, SaaS

PART 5

Step 2a – Linked service for Azure SQL Database

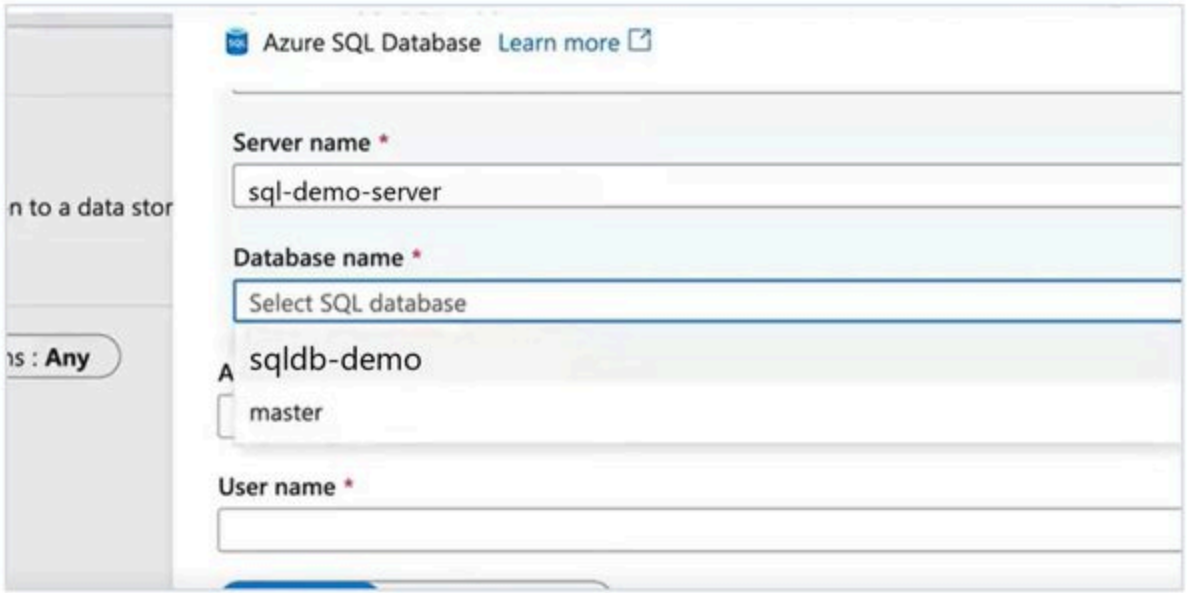


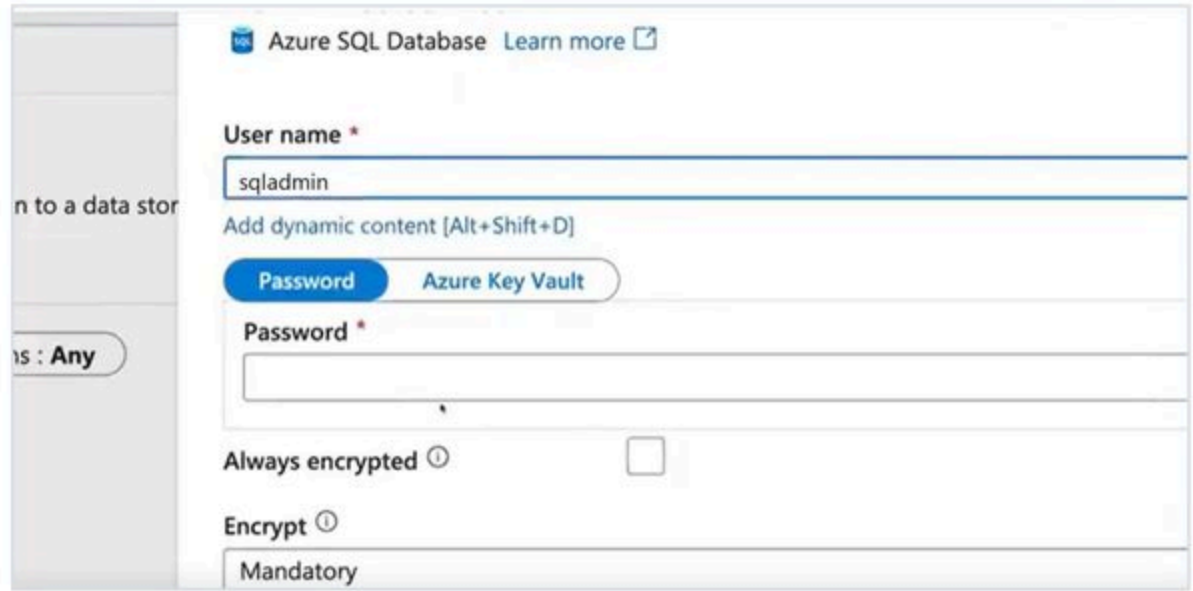


- Search "sql" → Azure SQL Database → Continue
- Name: SQL_DB_LS
- Connect via integration runtime: AutoResolveIntegrationRuntime
- Version 2.0 (Recommended)

PART 5

Step 2a – Server, database and credentials

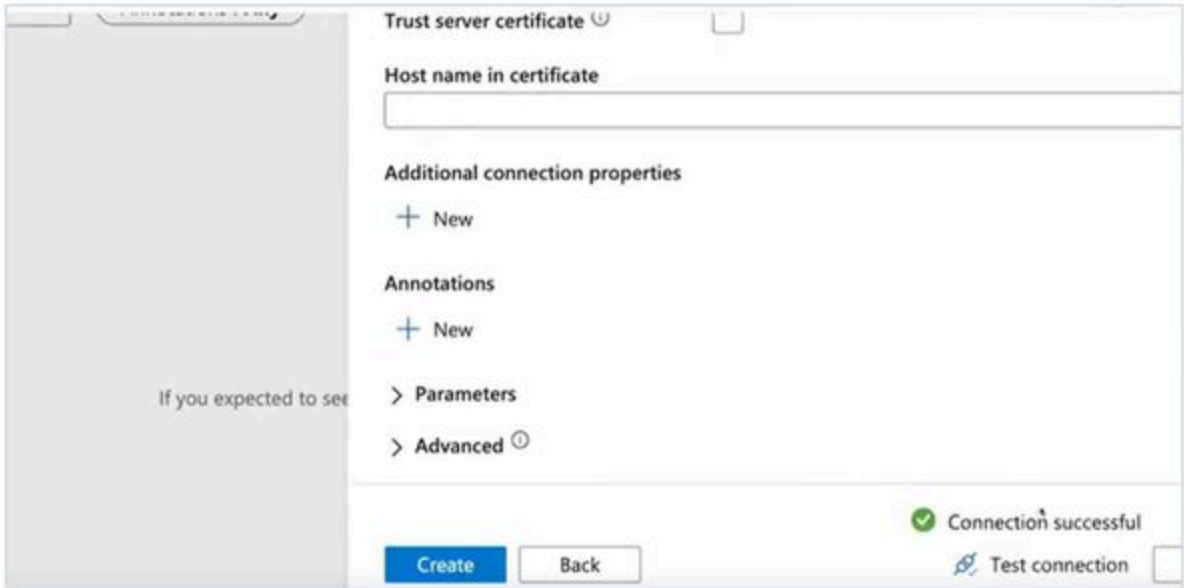


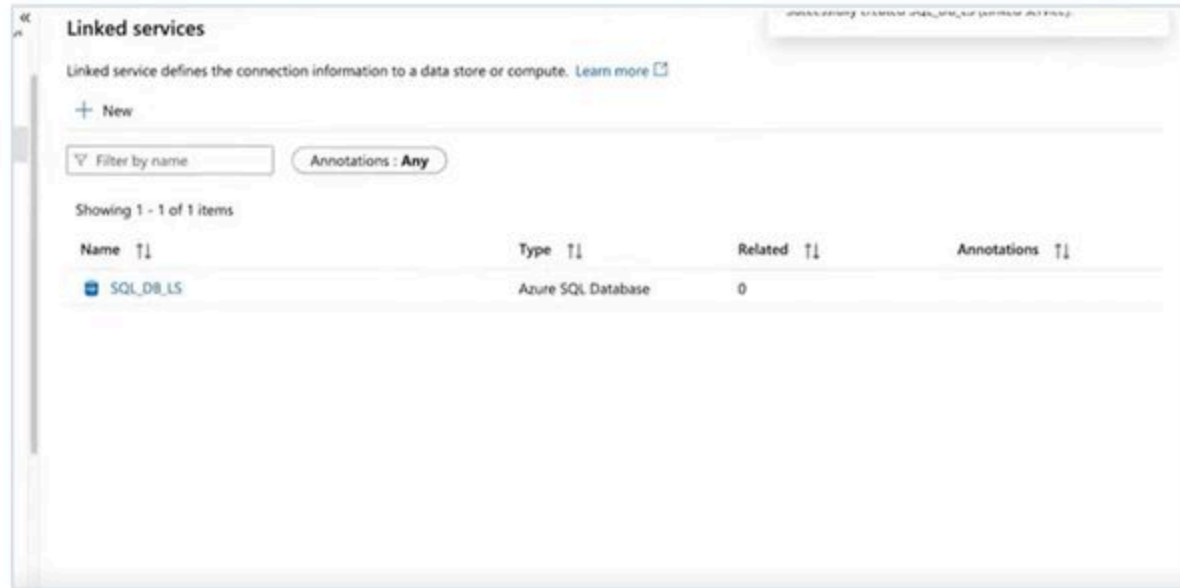


- Account selection: From Azure subscription
- Server name: sql-demo-server; Database: sqldb-demo
- Authentication: SQL authentication
- User name: sqladmin + Password
- Production: store the password in Azure Key Vault

PART 5

Step 2a – Test connection → Create



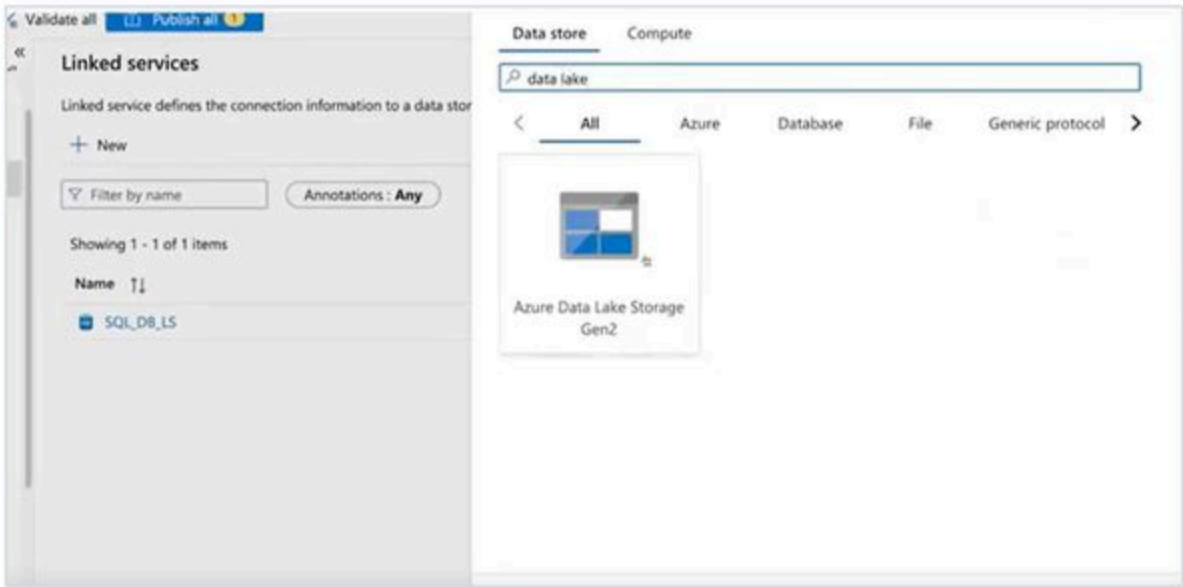


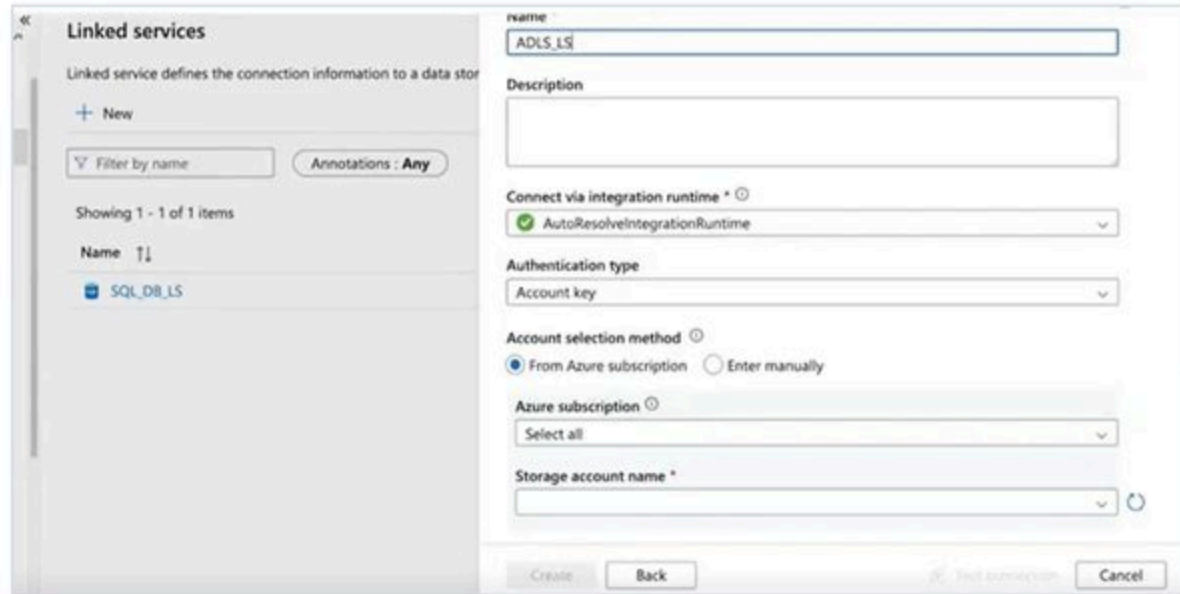
- Test connection → Connection successful
- Create
- SQL_DB_LS appears in the list

⚠ Test fails? Check: firewall "Allow Azure services" = Yes, correct server/database, correct password.

PART 5

Step 2b – Linked service for ADLS Gen2

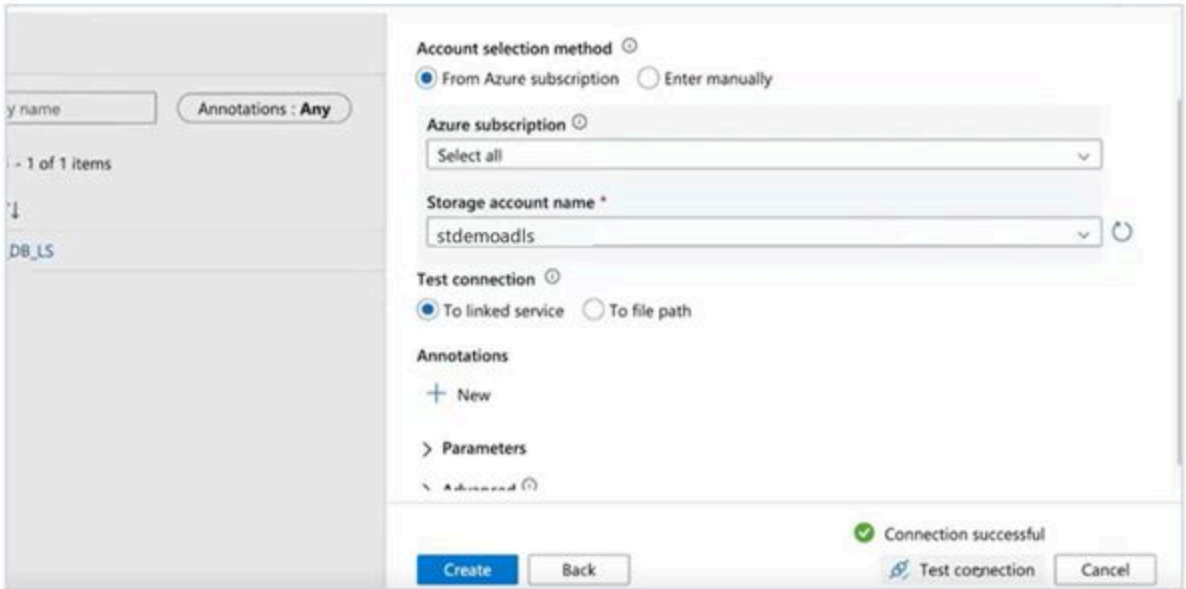


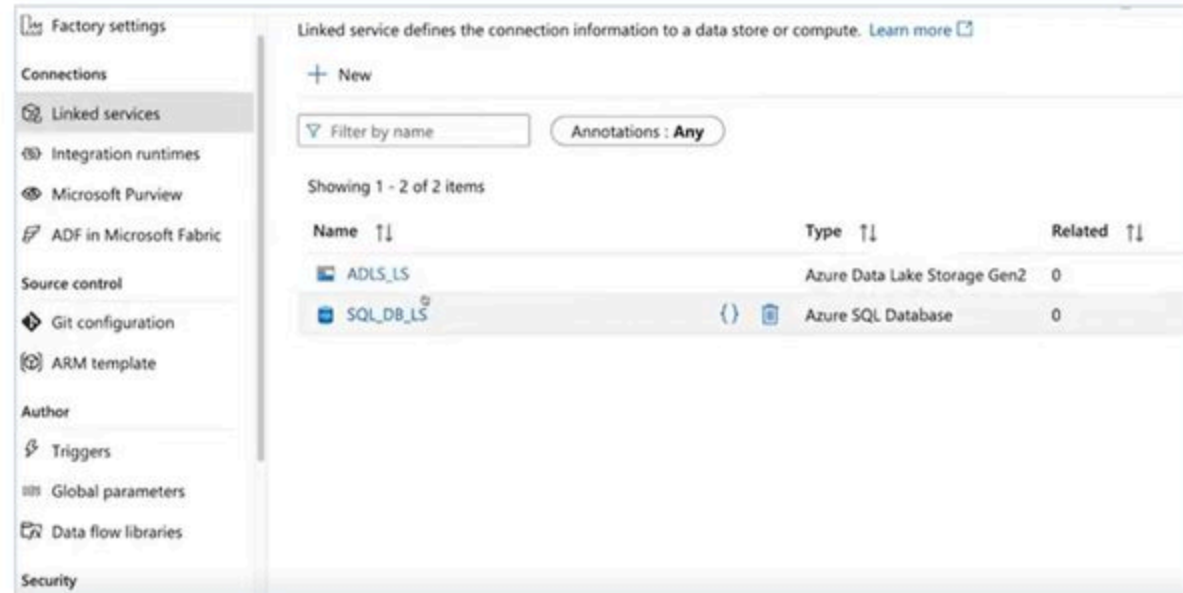


- + New → search "data lake" → Azure Data Lake Storage Gen2
- Name: ADLS_LS
- Authentication type: Account key
- Account selection: From Azure subscription

PART 5

Step 2b – Select storage account and test



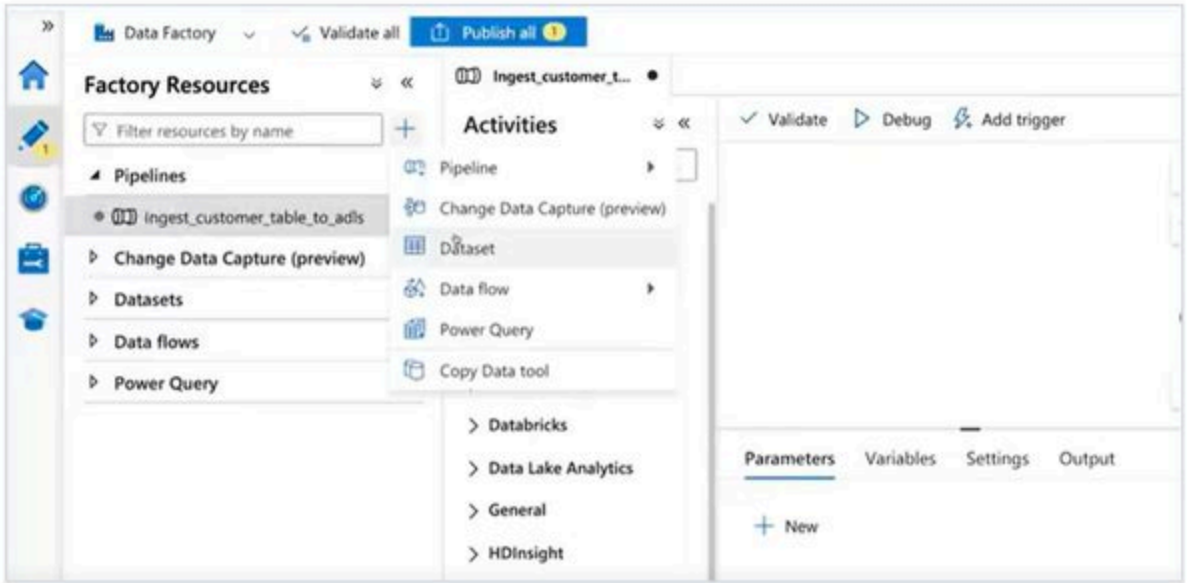


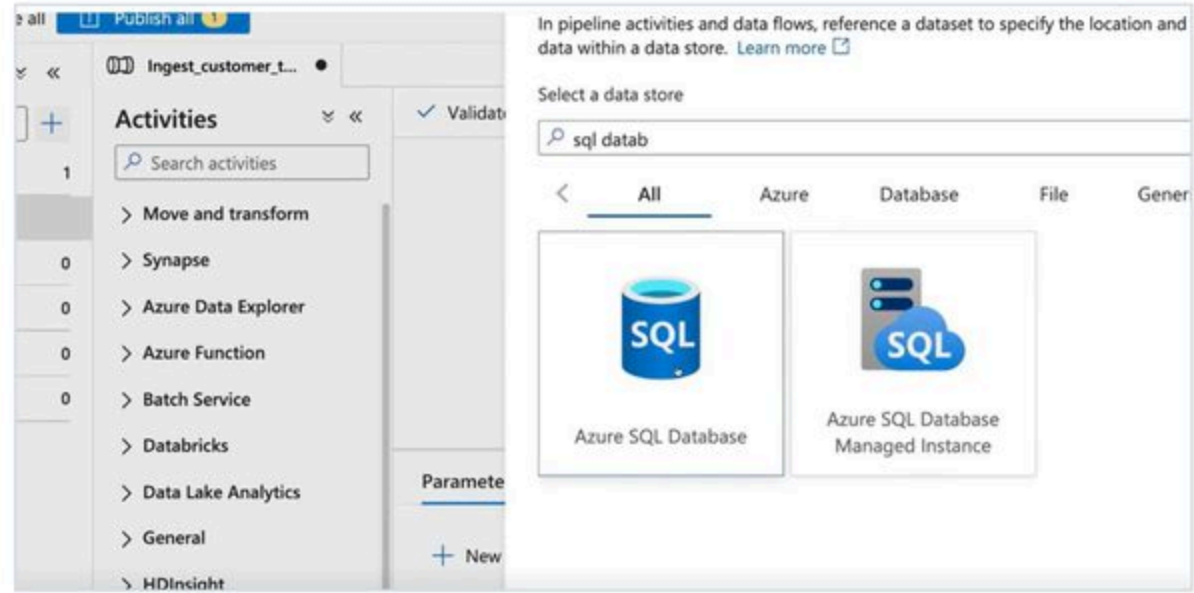
- Storage account name: stdemoadls
- Test connection → successful → Create

- Now we have 2 linked services: ADLS_LS and SQL_DB_LS

PART 5

Step 3 – Create the source dataset

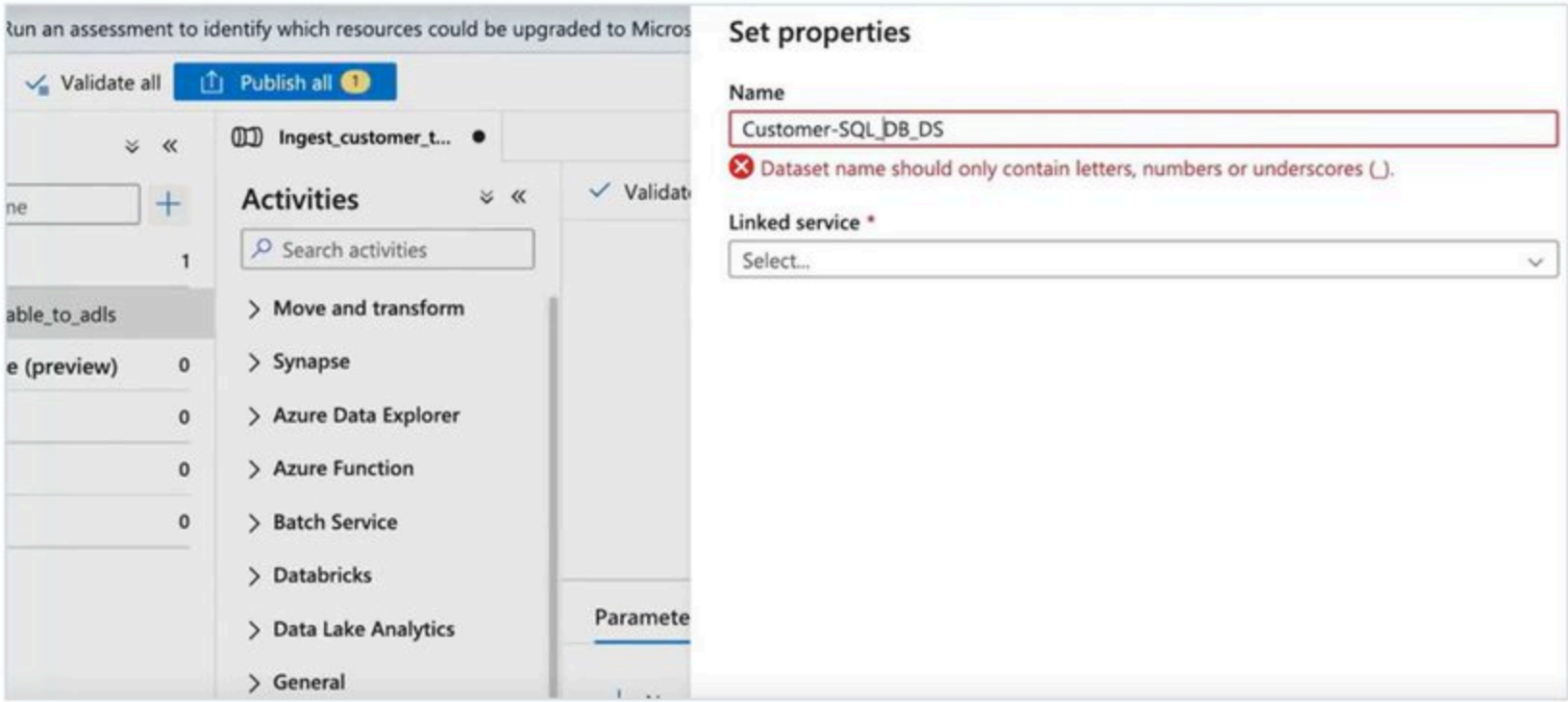




- Author → + → Dataset
- Search "sql" → Azure SQL Database → Continue

PART 5

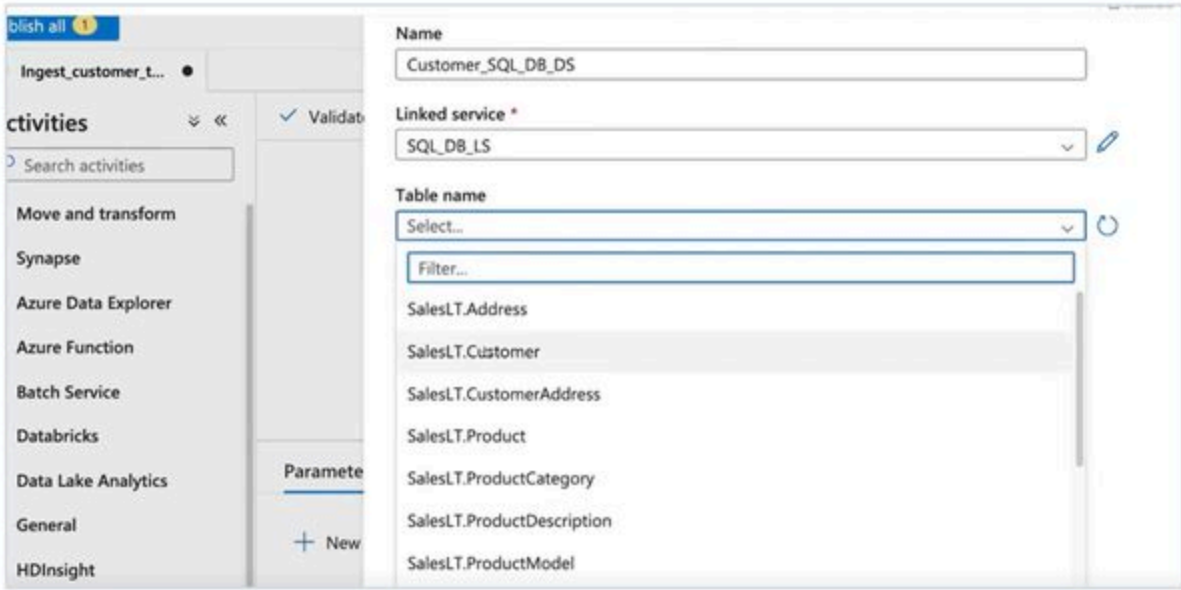
Dataset naming rule – a common error

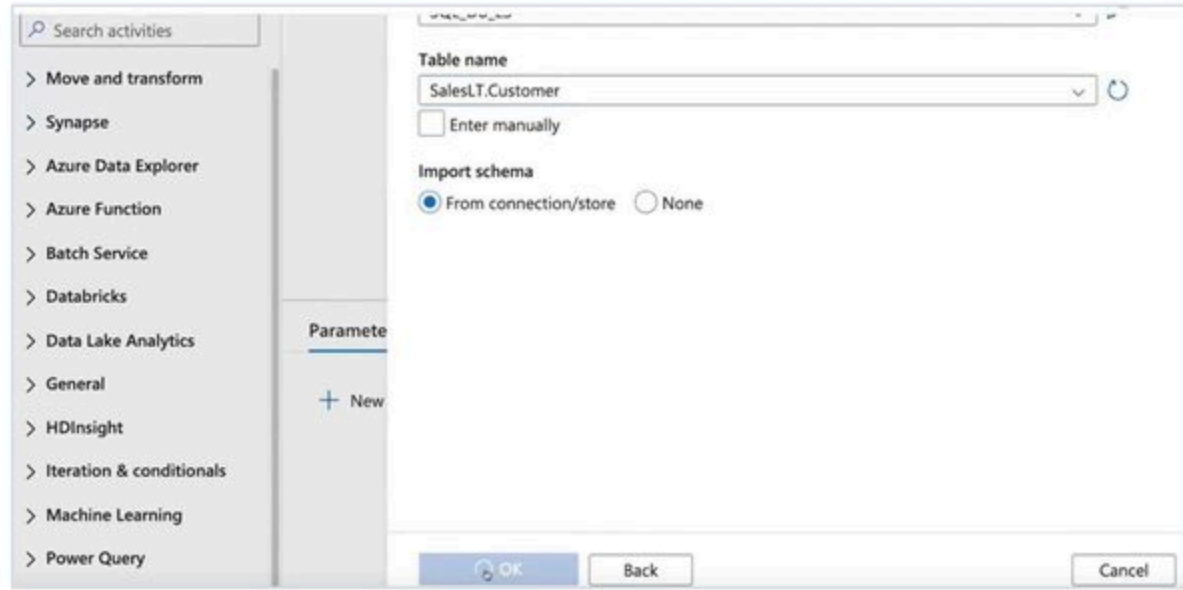


- Name “Customer-SQL_DB_DS” is rejected
- “Dataset name should only contain letters, numbers or underscores”
- Use Customer_SQL_DB_DS

PART 5

Step 3 – Set properties: linked service + table

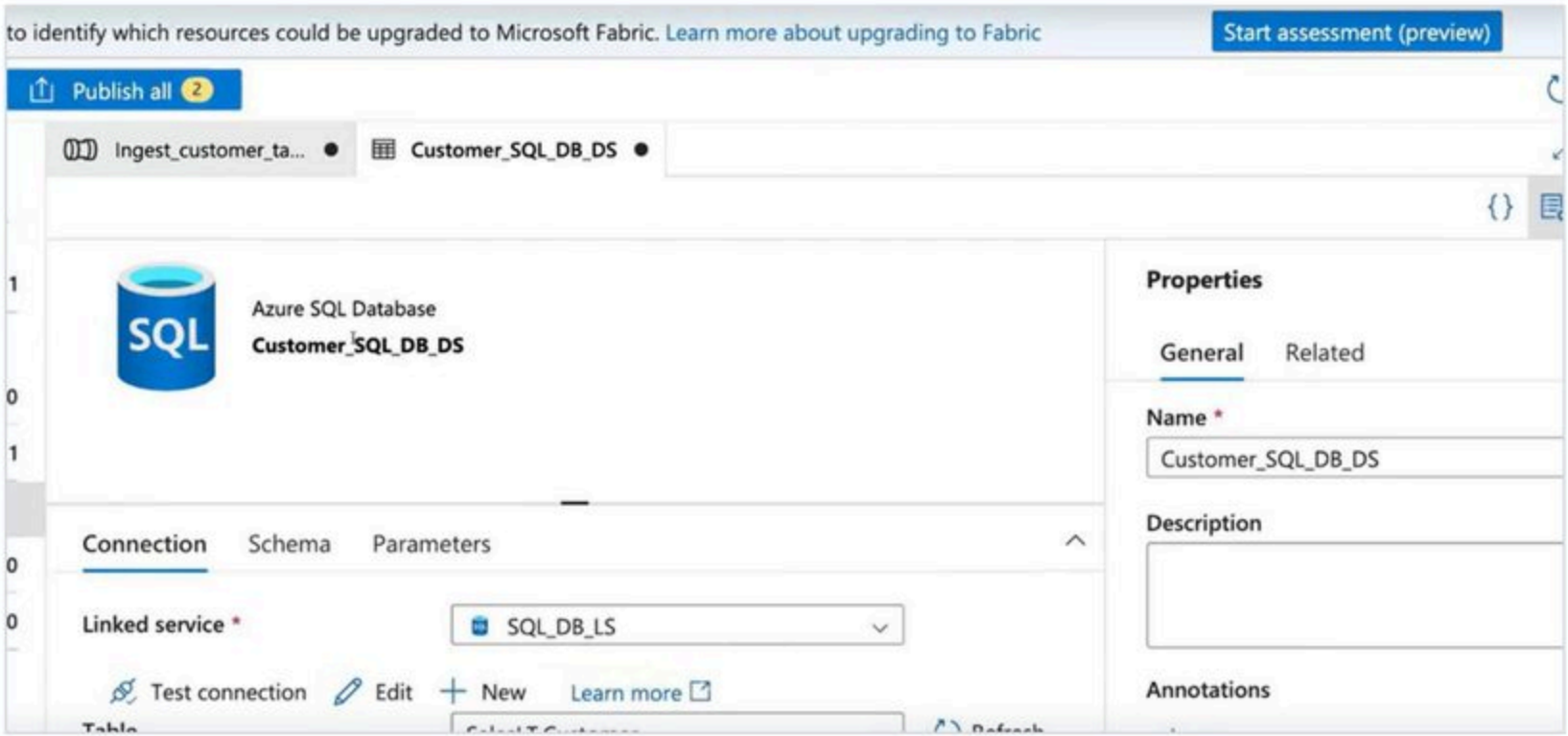




- Name: Customer_SQL_DB_DS
- Linked service: SQL_DB_LS
- Table name: SalesLT.Customer
- Import schema: From connection/store → OK

PART 5

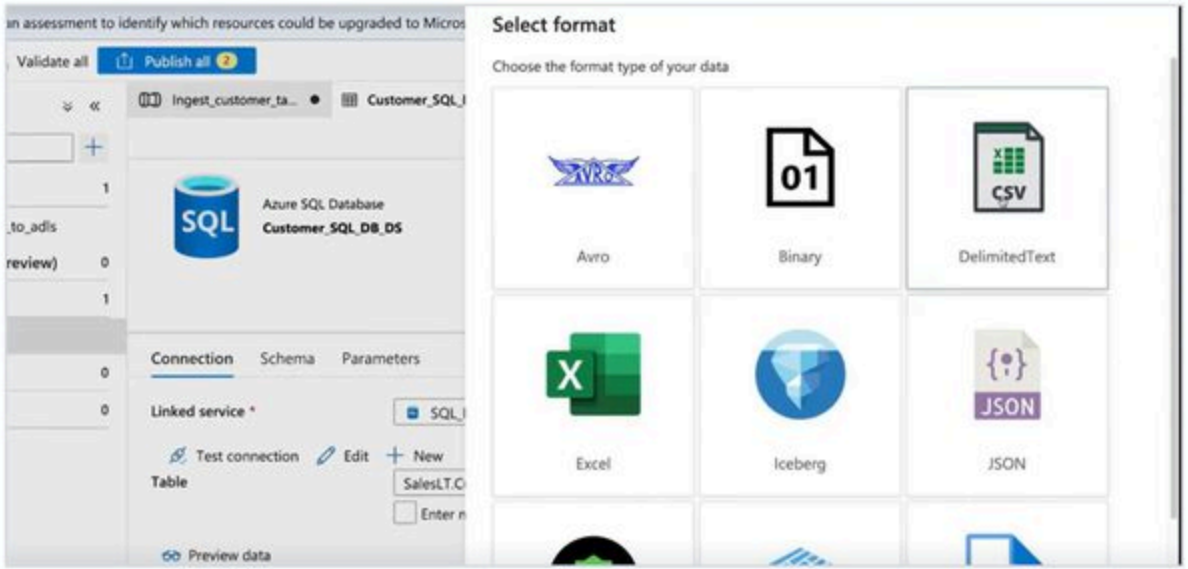
Source dataset created

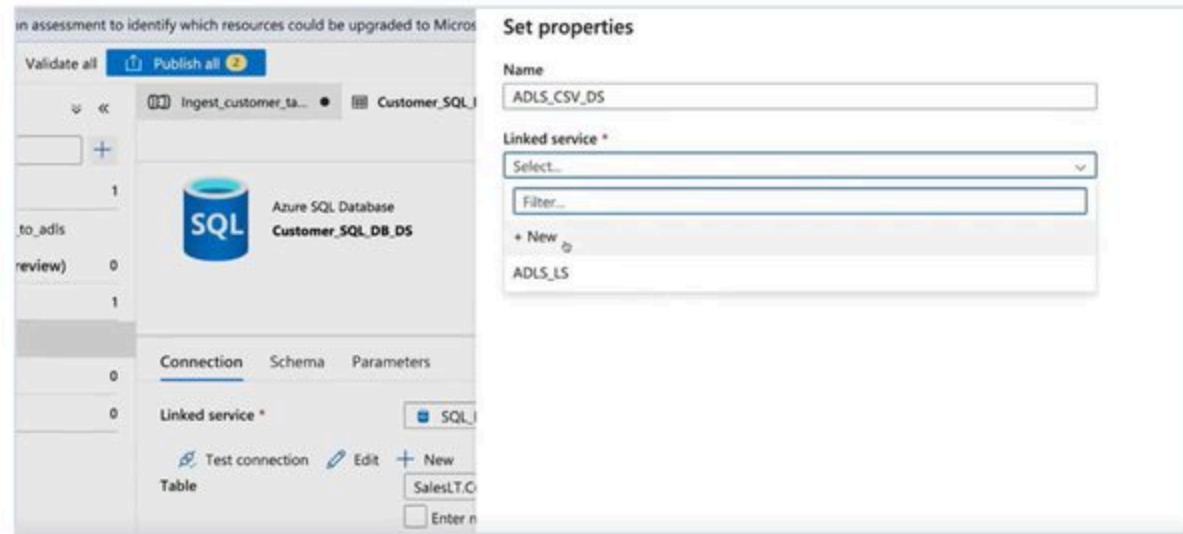


- Connection tab: linked service + table
- Schema tab: imported columns
- Preview data to check the rows

PART 5

Step 4 – Create the sink dataset (CSV)

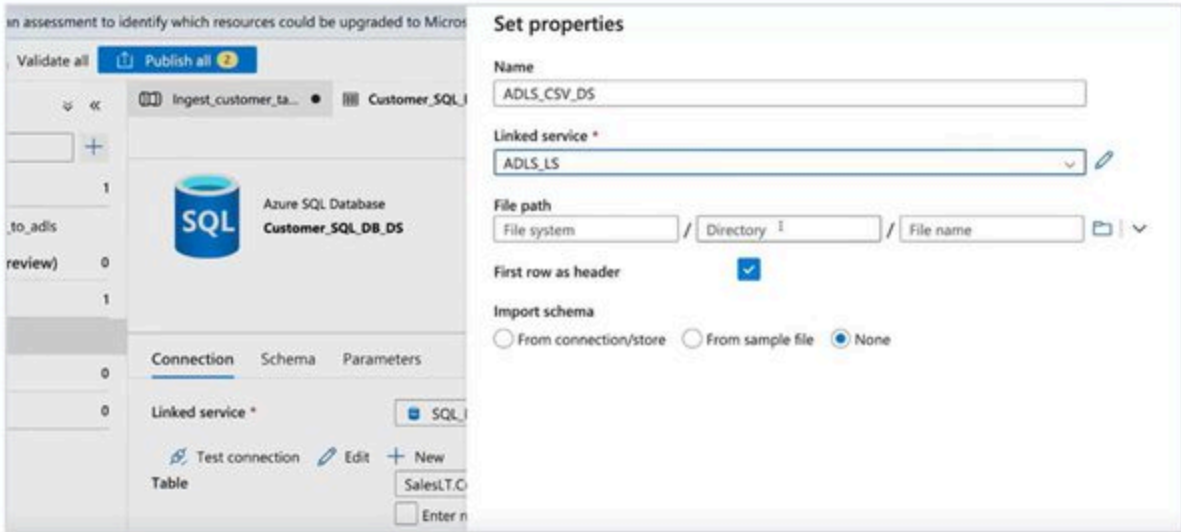


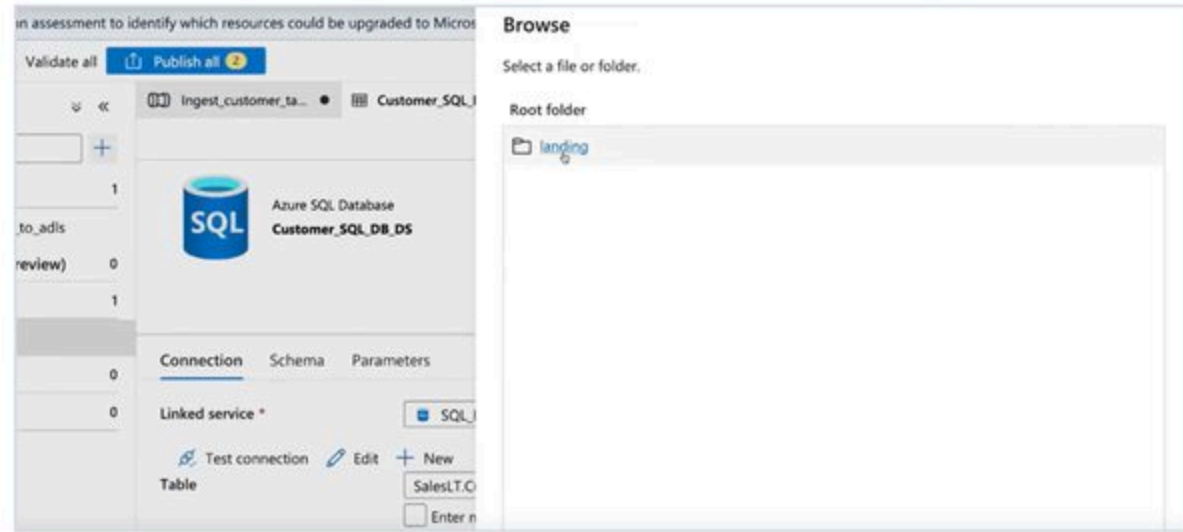


- + Dataset → Azure Data Lake Storage Gen2
- Select format: DelimitedText (CSV)
- Name: ADLS_CSV_DS; Linked service: ADLS_LS

PART 5

Step 4 – File path and header

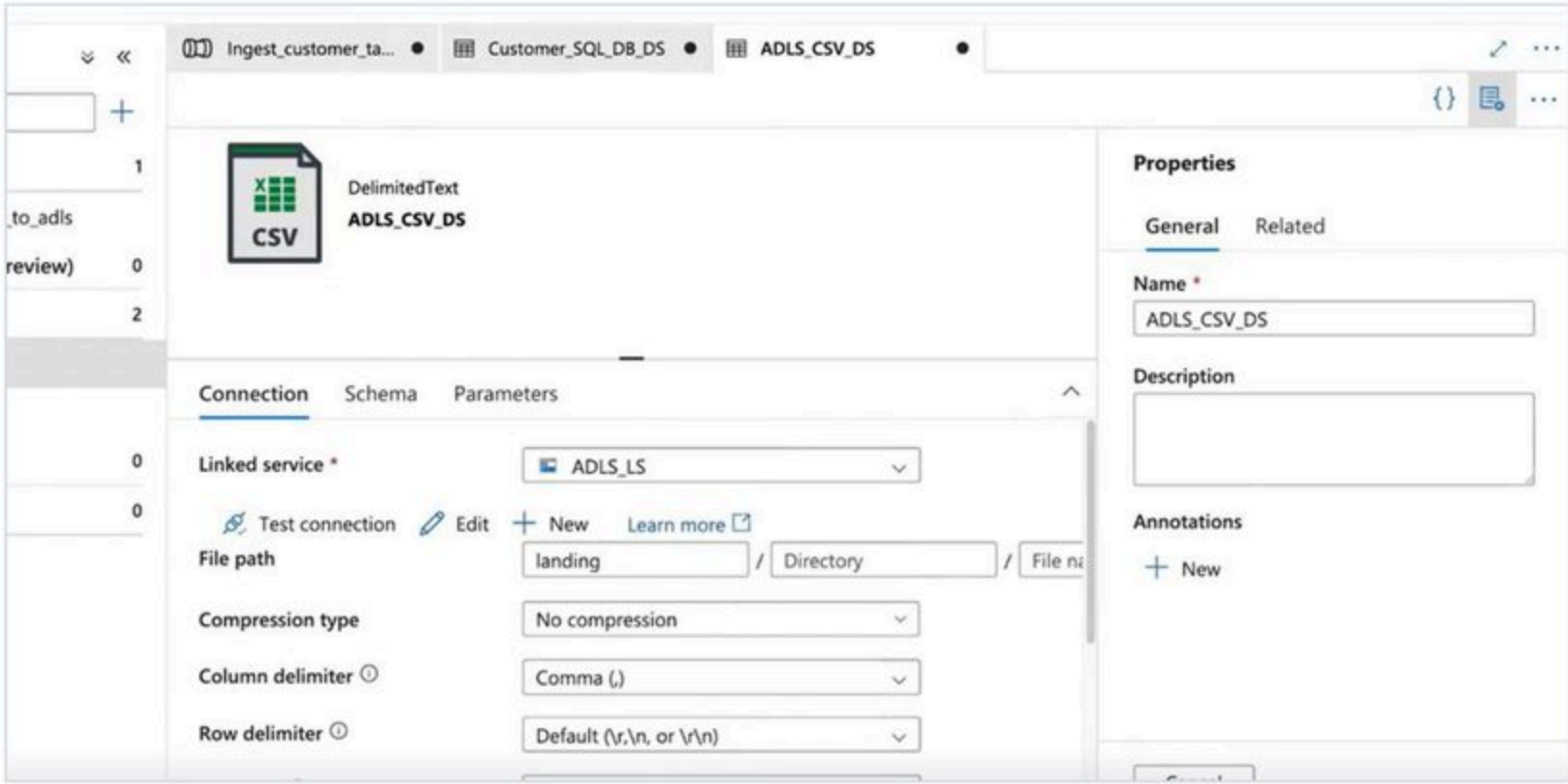




- File path: container / directory / file name
- Browse → container landing
- First row as header: ticked
- Import schema: None (file does not exist yet)

PART 5

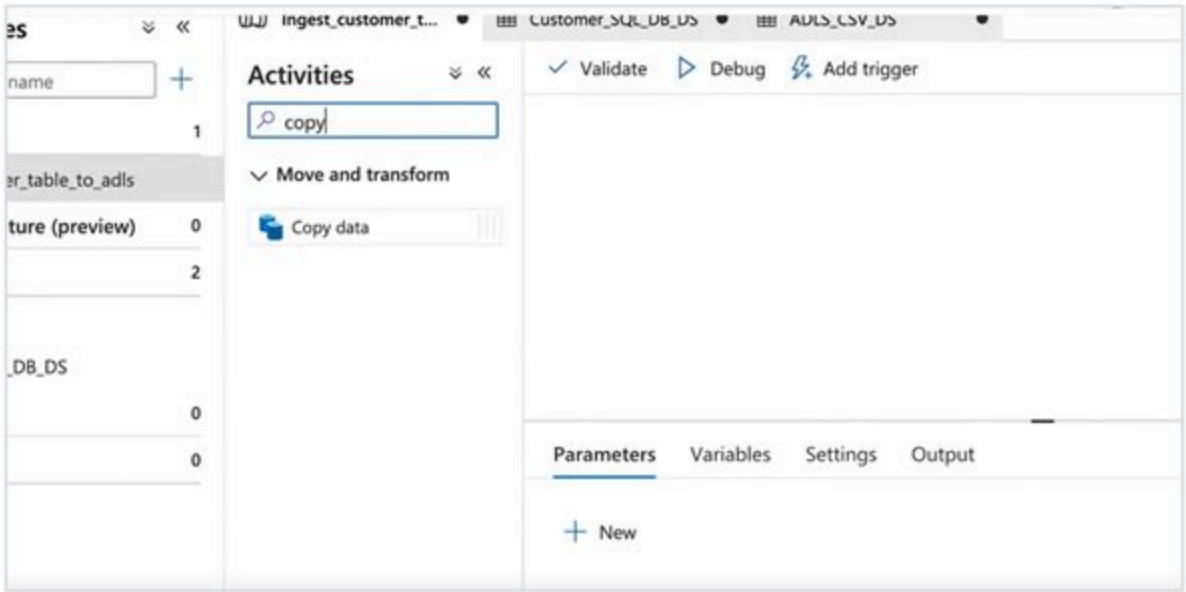
Sink dataset settings

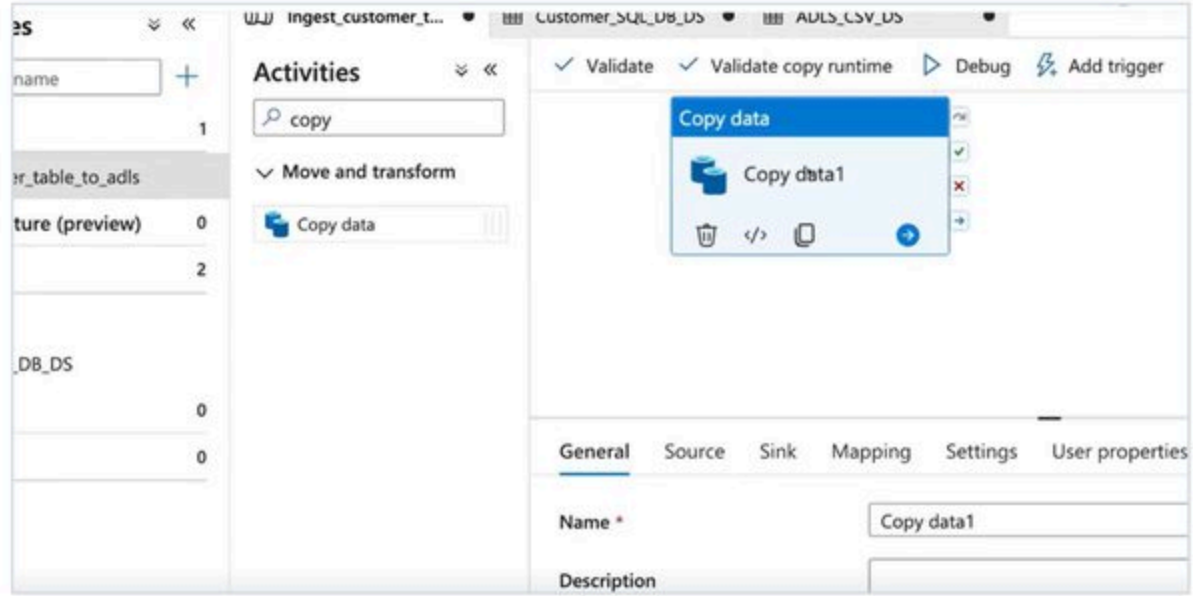


- Compression: none
- Column delimiter: comma (,)
- Row delimiter: default (\r\n or \n)
- Encoding: UTF-8

PART 5

Step 5 – Add the Copy data activity

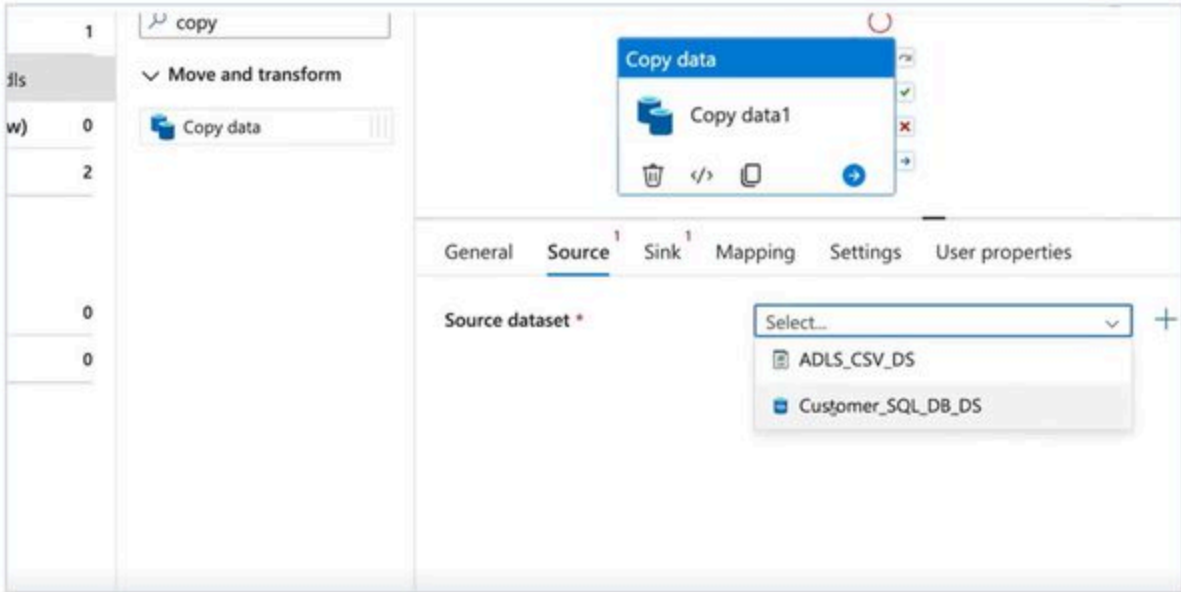


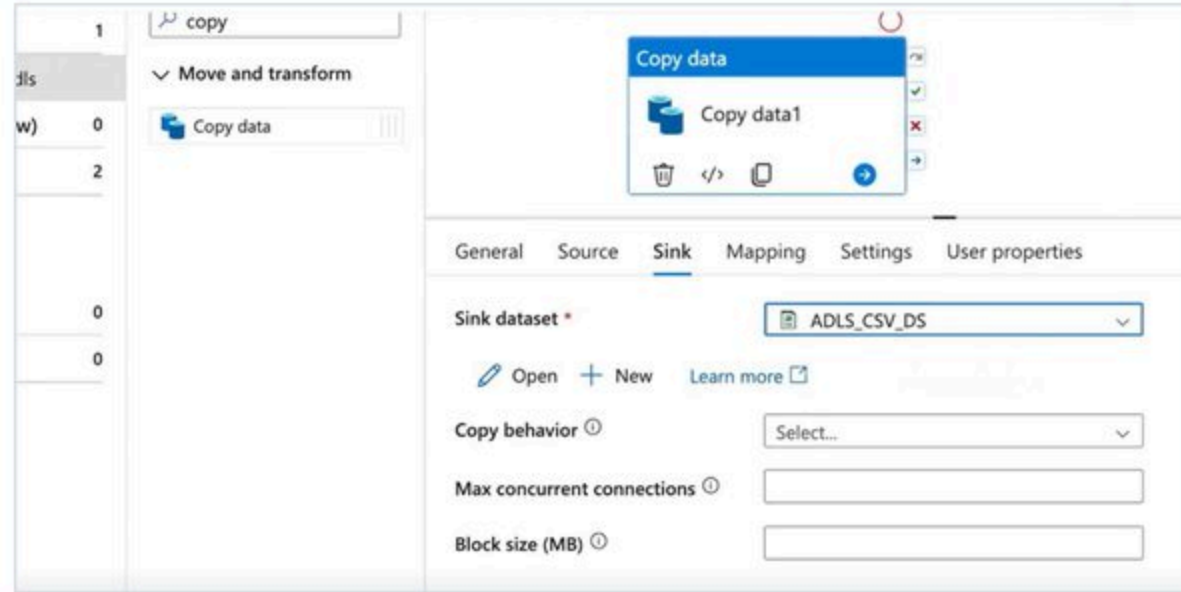


- Open the pipeline tab
- Activities → Move and transform → Copy data
- Drag onto the canvas; rename on the General tab

PART 5

Step 5 – Source and Sink tabs

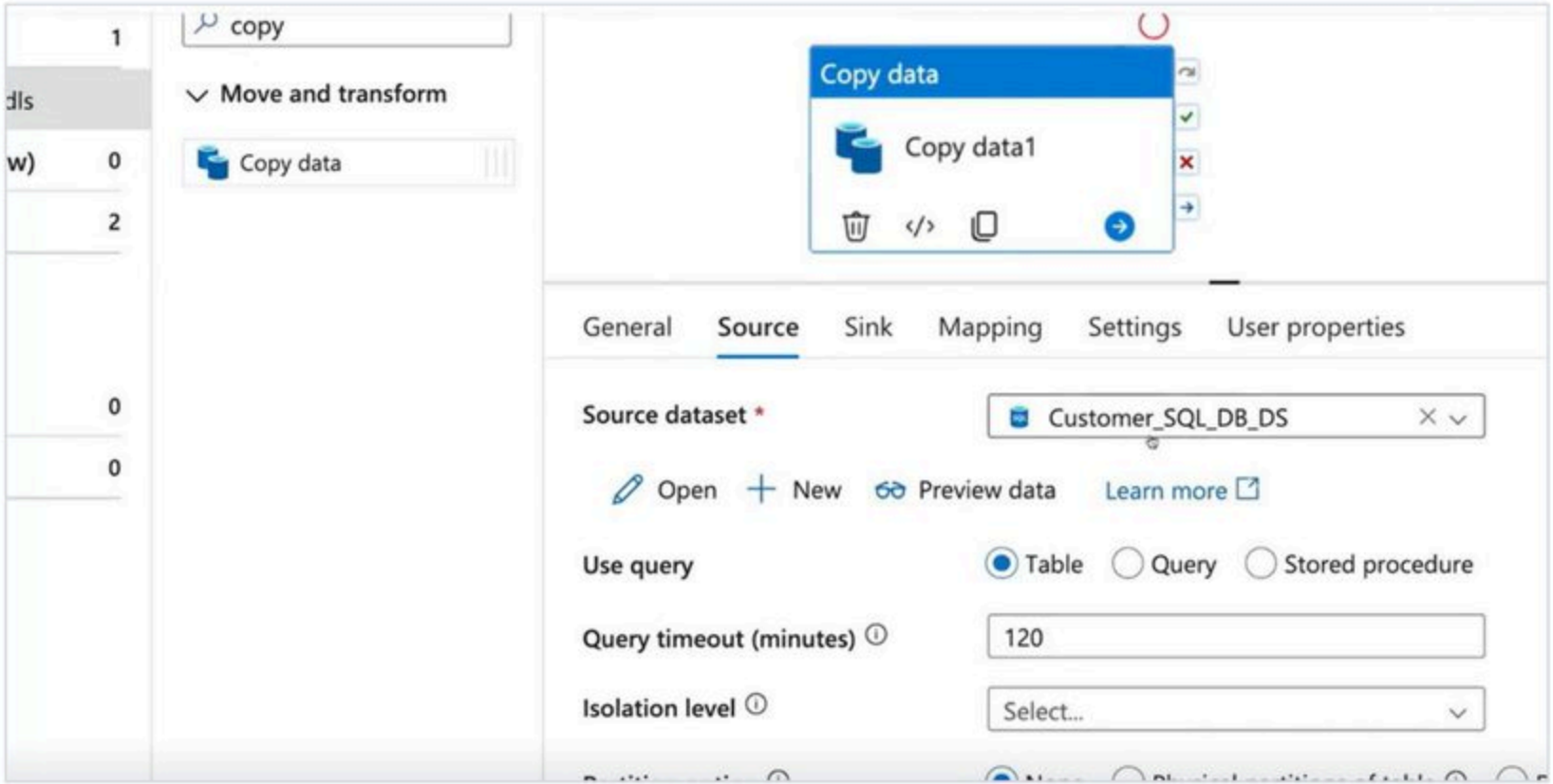




- Source dataset: Customer_SQL_DB_DS
- Sink dataset: ADLS_CSV_DS
- Red “1” markers disappear when required fields are filled

PART 5

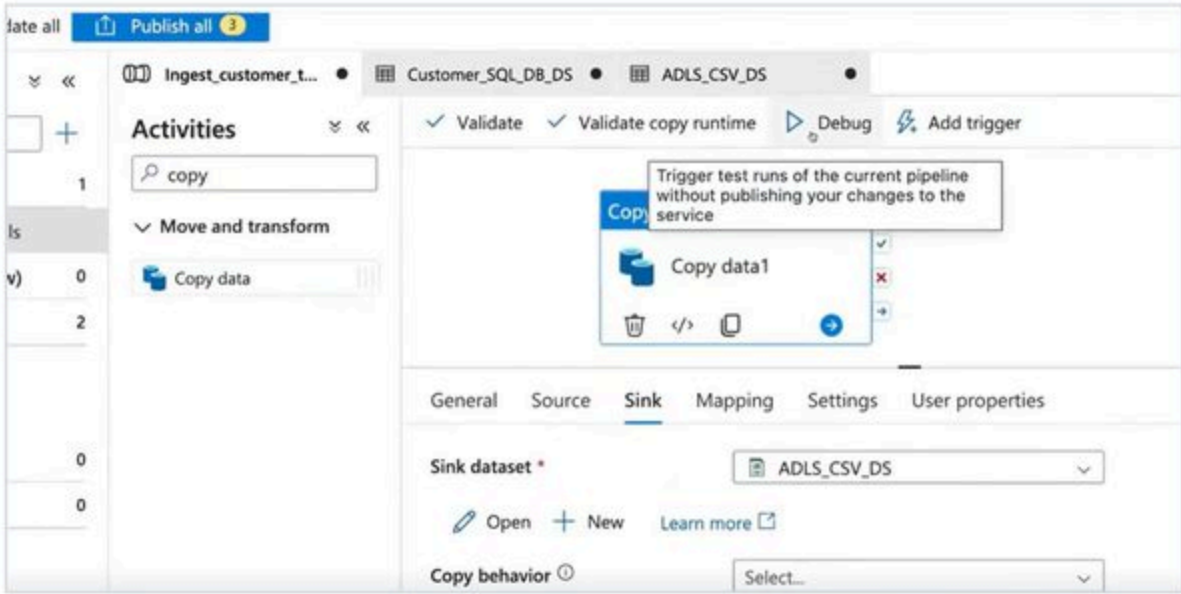
Source options worth knowing

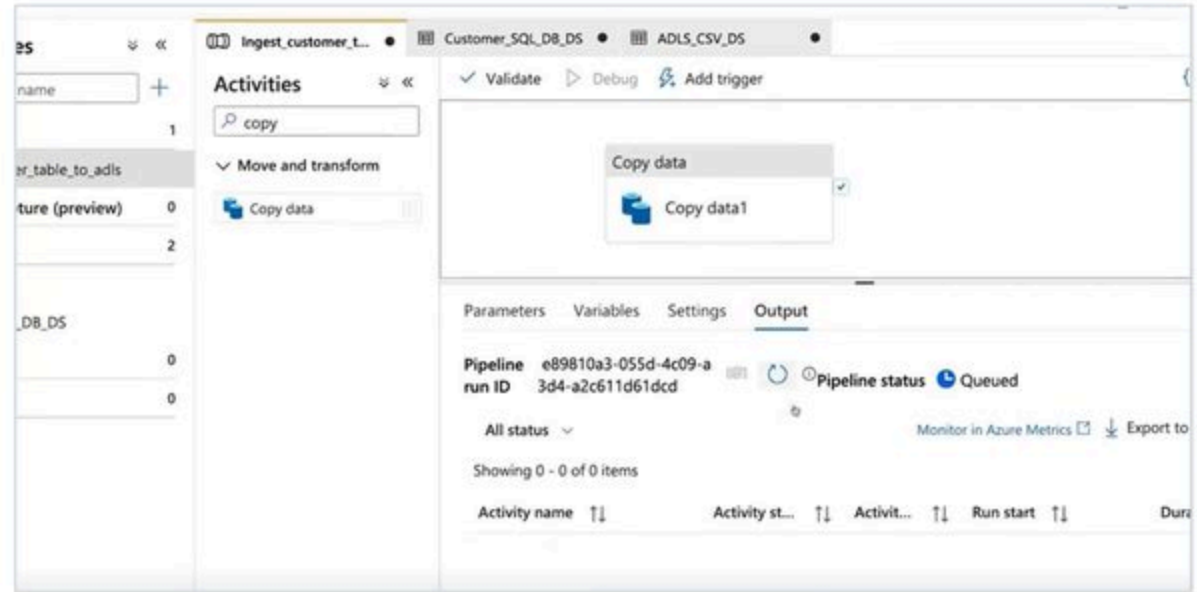


- Use query: Table / Query / Stored procedure
- Query lets you filter columns and rows (SELECT … WHERE …)
- Query timeout, isolation level, partition option

PART 5

Step 6 – Debug the pipeline

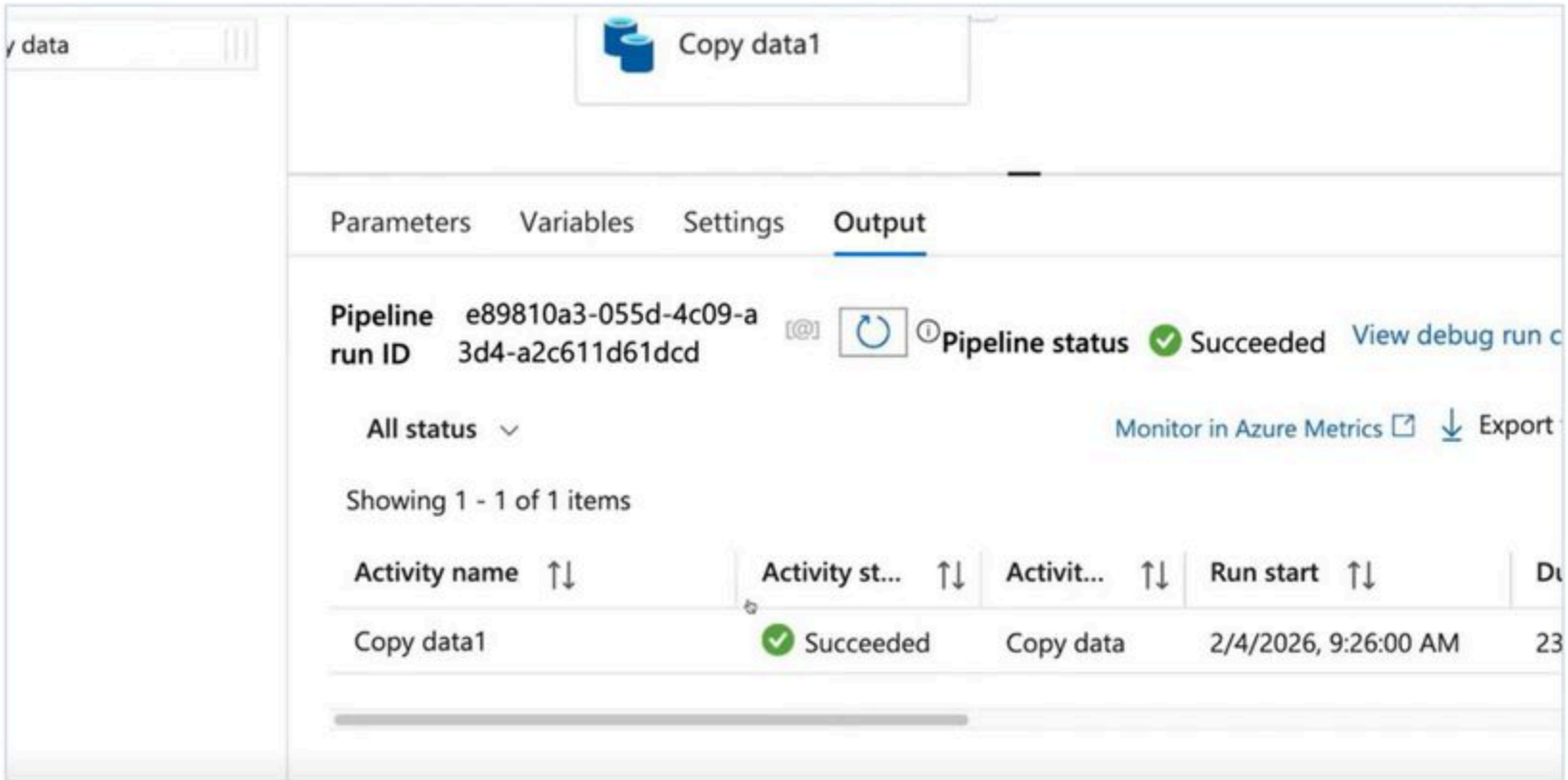




- Debug = test run without publishing
- Output tab shows pipeline run ID and status
- Queued → In progress → Succeeded

PART 5

Debug succeeded

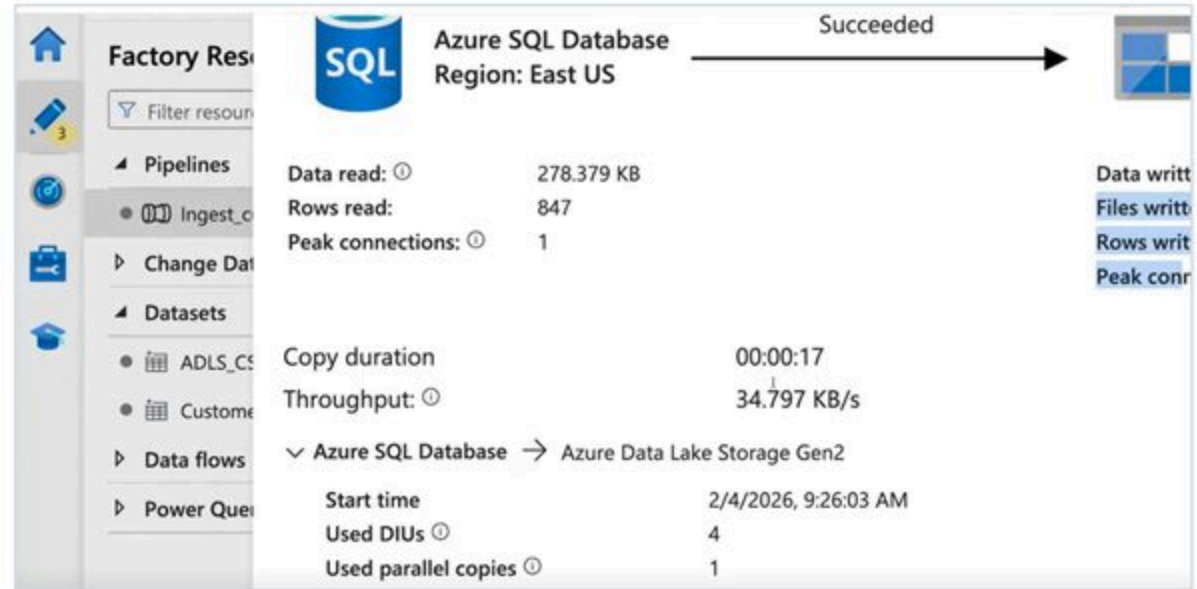


- Pipeline status: Succeeded
- Activity Copy data1: Succeeded
- Glasses icon = Details of the copy

PART 5

Copy details – what happened?



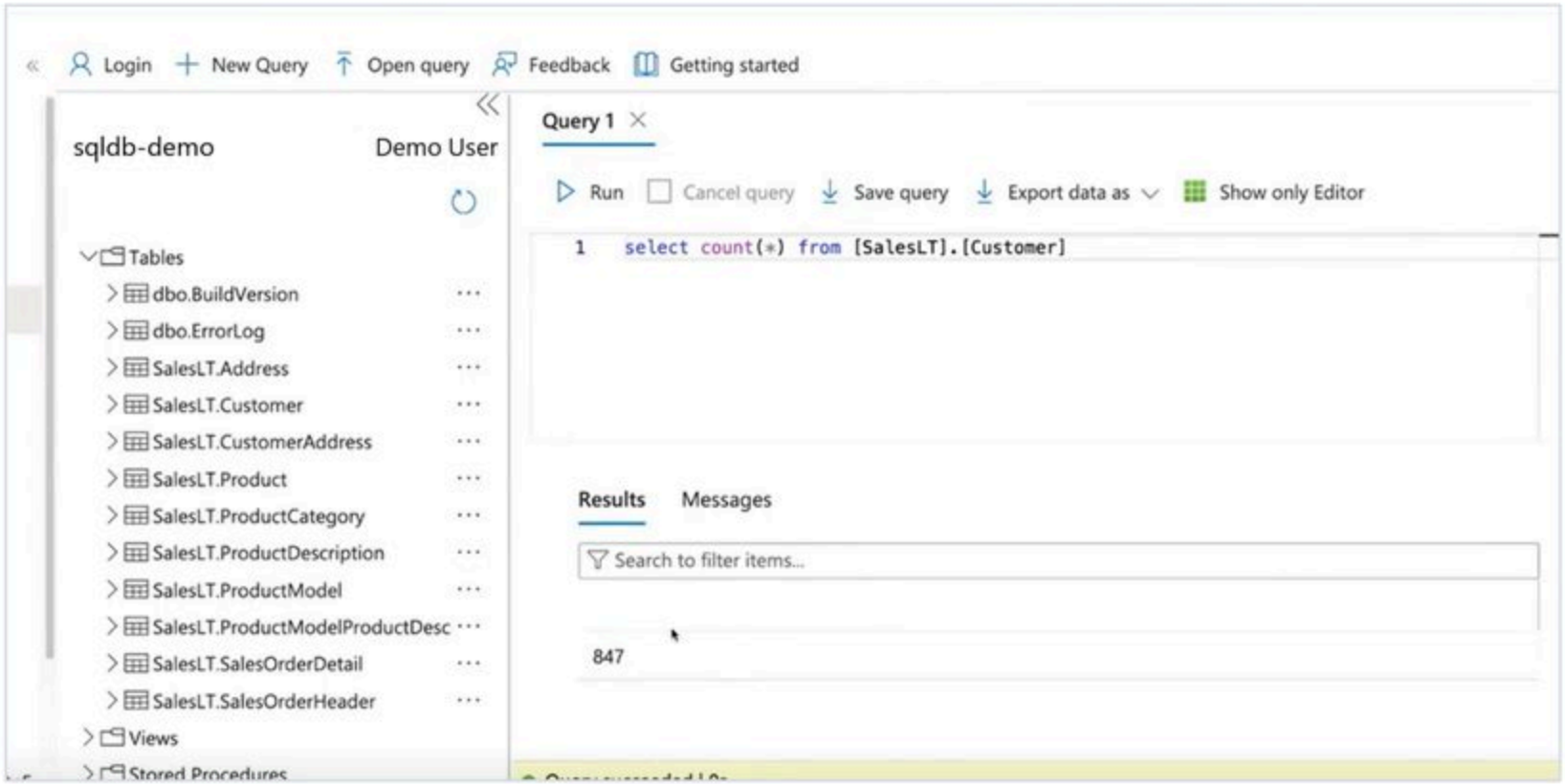


- Rows read: 847 → Rows written: 847
- Files written: 1

- Copy duration, throughput, Used DIUs: 4
- Source region vs sink region

PART 5

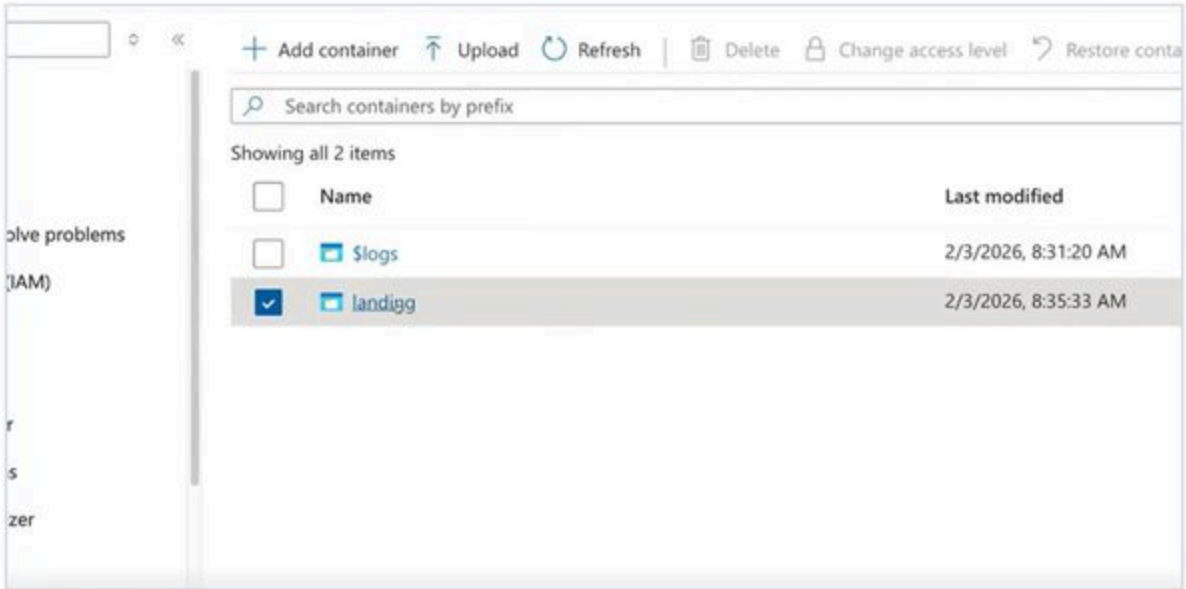
Validate in the source: row count

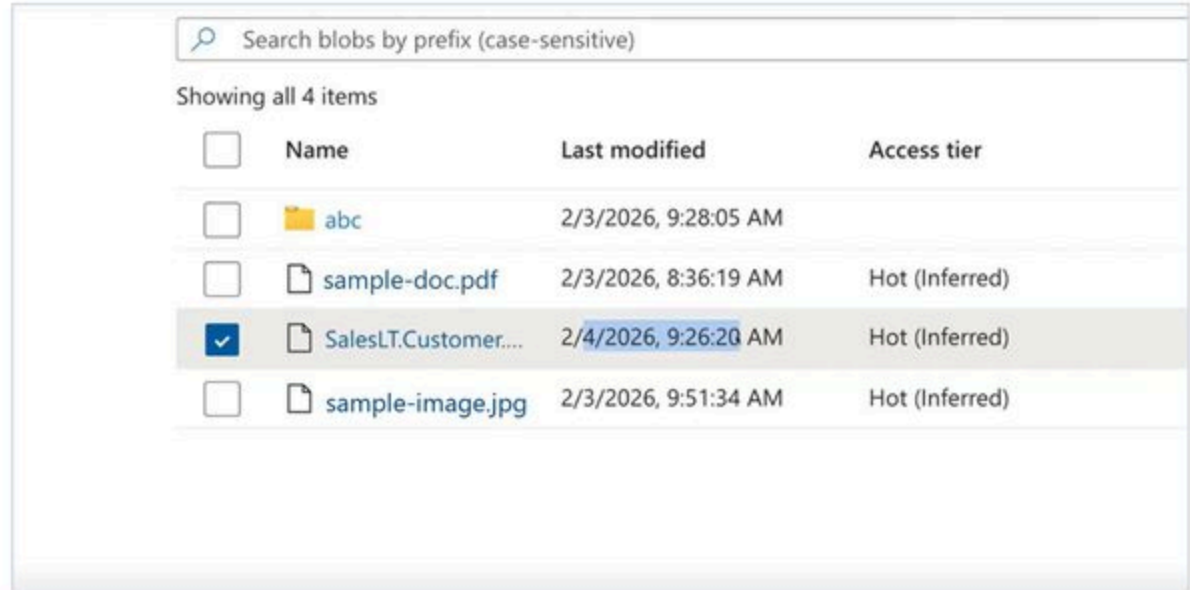


- SELECT COUNT(*) FROM [SalesLT].[Customer]
- Result: 847
- Matches rows written ✓

PART 5

Validate in ADLS: the file

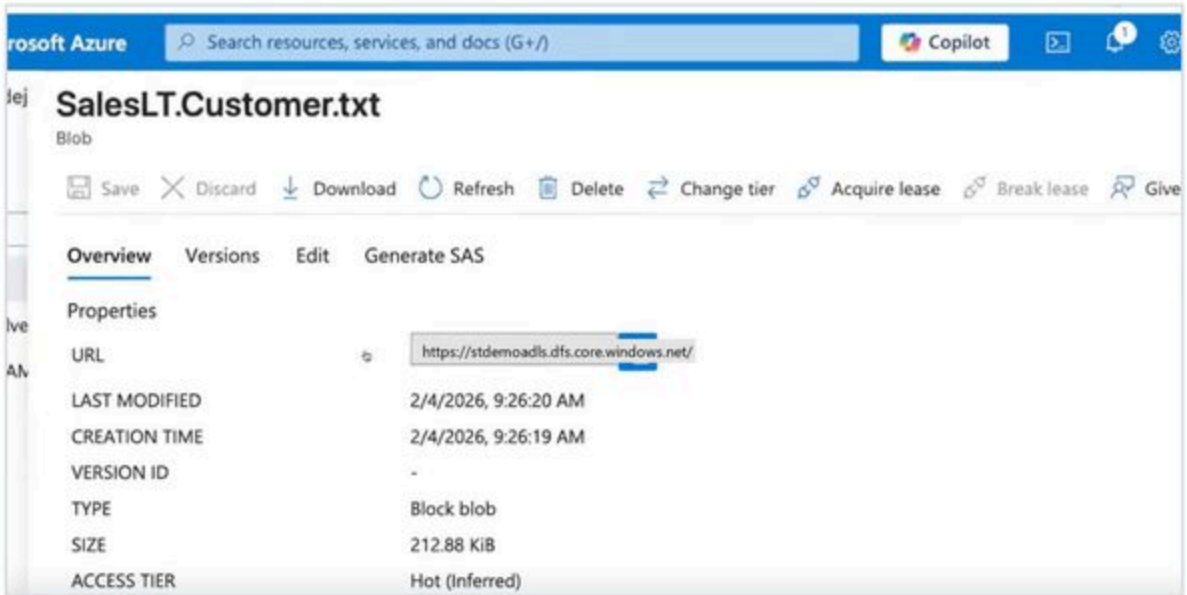


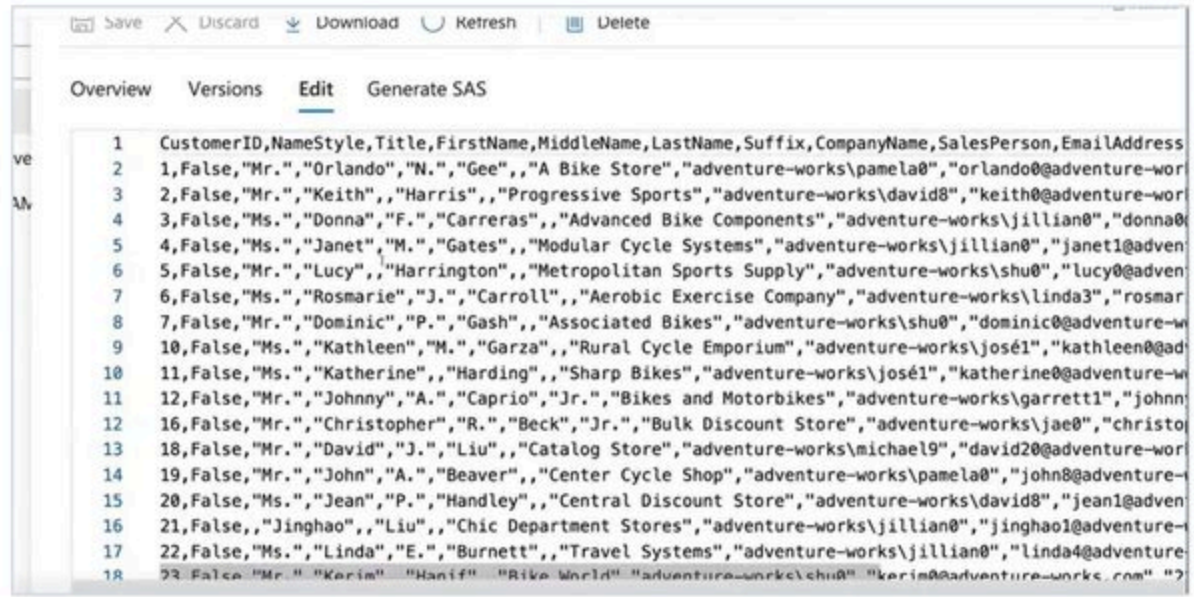


- Storage account → Containers → landing
- New file: SalesLT.Customer.txt
- File name comes from the source table name

PART 5

Open the CSV file

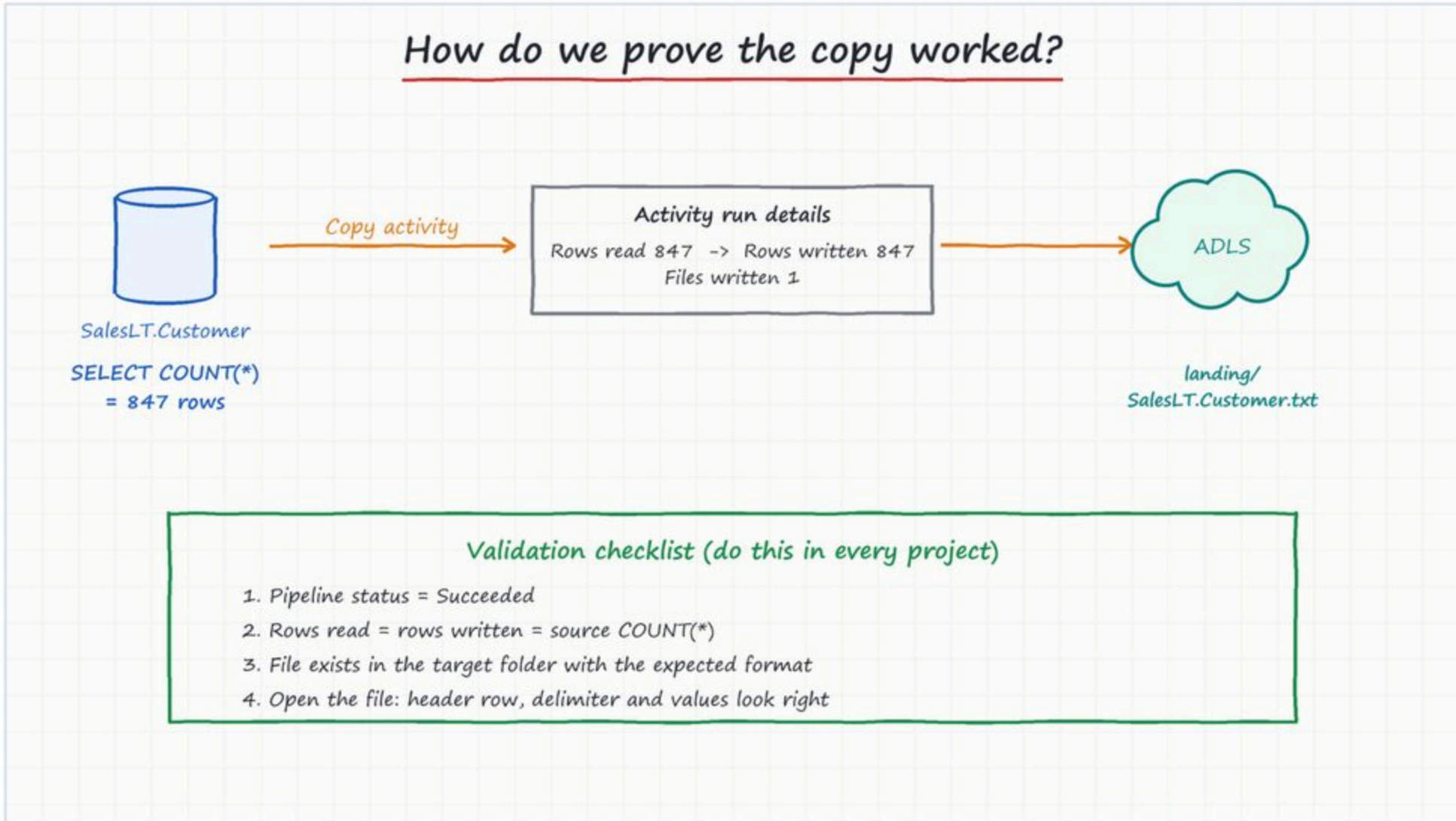




- Blob properties: URL, size, access tier
- Edit tab shows the content

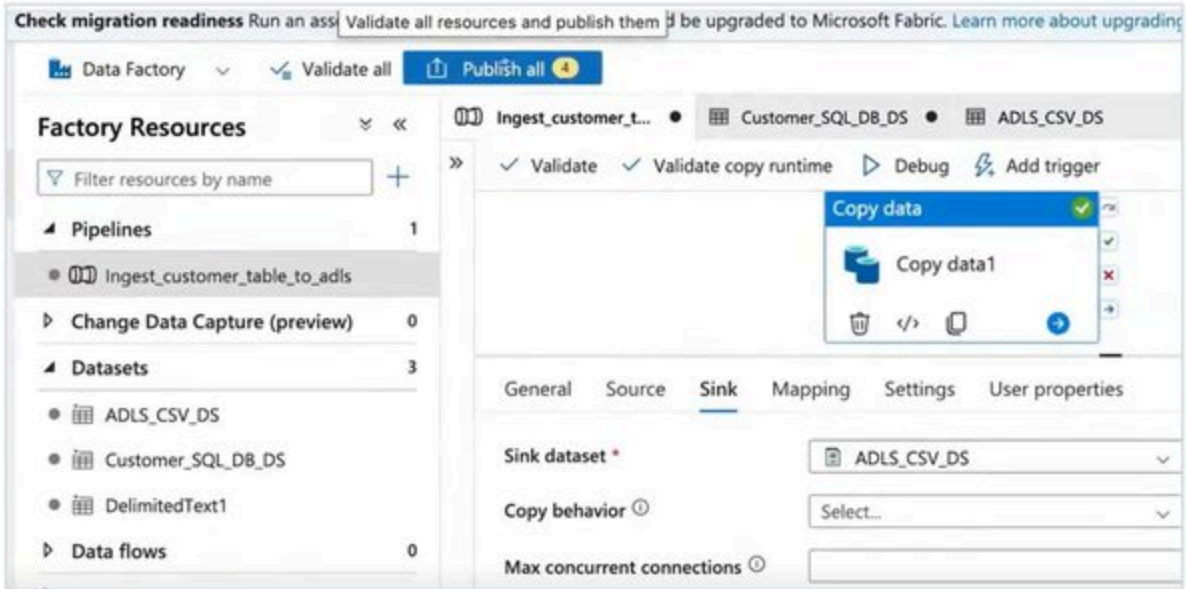
- Header row + comma-separated values ✓

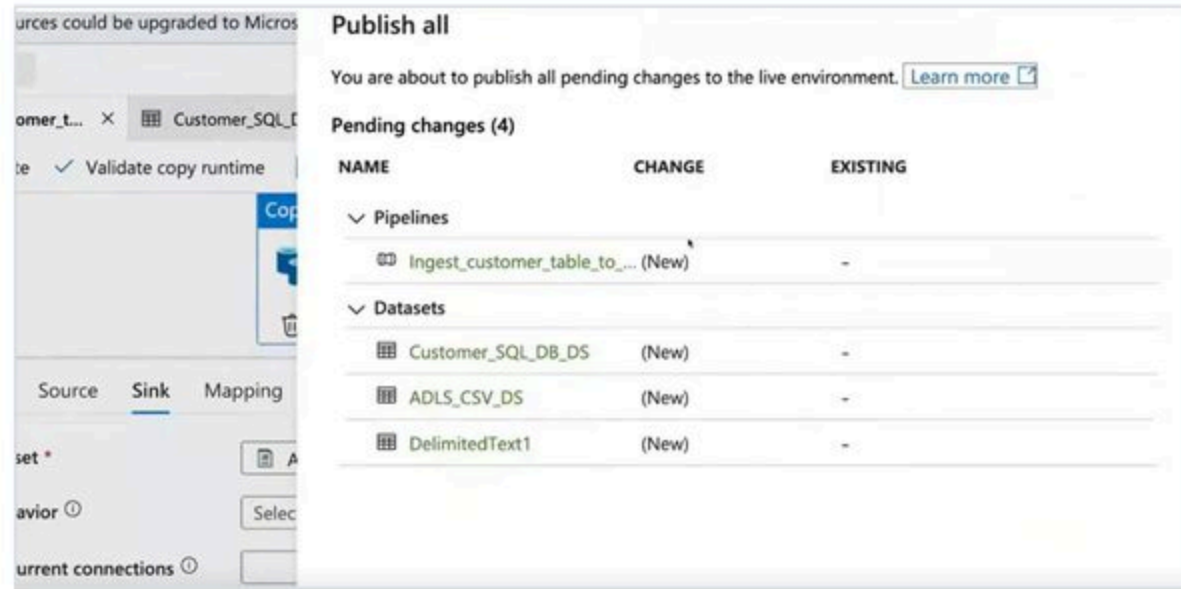
How do we prove the copy worked?



PART 5

Step 7 – Publish all



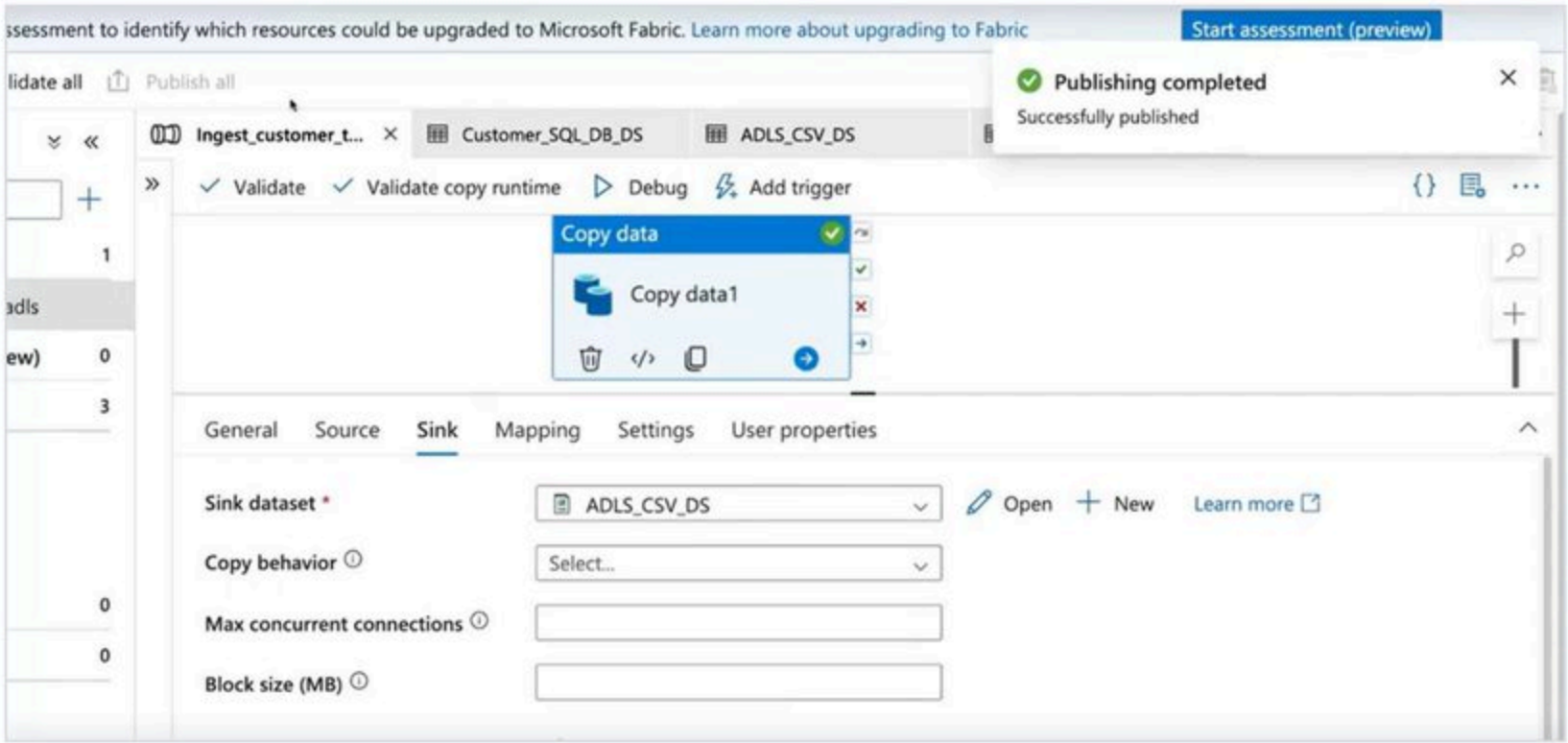


- Live mode: nothing is saved until you Publish
- Publish all → review pending changes → Publish
- Pipelines + datasets (+ linked services) are saved

⚠ Closing the browser before publishing = losing your work in Live mode.

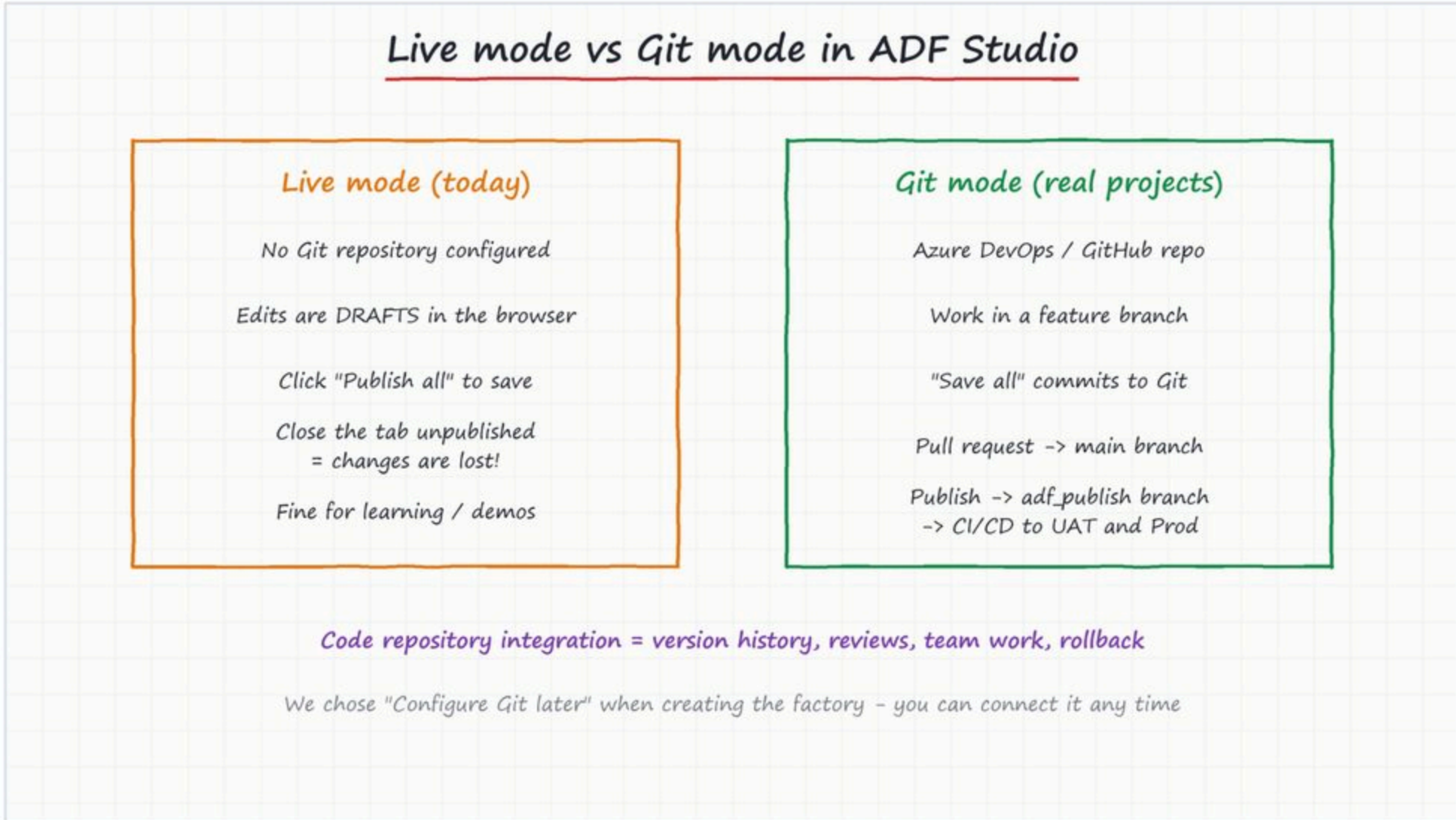
PART 5

Publishing completed



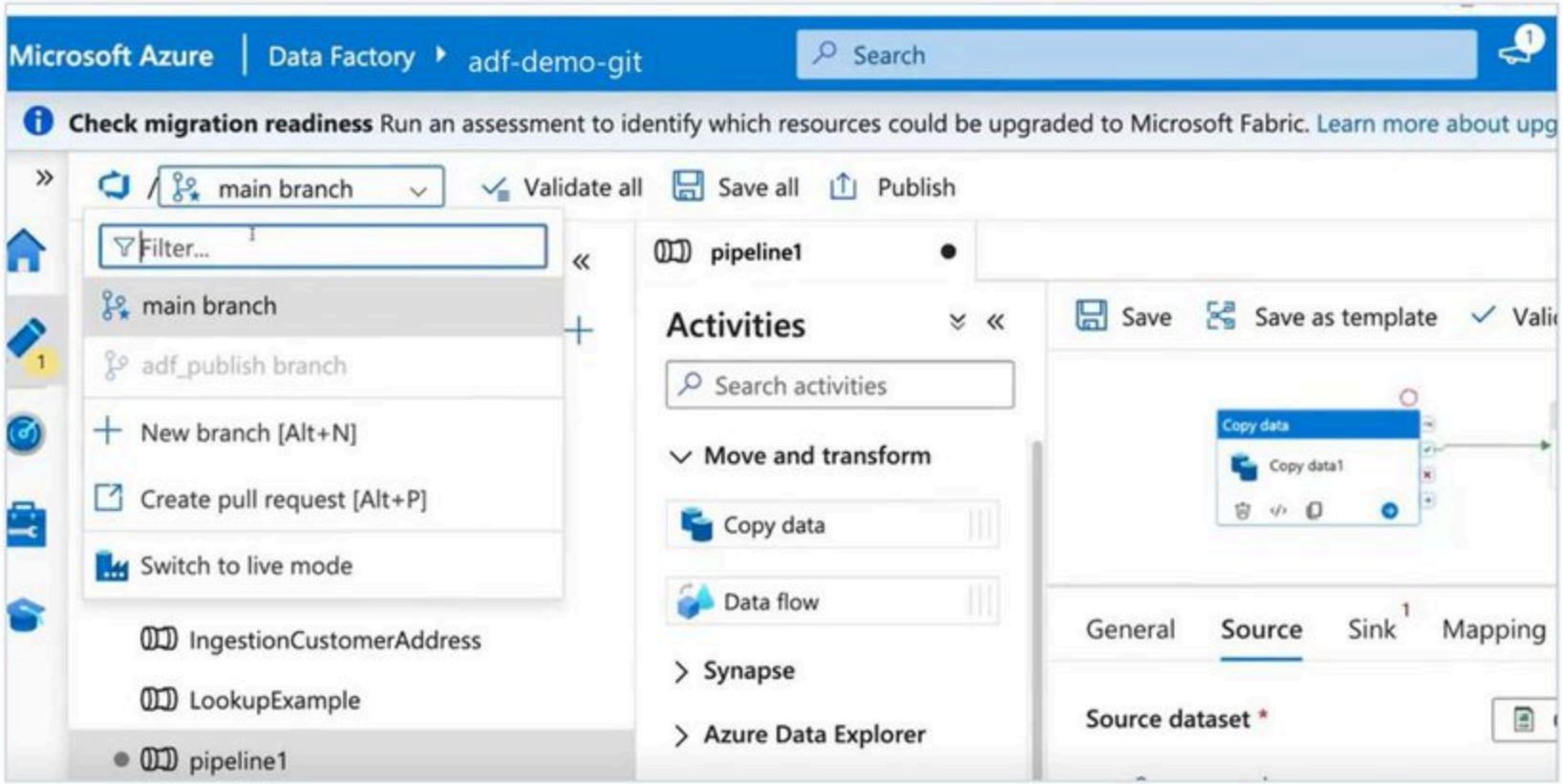
- "Publishing completed – Successfully published"
- Publish all counter returns to 0
- Rename activities to meaningful names (e.g. Customer table to ADLS CSV)

Live mode vs Git mode



PART 5

What Git mode looks like



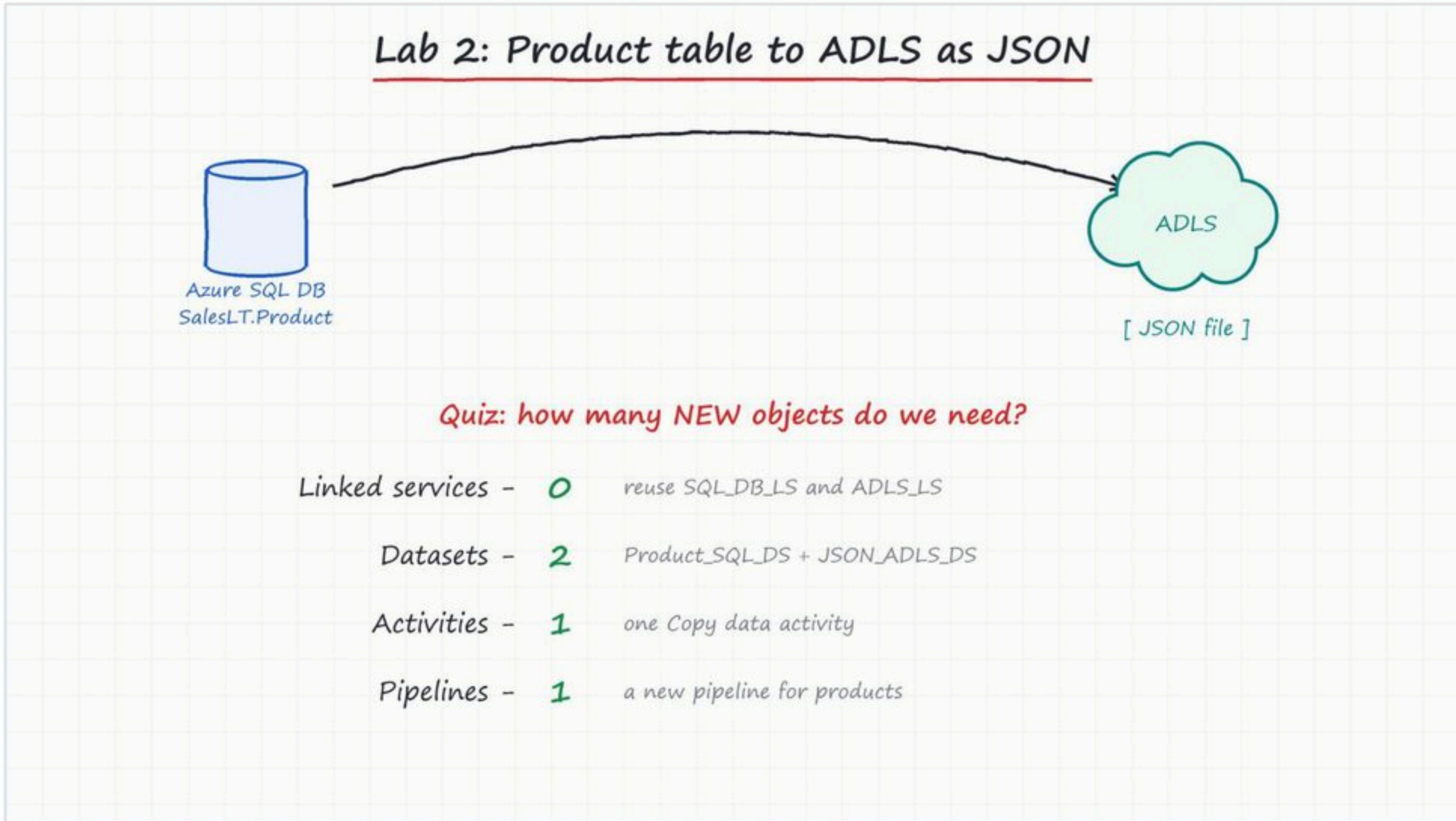
- Branch selector: main branch, adf_publish branch
- New branch, Create pull request, Switch to live mode
- Toolbar: Save all + Publish

PART 6

Lab 2: Azure SQL DB → ADLS as JSON

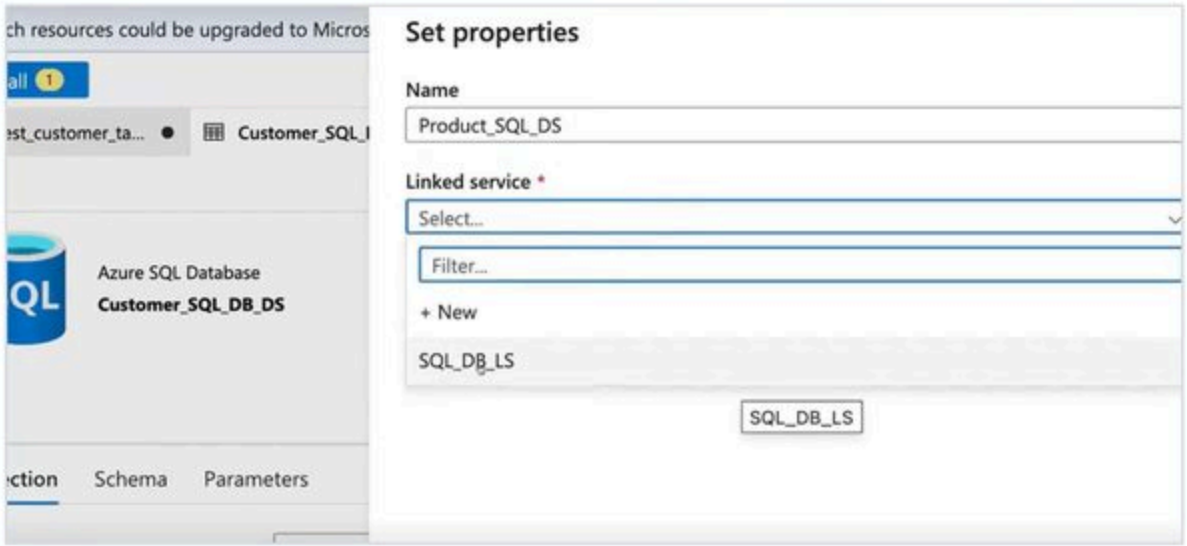
Reuse what you built – only the format changes

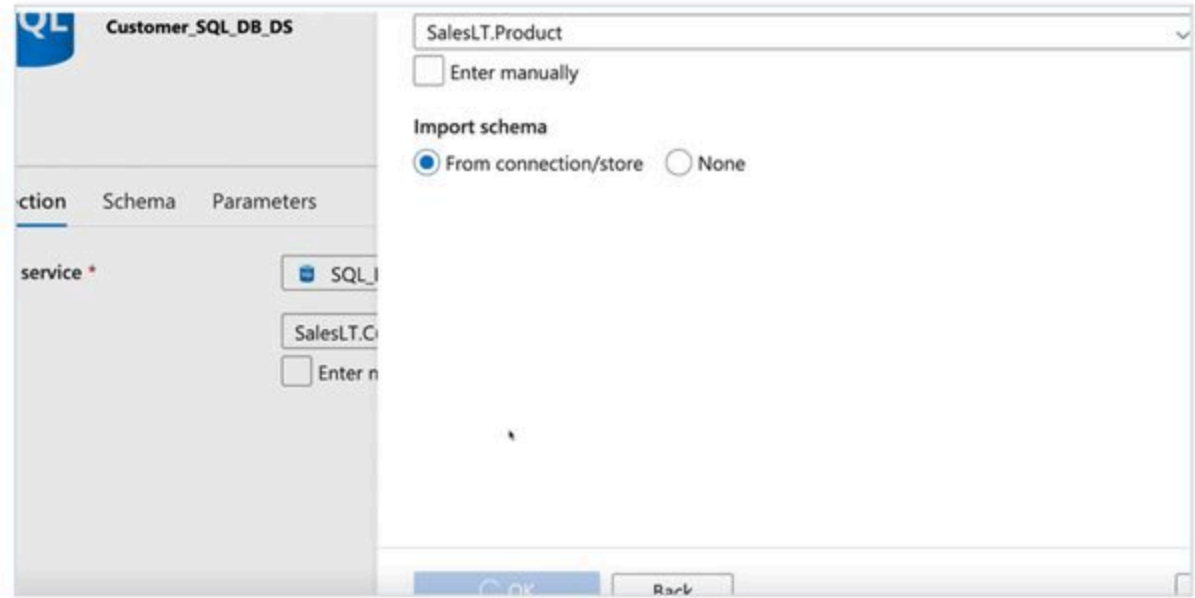
Lab 2 – how many new objects?



PART 6

Source dataset for Product

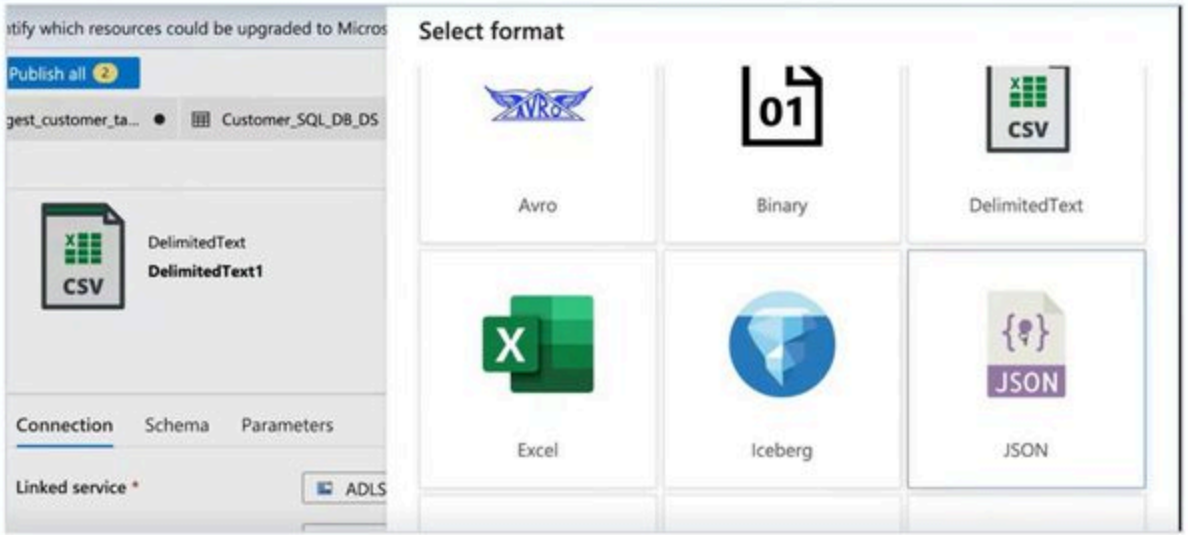


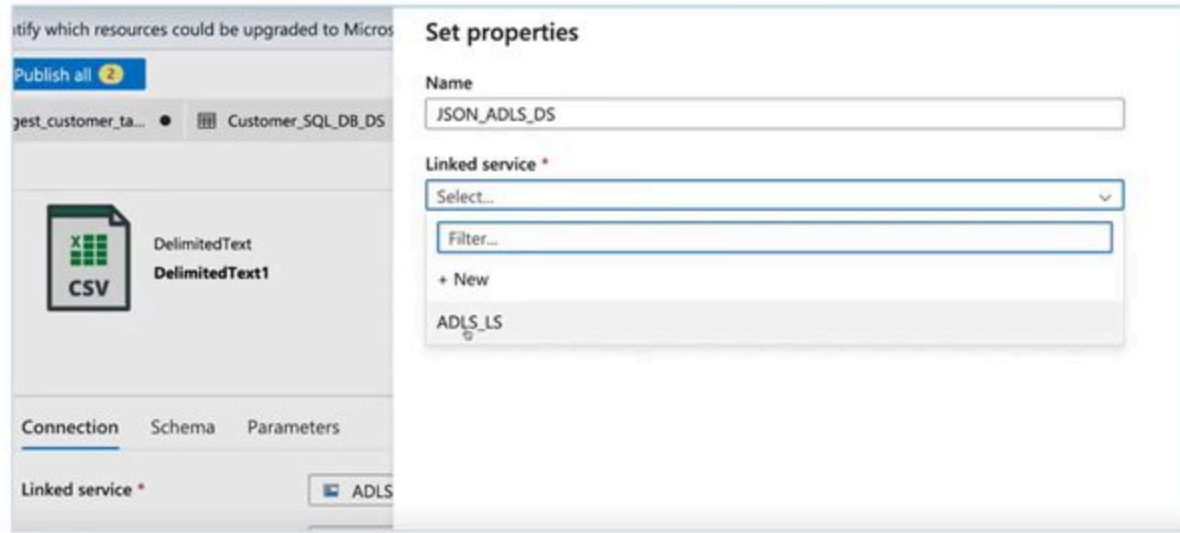


- + Dataset → Azure SQL Database
- Name: Product_SQL_DS
- Linked service: SQL_DB_LS (reused)
- Table: SalesLT.Product

PART 6

Sink dataset in JSON format

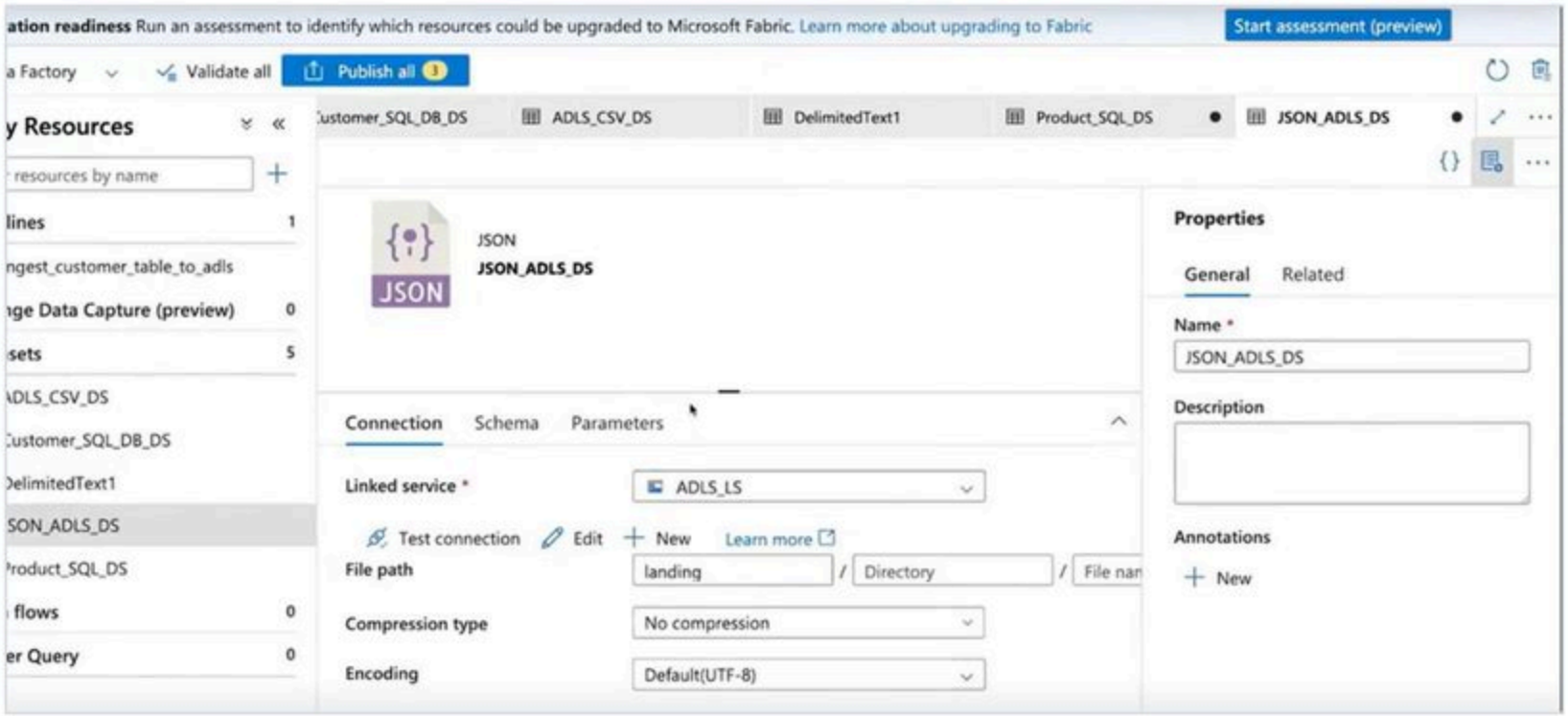




- + Dataset → ADLS Gen2 → format JSON
- Name: JSON_ADLS_DS
- Linked service: ADLS_LS (reused)

PART 6

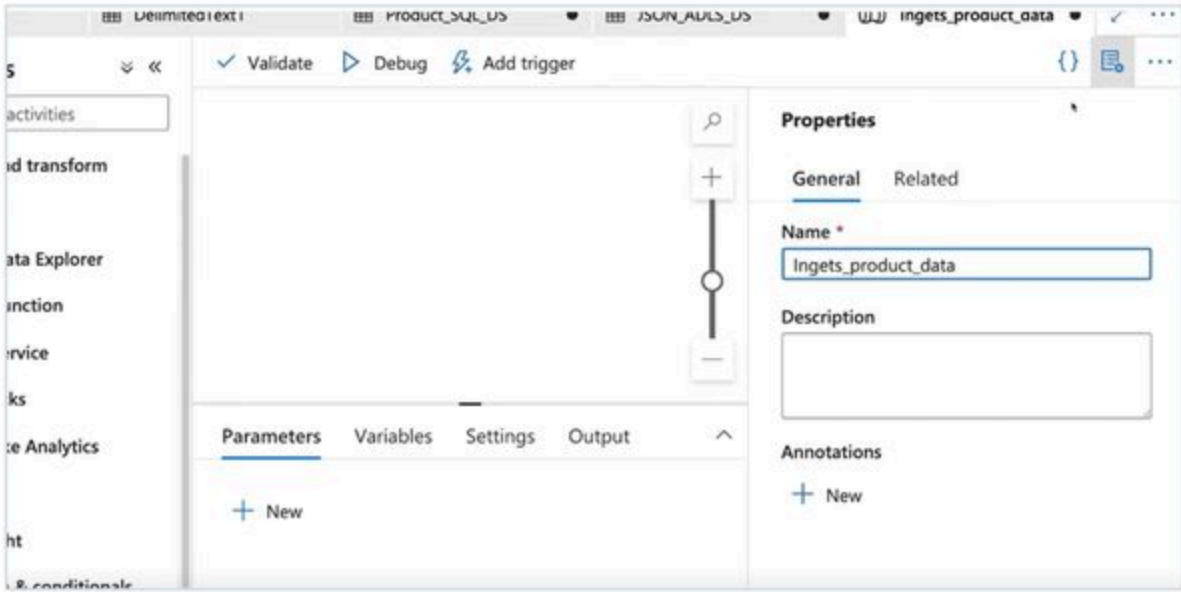
JSON dataset settings

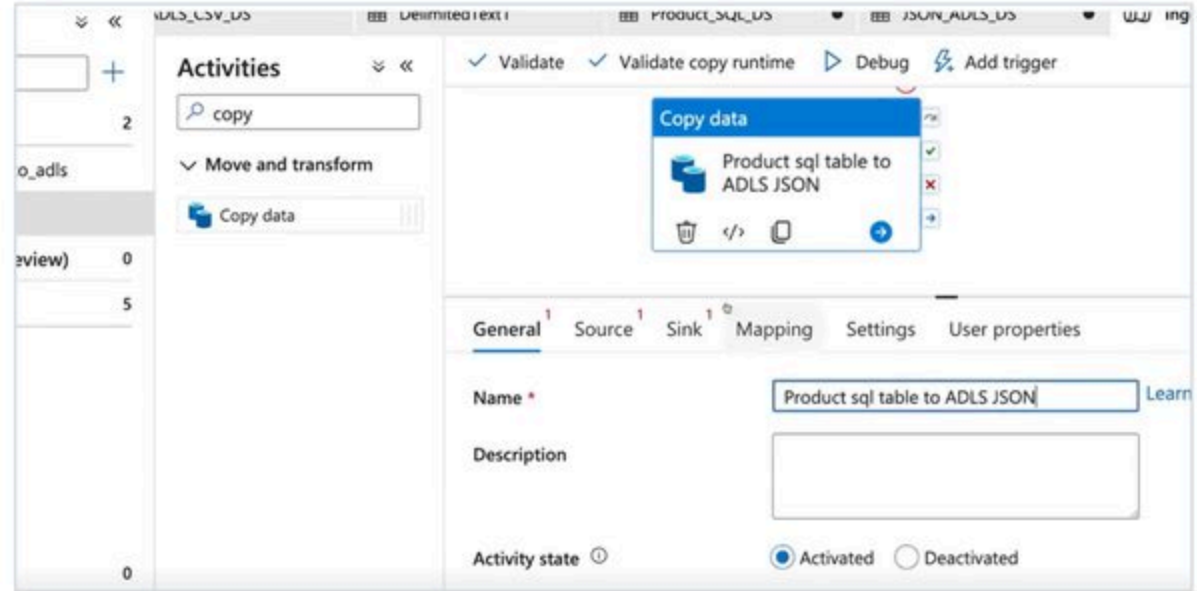


- File path: landing
- Compression: none; Encoding: UTF-8
- Factory resources now show 5 datasets

PART 6

Pipeline + Copy activity for Product



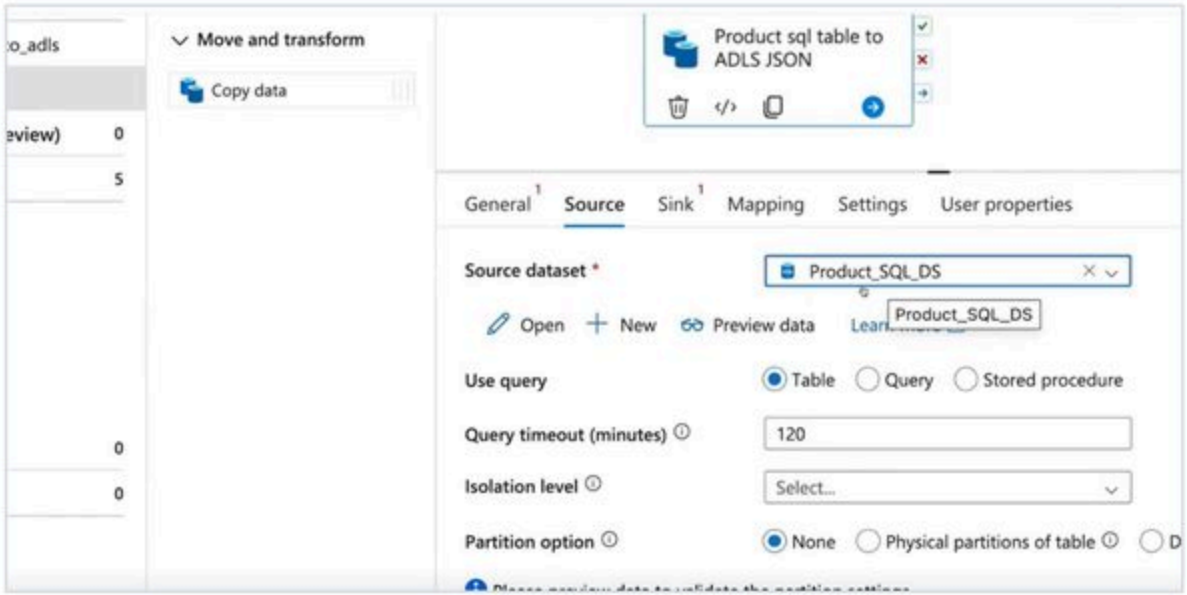


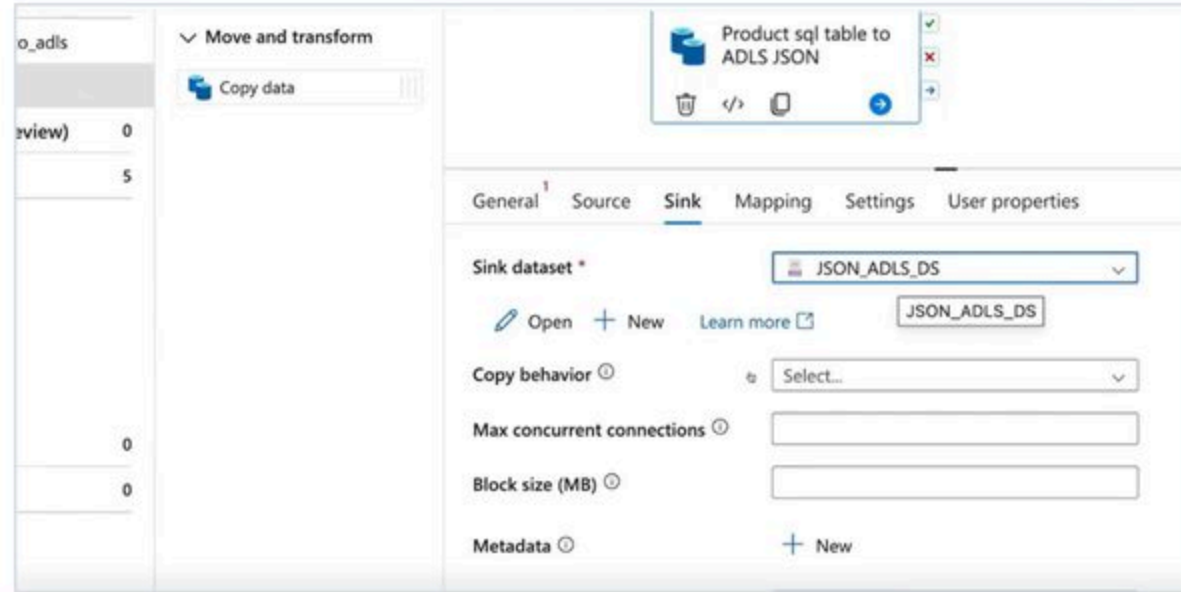
- New pipeline: Ingest_product_data
- Copy data activity: Product sql table to ADLS JSON

⚠ Check spelling in names – "Ingets_product_data" in the live demo should be Ingest_product_data.

PART 6

Source = Product_SQL_DS, Sink = JSON_ADLS_DS

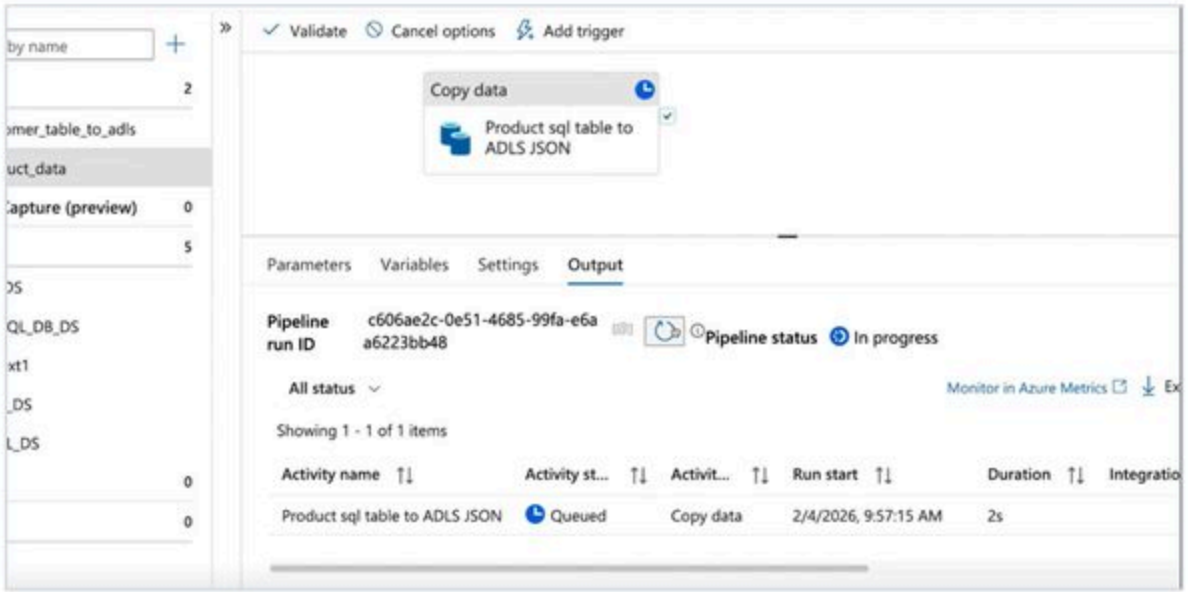


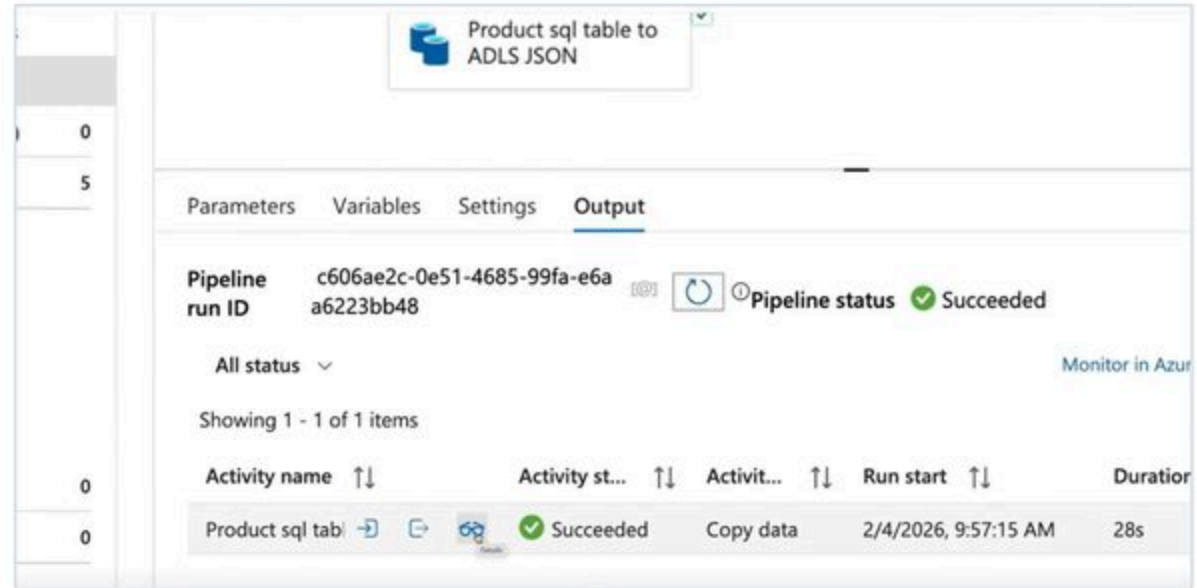


- Source tab: Product_SQL_DS, Use query = Table
- Sink tab: JSON_ADLS_DS

PART 6

Debug and monitor

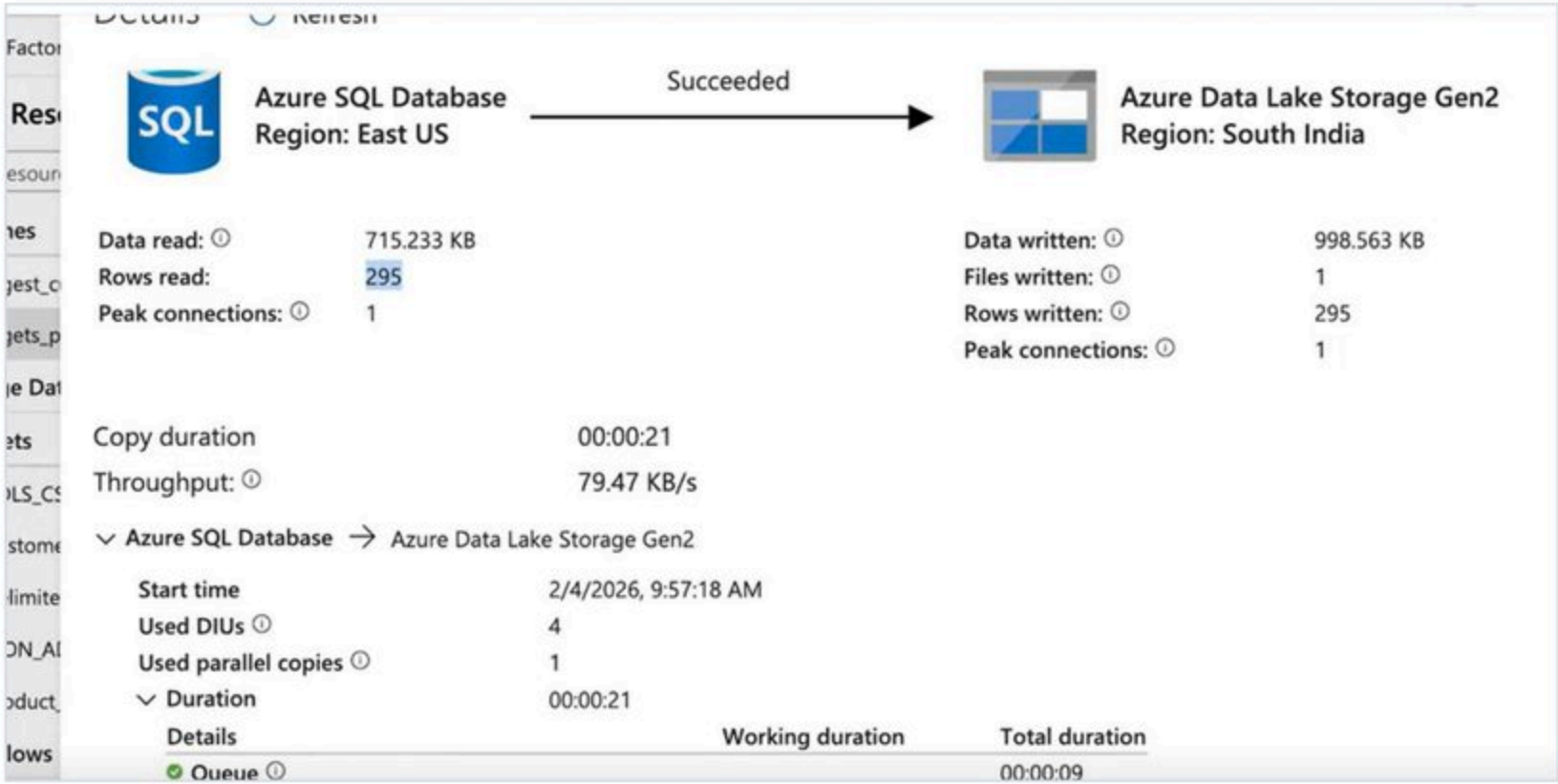




- Debug → Queued → In progress → Succeeded
- Duration ~30 seconds

PART 6

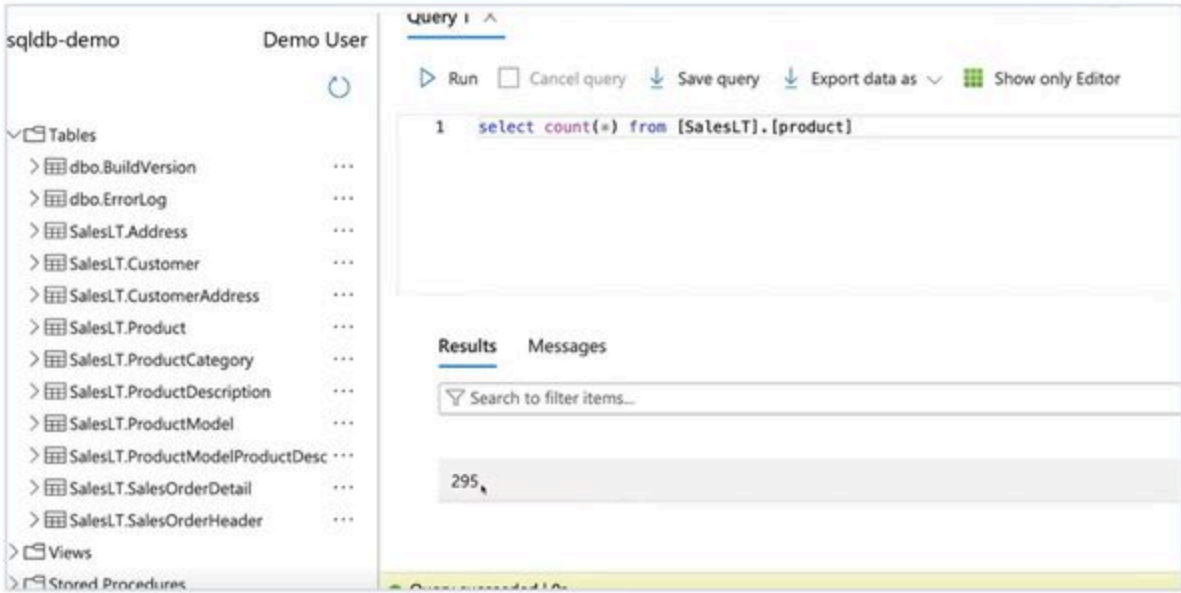
Copy details for the JSON load

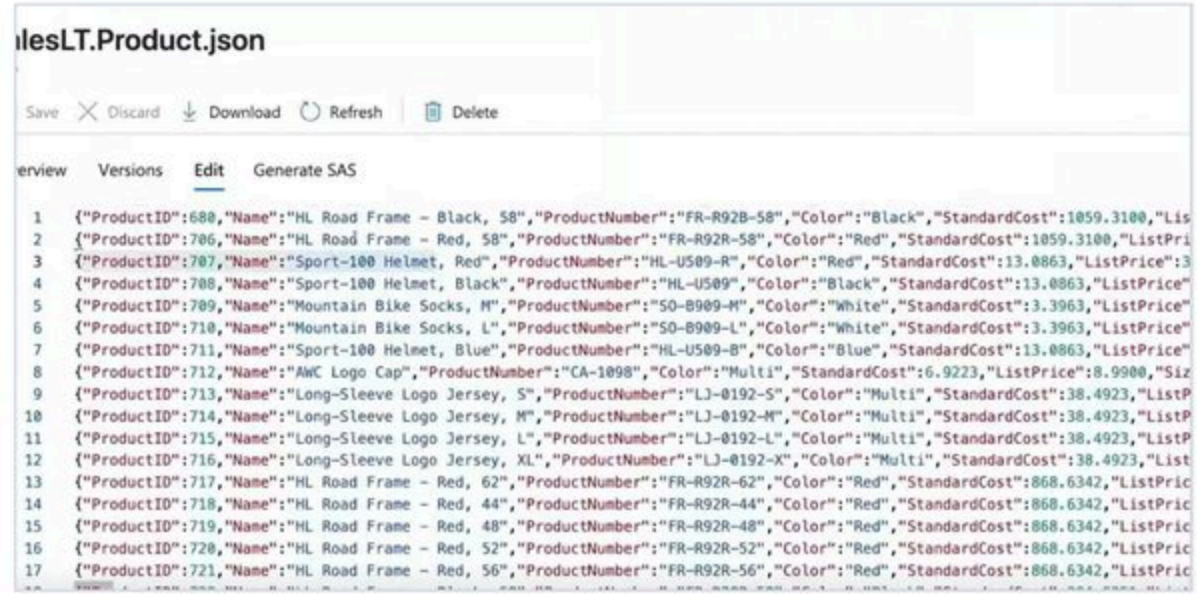


- Rows read 295 = rows written 295
- Data read ~715 KB, data written ~999 KB
- JSON is bigger: every row repeats the column names

PART 6

Validate the JSON output



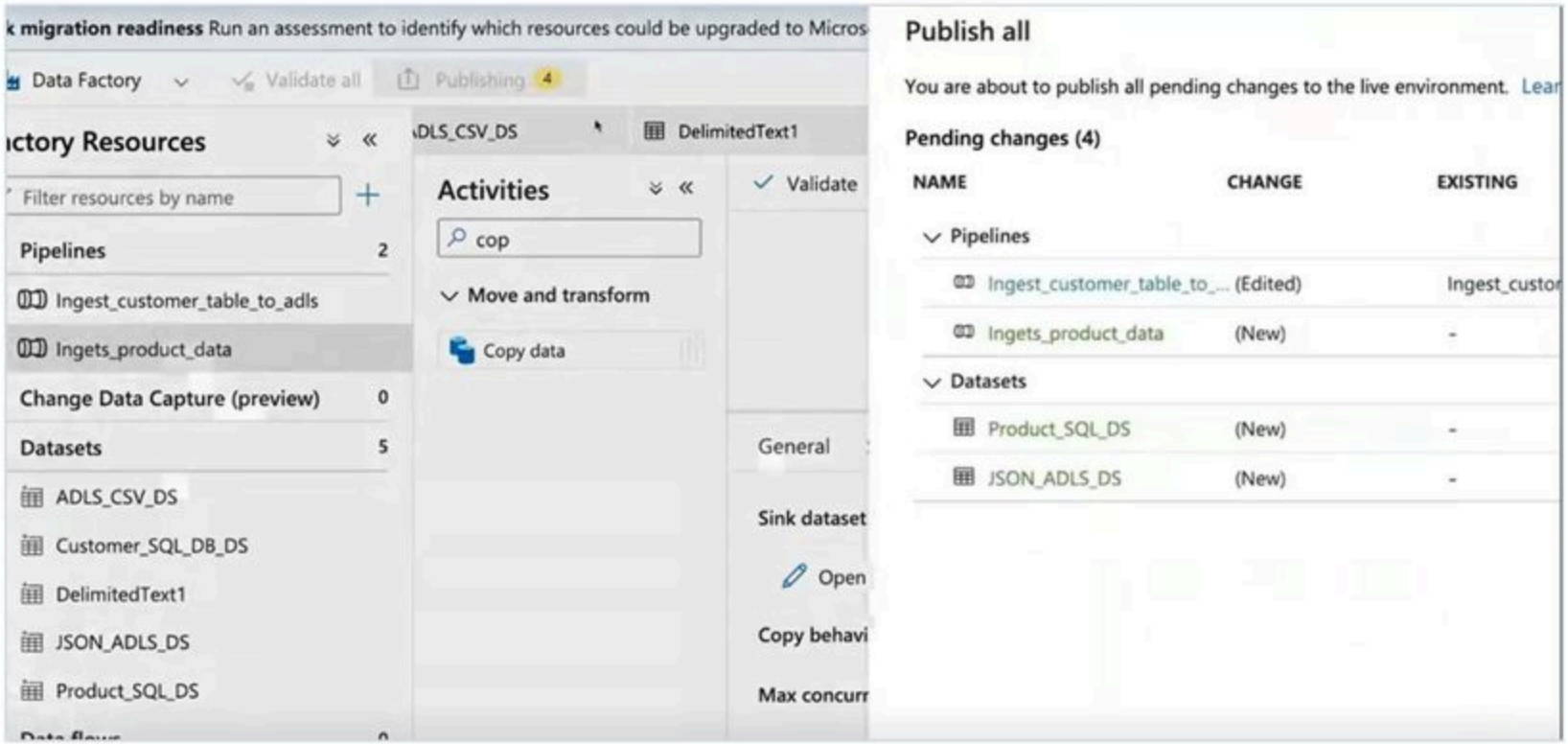


- SELECT COUNT(*) FROM [SalesLT].[Product] → 295 ✓
- landing/SalesLT.Product.json

- One JSON object per line (JSON Lines / set of objects)

PART 6

Publish all again



- Pending changes: new pipeline + datasets (New), edited pipeline (Edited)
- Publish → work is saved

PART 7

Wrap-up

Troubleshooting • Best practices • Q&A • Homework

PART 7

Troubleshooting cheat-sheet

Symptom	Likely cause	Fix
Name already taken	Global uniqueness (ADF, storage, SQL server)	Add initials / numbers
SQL deployment failed: invalid Password	Complexity rules not met	Use 3 of 4 character types, 8+ chars
Client IP not allowed (Query editor)	Firewall rule missing / IP changed	Allowlist IP or add client IP in Networking
Linked service test fails (SQL)	"Allow Azure services" = No, wrong password	Enable it; re-enter credentials
Linked service test fails (ADLS)	Wrong account, key rotated, network rules	Re-select account; check firewall/key
Dataset name rejected	Hyphens or spaces in name	Use letters, numbers, underscores
Work disappeared after closing browser	Not published (Live mode)	Always Publish all (or use Git)
Rows written ≠ source count	Query filter, wrong table, partial failure	Check source query and activity output

PART 7

Best practices from day one

- Naming convention for every object (LS / DS / PL prefixes or suffixes)
- Reuse linked services; one linked service per data store, per environment
- Never hard-code secrets: Azure Key Vault or managed identity
- Keep source, ADF and storage in the same region
- Test connection, Debug, then Publish; validate row counts every time
- Git integration + CI/CD for real projects (Dev → UAT → Prod)
- Use Serverless / free tiers and delete resources you do not need

PART 7

Frequently asked questions

Question	Short answer
Is ADF ETL or ELT?	Both. Copy = E + L; transform in ADF data flows or in Databricks/SQL (ELT).
Linked service vs dataset?	Linked service = how to connect; dataset = which data + format.
Why Debug and not Trigger now?	Debug tests the draft; Trigger runs the published version.
Can one pipeline copy many tables?	Yes – parameters + Lookup + ForEach (coming in later sessions).
On-prem source?	Install a Self-hosted Integration Runtime.
How is ADF billed?	Per activity run, data-movement DIU-hours, and data flow cluster time.
ADF vs Fabric Data Factory?	Same concepts; Fabric is SaaS inside Microsoft Fabric. ADF skills transfer.
CSV, JSON or Parquet in the lake?	Parquet/Delta for analytics; CSV/JSON for simple exchange.

Homework / practice

- Rebuild Lab 1 and Lab 2 from scratch without the recording
- Copy SalesLT.Address to ADLS as CSV into a folder named address/
- Use a Query in the source: only customers whose CompanyName starts with ‘A’
- Set the sink file name to customer.csv instead of the default name
- Bonus: try the Parquet format and compare the file size with CSV and JSON
- Write 5 lines explaining linked service, dataset, activity, pipeline, IR

PART 7

Next session preview

- Parameterised linked services and datasets (one dataset, many tables)
- Lookup, ForEach and control-flow activities
- Triggers: schedule and event-based
- Monitoring and alerts

Thank you

Questions? • Publish your work • Practise Lab 1 and Lab 2